



DA2005: Programmeringsteknik

Lars Arvestad, Christian Helanow, Anders Mörtberg, Kristoffer Sahlin

15 juni 2026

Innehåll

1	Kom igång med Python	5
1.1	Programmering och Python	5
1.2	Stil i kompendiet	6
1.3	Variabler	6
1.4	Stränghantering och print	8
1.5	Aritmetiska operationer	9
1.6	Input	10
1.7	Speciella ord och namn i Python	10
1.8	Uppgifter	11
2	Grunder i Python-programmering	13
2.1	Syntax och semantik	13
2.2	Datatyper	14
2.3	Booleska uttryck i Python	16
2.4	Tilldelningsoperatorer	18
2.5	Funktioner	18
2.6	Globala och lokala variabler	23
2.7	Villkorssatser	25
2.8	Uppgifter	27
3	Listor och iteration	29
3.1	Listor	29
3.2	for loopar	34
3.3	while loopar	39
3.4	Uppgifter	42
4	Uppslagstabeller, filhantering och mer om loopar	44
4.1	Uppslagstabeller	44
4.2	Filhantering	48
4.3	Mer om loopar: nästling	51
4.4	Algoritmiskt tänkande	52
4.5	Representation av data	55
4.6	Uppgifter	58
5	Felhantering, särfall, listomfattning, och felsökning	60
5.1	Felhantering och särfall	60
5.2	Listomfattningar	68

5.3	Felsökning	70
5.4	Uppgifter	78
6	Sekvenser och generatorer	80
6.1	Sekvenser	80
6.2	Generatorer	84
6.3	Uppgifter	87
7	Moduler, bibliotek och programstruktur	91
7.1	Moduler	91
7.2	Populära bibliotek och moduler	94
7.3	Tumregler för programstruktur och bra kod	97
7.4	Uppgifter	103
8	Funktionell programmering	106
8.1	Rekursion	106
8.2	Anonyma och högre ordningens funktioner	115
8.3	Uppgifter	119
9	Objektorientering 1: Klasser	121
9.1	Objektorientering	121
9.2	Modularitet genom objektorientering	127
9.3	Ett till exempel på objektorientering: talmängder	128
9.4	Uppgifter	131
10	Objektorientering och arv	134
10.1	Klassdiagram	136
10.2	Substitutionsprincipen	136
10.3	Funktioner som ger information om klasser	137
10.4	Exempel: Geometriska figurer	138
10.5	Exempel: Monster Go	145
10.6	Uppgifter	149
11	Objektorientering 3: mer om arv	152
11.1	Olika typer av arv	152
11.2	Exempel: Algebraiska uttryck	155
11.3	Exempel: Binära träd	159
11.4	Uppgifter	161
12	Defensiv programmering	162
12.1	Assert	162
12.2	Testning	166
12.3	Uppgifter	171
A	Exempellösningar	173
A.1	Kapitel 1	173
A.2	Kapitel 2	174
A.3	Kapitel 3	176
A.4	Kapitel 4	180
A.5	Kapitel 5	183

A.6	Kapitel 6	185
A.7	Kapitel 7	188
A.8	Kapitel 8	191
A.9	Kapitel 9	194
A.10	Kapitel 10	197
A.11	Kapitel 11	201
A.12	Kapitel 12	206

Förord

Det är många som hjälpt oss med att påpeka misstag och förbättra texten. Särskilda insatser har gjorts av

- Rikard Erngren
- Samuel Lundqvist
- Evan Cavallo, som översatt till engelska!
- Lars Lidvall

Tack alla!

Kapitel 1

Kom igång med Python

1.1 Programmering och Python

Detta kompendium handlar om *programmering*, det vill säga att skriva program som en dator sedan kan köra för att lösa olika typer av problem. I kompendiet använder vi programmeringsspråket Python, men många av de underliggande idéerna går att överföra till andra språk (t.ex. Java, JavaScript, C/C++, Ruby, Scala, Haskell, OCaml). Python är ett väldigt populärt programmeringsspråk (testa att Googla “*most popular programming languages*”) som kan användas att skriva mjukvara till i princip alla sorters tillämpningar. Det används på många av de största IT-företagen, som Instagram, Google, YouTube, Spotify, och Amazon.

Vi kommer att använda version 3.8, eller senare, av Python. Vi rekommenderar även en installation av *Anaconda*, ett verktyg för att hantera paket och omgivningar för Python. Installationsinstruktioner finns på kurshemsidan samt genom en videotutorial som ligger på YouTube.

En *programmeringseditor* stödjer “syntax highlighting” och operationer som gör det lättare att programmera. Vi kommer att använda oss av *Spyder* som följer med installationen av *Anaconda* i denna kurs.

Exempel på andra programmeringseditorer:

- MS Visual Studio Code — gratis och idag mycket spridd. Du har växtmån.
- PyCharm — stark integration med Python
- För entusiaster (kan vara svåra att komma igång med):
 - Emacs
 - Vim

1.1.1 Att programmera i Python

När man programmerar i Python skriver man kod i en `.py`-fil. Denna kod laddas sedan i Python-tolken (eng. *interpreter*).

Startar man Spyder-applikationen inifrån Anaconda Navigator får man ett fönster med tre paneler: ‘editor’, ‘help’, och ‘console’. I kompendiet är kod som är skriven i editorn inramad med grå bakgrund. Vi kan till exempel skriva följande tre rader i Spyderns editor:

```
print(17)
print(1.1)
print('DA2005')
```

Om vi trycker på den gröna triangeln uppe till vänster eller tangentbordskommando F5 tolkas denna kod i Python-tolken och skrivs ut i konsolen, rutan i Spyder (och liknande program) där man kan evaluera Python-uttryck. Om man inte sparat filen så dyker det först upp en ruta med “Save file”. Spara filen på valfritt ställe på hårddisken, men kom ihåg att ha bra struktur så att du hittar filerna senare. Ett smidigt sätt är att skapa mappen “DA2005” och ha alla lämpliga undermappar (t.ex. “labb 1”) där man sparar filerna för varje del av kursen.

Vi döper filen till `f1.py` och när vi kör den får vi följande i konsolen på macOS (det ser lite annorlunda ut på Linux och Windows):

```
runfile('/Users/kxs624/kurser/DA2005/f1.py',
        wdir='/Users/kxs624/kurser/DA2005/')
17
1.1
DA2005
```

Obs: i kompendiet är kod och resultat som skrivs ut i konsolen inramad med turkos bakgrund.

1.2 Stil i kompendiet

1.2.1 Kodrepresentation

Vi kommer att beskriva programmering med svenska termer, men när man programmerar är det bra att använda engelska för variabelnamn, kommentarer, dokumentation, och utskrifter. Att använda engelska är vedertagen praxis då man ofta jobbar i internationella miljöer.

När det är underförstått att vi skriver kod i texteditorn som vi vill exekvera, “köra”, och skriva ut i konsolen kommer vi visa den grå och turkosa miljön efter varandra på följande sätt:

```
print("welcome to DA2005")
print(2*2)
```

```
welcome to DA2005
4
```

1.2.2 Uppgifter

När det gäller uppgifter i kompendiet kommer vi att markera uppgifter som vi anser vara svårare med symbolen † först i uppgiftstexten.

1.3 Variabler

En av grundstenarna i programmering är *variabler*. En vanlig intuitiv modell för variabler är att se dem som lådor med namn som man kan stoppa in värden i.

Här är ett exempel där vi räknar ut medelhastigheten för en resa över 67 km på 1,25 timmar.

```
s = 67
t = 1.25
v = s / t
print(v)
```

Vi *tilldelar* värdet 67 till variabeln *s* och värdet 1,5 till variabeln *t*. Lägg märke till att decimalkomma måste skrivas på engelskt sätt, med en punkt. Därefter tilldelas variabeln *v* resultatet av beräkningen s / t . I Spyder, VS Code, eller motsvarande brukar man kunna se att variabeln *v* nu har värdet 53,6, men instruktionen `print(v)` gör också att värdet skrivs ut.

Ordningen på instruktioner är viktig, så beräkningen kan inte göras innan variablerna till höger om `=` är definierade.

Exemplet ovan kan skrivas tydligare! Variablerna vi använde kanske fungerar bra för den som är van vid att just *s*, *t* och *v* används för sträcka, tid, respektive hastighet, men i de flesta sammanhang kan man inte hoppas på igenkänning av det slaget.

I allmänhet bör man välja variabler som är förklarande, eller till och med 'självdokumenterande'. Det vill säga, man ska inte behöva förklara vad som menas med en variabel utan det ska framgå ändå. Om vi skriver om exemplet ovan med ambitionen att långt senare kunna återkomma till koden och genast förstå vad som menas, så kan det se ut så här:

```
distance = 67
time_used = 1.25
average_velocity = distance / time_used
print(average_velocity)
```

De långa identifierarna kan upplevas som jobbiga att skriva, men erfarenheten är att det i längden lönar sig att skriva kod som är tydlig (eller till och med övertydlig). Om man har valt sina identifierare rätt så blir det ofta lätt att förstå koden.

1.3.1 Kommentarer

Om man vill förtydliga vad en variabel betyder, eller vad som är tänkt att ske i ett komplicerat kodavsnitt, så kan man lägga till kommentarer:

```
distance = 67      # Estimated from map
time_used = 1.25   # Measured by GPS
average_velocity = distance / time_used
print(average_velocity)
```

Tecknet `#` markerar att 'här börjar en kommentar' och allt fram till nästa rad ignoreras helt av Python-tolken. En kommentar kan börja var som helst på en rad.

1.3.2 Fler värden

Vi kan arbeta med olika sorters värden.

```
x = 17
y = 'DA2005'
z = 22/7
q = 22//7      # Operatörn // är för heltalsdivision
```



```
print(x)
print(y)
print(z)
print(q)
print(x, y, z, q)
```

```
17
'DA2005'
3.142857142857143
3
17 DA2005 3.142857142857143 3
```

I exemplet ovan har vi använt följande *typer* av värden:

- x och q är heltal (eng. `int`),
- y är en sträng (eng. `string`), vilket betyder en serie bokstäver,
- z tilldelades kvoten av två heltal och blir då ett *flyttal* (eng. `float`), som är ett sätt att approximera reella tal.

Python har fler primitiva typer vilka vi kommer återkomma till.

Man kan även tilldela flera variabler på en gång:

```
x, y = 2, 3
print(x)
print(y)
```

```
2
3
```

Här används alltså kommatecken för att separera variabler och värden.

Notera att om x och y redan har definierats ovan i koden (t.ex. som 17 och 'DA2005' tidigare) så kommer de att skrivas över med värdena 2 och 3.

1.4 Stränghantering och print

Strängar kan skrivas inom antingen enkelfnuttar ' eller dubbelfnuttar " (jo, programmerare brukar kalla tecknen så istället för apostrof och citattecken!). Man kan på så sätt enkelt blanda ' och ":

```
print("hej")
print('hej')
```

```
hej
hej
```

Eftersom fnuttar kan betyda "start på sträng" och "slut på sträng" kan man inte stoppa in dem hur som helst i strängar, och det är därför som man valt att använda två varianter i Python. Se följande exempel:

```
print("Sa du 'hej'? ")
print('Ja, jag sa "hej".')
```

```
Sa du 'hej'?
Ja, jag sa "hej".
```

Det är rekommenderat att man väljer antingen enkelfnuttar eller dubbelfnuttar som sin standard, och bara använder den andra när det behövs. Så om du valt att använda dubbelfnuttar som din standard så använder du bara enkelfnuttar ifall du ska skriva en sträng som har dubbelfnuttar i sig.

Radbrytningar skrivs med \n:

```
print("Hipp\nhipp\nhurra!")
```

```
Hipp
hipp
hurra!
```

Vill man skriva ut flera värden med mellanslag mellan ger man flera argument till print:

```
print('hej', 'på', 'dig')
```

```
hej på dig
```

1.5 Aritmetiska operationer

Python stödjer de normala räknesätten:

- addition (+)
- subtraktion (-)
- multiplikation (*)
- division (/)
- heltalsdivision (//)
- exponentiering (**)
- modulus/rest (%)

Prova att *evaluera* följande uttryck:

```
2+3 # ger 5
2-3 # ger -1
2*3 # ger 6
2/3 # ger 0.6666666666666666
2//3 # ger 0
2**3 # ger 8
2%3 # ger 2
```

Dessa fungerar både för heltal och för flyttal. Man kan även använda + på strängar för att sammanfoga dem:

```
'hej' + 'du' # ger 'hejdu'
```

Det läggs dock inte in något mellanslag mellan orden, så det måste man lägga in själv:

```
'hej' + ' ' + 'du' # ger 'hej du'
```

De olika räknesätten binder på förväntat sätt, det vill säga, de vedertagna räknereglerna gäller när man evaluerar aritmetiska uttryck och man behöver inte använda parenteser i onödan. Exempelvis är $2 * 3 + 4$ detsamma som $(2 * 3) + 4$:

```
2 * 3 + 4    # ger 10
(2 * 3) + 4  # ger 10
2 * (3 + 4)  # ger 14
```

I vissa fall kan man använda dem med olika typer av indata på samma gång:

```
2.0 ** 3 - 2.1312    # ger 5.8688
2 * 'hej'            # ger 'hejhej'
```

Men man kan till exempel inte addera en siffra till en sträng eller multiplicera två strängar:

```
'hej' * 'hopp'
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can't multiply sequence by non-int of type 'str'
```

En annan otillåten operation:

```
2 + 'hej'
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Här ser vi exempel på olika felmeddelanden som kan uppstå om man skriver något gale. Felmeddelanden kan ofta vara svårtolkade till en början, men man behöver inte förstå hela felmeddelandet för att få information från det. Om vi bara tittar på sista raden i meddelandet ovan kan vi förstå att felet var att vi hade fel typer på operanderna till operatoren '+', den kan inte hantera kombinationen 'int' och 'str'.

1.6 Input

Vi har sett att print kan användas för att skriva ut värden på skärmen. Funktionen input kan användas för att läsa in data från en användare. Om man ger en sträng som parameter så ges den som "prompt", det vill säga text visar att nu ska man skriva något. Ett exempel:

```
namn = input("Vad heter du?\n")
print("Hej " + namn)
```

Detta lilla program ger i konsolen

```
Vad heter du?
Hej Johan
```

Den första raden skrivs ut av Python, och den andra av den som sitter vid datorn. När man trycker på retur-tangenten returnerar input en sträng med det som användaren skrivit.

1.7 Speciella ord och namn i Python

1.7.1 Reserverade ord

Det finns ett antal *reserverade ord* i Python, alltså ord som tillhör programmeringsspråket:

```
and except lambda with as finally nonlocal while assert False None
yield break for not class from or continue global pass def if raise
del import return elif in True else is try
```

Om man försöker använda ett reserverat ord som variabelnamn får man ett felmeddelande:

```
and = 12
```

File "/Users/kxs624/.spyder-py3/DA2005/ht20/f1.py", line 1

```
and = 12
^
```

SyntaxError: invalid syntax

Det finns annars ganska få krav på hur namnen kan väljas. Det är dock bra att undvika namn på funktioner som redan finns definierade i python. Det finns exempelvis ingen teknisk begränsning som hindrar dig från att skriva `input = "Hejsan"`, men om du senare vill kalla på funktionen `input` så kommer den inte längre vara tillgänglig.

1.7.2 Identifierare

Identifierare är namn på resurser. Alla variabelnamn är exempel på identifierare, det vill säga, de håller en "resurs" eller ett värde. Det finns dock identifierare som inte är variabler. Till exempel kommer vi gå igenom funktioner, klasser och objekt senare i kursen. Dessa typer namnges med funktionsnamn, klassnamn och objektnamn som alla är identifierare.

Identifierare i Python:

- Kan bestå av bokstäver, siffror, och `_`.
- Får inte börja med siffra.

Python skiljer på versaler och gemener; `Year` och `year` är inte samma identifierare.

Det är viktigt att välja identifierare så att koden blir lätt att läsa. Att döpa en variabel som innehåller någons ålder till `age` är mycket mer informativt än till exempel bara `x`.

1.7.3 Litteraler

En litteral är ett konstant värde som man skriver i programkoden. Till exempel `17`, `2.5`, och `'SU'` är alla litteraler.

Undvik helst att ha litteraler spridda över programmet! Att använda en litteral i ett uttryck kallas ibland för att "hårdkoda" ett värde, och avser det faktum att värdet är fixerat på den platsen i koden. Det är bättre att använda sig av en variabel som definierats på lämplig plats, kanske högst upp i programmet. En variabel är ofta lättare att förstå, givet att man har en lämplig identifierare, än en konstant som dyker upp i programmet. Dessutom så minskar risken för programmeringsfel. Om en konstant används på flera platser i ett program så kanske man inte kommer ihåg att ändra på alla platser om man har anledning att byta värde på konstanten. Om man använder en variabel, som definieras på en rad, så är det endast en plats som man antagligen behöver editera.

1.8 Uppgifter

1. Vad händer om vi kör nedanstående kod och varför blir det som det blir? Undersök!

```
x,y = 2,3
x,y = y,x
print(x)
print(y)
```

2. Nedan har vi kod som definierar variablerna x och y med tilldelade värden. Hitta på ett aritmetiskt uttryck i x och y som ger värdet

100.

```
x = 4
y = -3
```

3. Låt

```
s = 'hej'
```

Skriv kod som skriver ut 'hejhejhejhejhejhejhejhejhejhejhejhejhejhejhej' (det vill säga, hej sexton gånger på raken).

4. Låt

```
s = 'hej'
```

Skriv kod som skriver ut 'hejhejhejhejhejhejhejhejhejhejhejhejhejhejhej' genom att bara använda operatörn +, men så få gånger som möjligt.

5. Skriv kod som frågar användaren om dess förnamn och efternamn samt sparar resultatet i två variabler. Skriv sedan ut dessa. Ta inspiration från stycket 'Input' ovan (avsnitt 1.6). Till exempel ska det fungera så här:

```
Vad heter du i förnamn?    # Skriv in ditt förnamn här, tex 'Johan'
Vad heter du i efternamn?  # Skriv in ditt efternamn här, tex 'Jansson'
Hej Johan Jansson
```

6. Skriv kod som frågar användaren om dess ålder (spara i variabeln age) samt favorittal (spara i variabeln num). Vad händer om du adderar dessa variabler? Blir det vad du förväntar dig? Vad drar du för slutsatser om age och num?
7. Gör beräkningen $70 / 1.5$ i Python. Vad är det som kan uppfattas som konstigt med svaret och vad beror det på?

Kapitel 2

Grunder i Python-programmering

2.1 Syntax och semantik

Inom programmering pratar man ofta om *syntax* och *semantik*:

- *Syntax* handlar om regler för *hur* man skriver.
- *Semantik* handlar om *betydelsen* av vad man skriver.

Som ett exempel kan vi tänka på följande mening på Engelska:

Colorless green ideas sleep furiously.

Mening är syntaktiskt korrekt, dvs den innehåller inget *grammatiskt* fel, men *innebörden* är inte något meningsfullt: semantiken är nonsens.

Syntax är speciellt viktigt, och kan vara svårt, när man börjar lära sig att programmera. Precis som i ett vanligt språk så handlar det om att lära sig reglerna för vad som är korrekt. Om man använder Python handlar det till exempel om vad som är en giltig identifierare, var och hur man måste indentera kod-block, hur man skriver if-satser och for-loopar (vi kommer att gå igenom dessa i senare kapitel), och så vidare. Syntax är specifikt för ett visst programmeringsspråk, även om det ofta finns många likheter mellan språk. Python har till exempel många likheter med språket C. Att lära sig syntaxen går relativt snabbt genom övning, och fokus flyttas sedan till semantiken: vad händer i programmet (eller vad *borde* hända i programmet)?

Om du ser någonting i stil med `SyntaxError` i ett felmeddelande när du exekverar Python-kod så betyder det att du har skrivit något litet fel och att Python inte kan förstå din kod. I dessa fall får man ofta ett ganska instruktivt felmeddelande, eftersom Python kan säga till om vad som går fel. Till exempel kan följande felmeddelande fås:

```
3a = 2
^
SyntaxError: invalid syntax
```

Felmeddelandet visar var den ogiltiga syntaxen förekommer: resultatet av ett ogiltigt namn för en identifierare.

Semantiska fel är svårare att hitta, eftersom Python inte säger till om något fel. Allt är giltig kod. Resultatet av semantiska fel är att programmet inte gör vad man förväntar sig. Detta kan undvikas genom att skriva

välorganiserad, väldokumenterad och välstrukturerad kod, dvs att skriva “bra” kod. Om det är tydligt vad man *vill* att koden ska göra när någon (eller man själv) läser koden, blir det mycket lättare att hitta semantiska fel.

```
x = 7
y = 5
x_minus_y = y - x
print("x minus y = ", x_minus_y)
```

Här har vi ett exempel på ett semantiskt fel. Koden kör från början till slut utan problem men resultat blir fel eftersom vi råkar beräkna $y - x$ istället för $x - y$ på rad 3. Med det tydliga variabelnamnet `x_minus_y` är det ganska lätt att hitta vart felet uppstod när man kollar igenom koden.

2.2 Datatyper

Alla värden i Python har en typ. Vi har redan använt oss av olika typer när vi till exempel tilldelat variabler värden som heltal, flyttal och strängar. De vanligaste typerna i Python som vi kommer att använda är:

- `str` – strängar, dvs bokstäver (eller generellt tecken) inom `'` eller `"`
- `int` – heltal, av obegränsad storlek
 - I de flesta programmeringsspråk är heltal begränsade till 32 bitar, vilket ger möjliga värden från -2^{32} till $2^{31} - 1$.
- `float` – flyttal, en approximation av reella tal
 - För approximering av reella tal i intervallet $[-10^{308}, 10^{308}]$, med 15 siffrors decimal precision.
 - En float kodas med 64 bitar, varav en bit ger tecken (negativt/positivt), 11 används för exponenten x och 52 för “mantissan” m för att ge talet $m2^x$.
 - Detaljer kan fås genom `import sys` och objektet `sys.float_info`
 - Varför är resultatet inte alltid exakt vad man förväntar sig? Jo, Python representerar flyttal genom bråk i bas 2. För detaljer se: <https://docs.python.org/3/tutorial/floatingpoint.html>
 - Generellt gäller att det inte går att representera *alla* reella tal *exakt* på en dator, vilken talrepresentation som än väljs. Vissa reella tal kan representeras exakt medan andra får blir en approximation: fler bitar resulterar i en bättre approximation.
- `complex` – skrivs `1+2j`
 - Representeras med två flyttal
- `bool` – Booleska värden (dvs ‘sant’ eller ‘falskt’), representeras av de reserverade orden `True` och `False`.
- `NoneType` – för värdet `None`

Man kan i Python se vilken typ en variabel eller literal har genom att använda sig av `type`:

```
my_int = 1
my_float = 1.0
my_string = "1"
print(type(my_int))
print(type(my_float))
print(type(my_string))
print(type('hello'))
```

```
int
float
```

```
str
str
```

2.2.1 Typomvandling

Man kan omvandla en typ till en annan, men det finns regler om hur specifika typer omvandlas och till vilka typer de kan omvandlas. Ofta behöver man inte ens tänka på att en typ omvandlas, utan det funkar ganska mycket som man rent intuitivt tänker sig. En sådan typomvandling kallas *implicit*. Till exempel är det sällan ett problem att blanda numeriska typer i ett uttryck:

```
my_int = 1
my_float = 1.0
c = my_int + my_float
print(type(c))
```

```
float
```

Här omvandlas alltså värdet i `my_int` till ett flyttal som sedan adderas till flyttalet som finns i `my_float`, och resulterar i ett flyttal `c`.

Som visats tidigare så finns det situationer där en sådan konvertering inte fungerar. Skulle man till exempel utvärdera uttrycket `my_int + my_string` skulle man få ett `TypeError`. Man kan då använda sig av *explicit konvertering*, vilket man gör genom att ge typen man vill omvandla ett värde till värdet som argument. Om vi skulle vilja typomvandla `x`, så skulle vi kunna göra:

```
str(x)
int(x)
float(x)
complex(x)
bool(x)
```

Vi kan nu använda oss av explicit konvertering för att göra:

```
c = my_int + int(my_string)
print(type(c))
```

```
int
```

I detta fall kan värdet i `my_string` omvandlas till en `int` för att sedan adderas till `my_int`.

Varning: explicit typomvandling funkar inte alltid “magiskt”:

```
int("hello!")
```

```
ValueError: invalid literal for int() with base 10: 'hello!'
```

Här vet inte Python hur `'hello!'` ska omvandlas till ett heltal.

Explicit konvertering är ofta viktigt när man använder `input()`, som vid lyckad inläsning ger typen `str`. I vissa fall vill man inte att ens variabel ska vara en `str`, som nedan:

```
age = int(input("Din ålder: "))
added_age = age + 10 # would not work if age was of type str
print(added_age)
```


2.3 Booleska uttryck i Python

En operator är en symbol som använd på en eller flera värden, operander, ger ett nytt värde. Vi har tidigare sett exempel på typiska aritmetiska operatorer (+, -, *, /, ...). Resultatets typ beror av operandernas typer. Booleska uttryck använder sig av operatorer som resulterar i att en bool returneras, dvs True eller False. Sådana uttryck är väldigt användbara när man ska utvärdera om något är sant eller falsk givet ett test.

Det är värt att påpeka att det *mesta* i Python anses vara *sant*. Vi kan undersöka detta genom att använda typomvandling till bool:

```
bool(3)
bool(-427.2)
bool('hello')
```

```
True
True
True
```

Värden som anses vara *falska* är typiskt “tomma” värden, vad de än har för typ:

```
bool(0)
bool(0.0)
bool('')
bool(None)
```

```
False
False
False
False
```

2.3.1 Jämförelseoperatorer

Dessa operationer jämför indata genom följande relationer:

- Likhet: `x == y`
- Olikhet: `x != y`
- Strikt större än: `x > y`
- Strikt mindre än: `x < y`
- Större än eller lika: `x >= y`
- Mindre än eller lika: `x <= y`

Dessa fungerar som man förväntat för siffror (experimentera!), men som många operatorer i Python är det inte *endast* int eller float man kan jämföra. Till exempel:

```
print('hello shark' < 'hello shark')
print('hello shark' < 'hello shork')
print('hello shark' < 'hello')
print('hello shark' < 'Hello shork')
```

```
False
True
False
False
```

Sök på internet om hur Python jämför strängar!

2.3.2 Logiska operatorer

Python använder sig av de logiska operatorerna and, or och not.

- Är x *och* y sant?: x and y
x and y är True endast om både x och y är True.
- Är x *eller* y sant?: x or y
x or y är True endast om någon av x och y är True. Notera att att x or y alltså blir True även när både x och y är sanna.
- Negera ett värde: not x
not x är True om x är False och False annars.

Detta kan skrivas med sanningsvärdestabeller. Vi kan till exempel använda oss av logiska operatorer för att definiera ett uttryck som är ett exklusiv-eller, xor, som utvärderas till sant om exakt en av x eller y är sann:

```
x = True
y = False
xor_result = (not ((x and y) or (not x and not y)))
print(xor_result)
```

True

Python behandlar and och or lite speciellt om (båda) operanderna inte är av typen bool: en and/or-sats evalueras då inte till bool. En and-sats ger tillbaka den *andra* operanden om båda operanderna är sanna, men ger tillbaka värdet av den *falska* operanden om uttrycket innehåller en sådan. En or-sats ger tillbaka det *första* värdet om första operanden är sann, annars värdet av andra operanden. Några exempel:

```
(42 and 'hello')
(42 and '')
(False and '')
(42.7 or 13)
('42' or None)
(False or 0)
```

```
'hello'
''
False
42.7
'42'
0
```

De här evalueringsreglerna kan förefalla konstiga, men har att göra med hur mycket av satsen som behöver utvärderas för att garanterat veta resultatet av hela uttrycket. Om första operanden till and är falsk, då behöver man inte beräkna värdet på det andra operanden; oavsett om den evaluerar till True eller False så ska resultatet motsvara False. Tekniken kallas ibland “kortslutning” (eng: *short circuiting*), eller mer generellt “lat evaluerings” (eftersom det kan ses som “lathet” att inte beräkna allting). Syftet är förstås att få snabbare program genom att inte beräkna uttryck som inte behövs.

2.4 Tilldelningsoperatorer

Vi har redan sett att `=` används för tilldelning. En smidig funktionalitet i Python är att man kan kombinera detta med aritmetiska operatorer och få så kallade "tilldelningsoperatorer":

```
x += 3    # x = x + 3
x -= 3    # x = x - 3
x *= 3    # x = x * 3
x /= 3    # x = x / 3
x %= 3    # x = x % 3
x //= 3   # x = x // 3
x **= 3   # x = x ** 3
```

2.5 Funktioner

I och med att ens program blir större och kanske mer komplicerat, så märker man snabbt att det finns ett behov att kunna strukturera upp sin kod. Det kan man börja göra med hjälp av *funktioner*. Vi har redan använt oss av flera av Pythons inbyggda funktioner, till exempel `print`. Funktioner inom programmering har, som namnet antyder, likheter med matematiska funktioner: de tar ett eller flera argument och ger tillbaka ett returvärde. Inom programmering *behöver* dock inte en funktion ta några argument eller lämna tillbaka ett värde.

En av fördelarna med att använda funktioner är att du kan återanvända delar av koden istället för att upprepa samma kod på flera ställen. Detta gör ofta koden mer lättläst. Det gör också koden lättare att uppdatera då du bara behöver ändra på en plats istället för att ändra på alla platser där den upprepade koden var.

Funktioner delar också upp ditt program i mindre delar. Fördelarna med denna uppdelning är bland annat:

- Det tydliggör beroenden i koden: "indata" som argument, resultat efter **return**. Det här påminner om en funktion i matematiken.
- Det ger naturliga stycken att dokumentera.
- Det ger naturliga stycken att testa för att försäkra sig att koden fungerar som den ska.

Ett program uppdelat i många mindre funktioner som fungerar fristående från varandra kallas *modulärt* medan ett program där all kod körs som ett stort stycke kallas *monolitiskt*. Monolitiska program kan vara lättare att skriva i stunden men är både svårare att underhålla och att läsa och förstå.

Funktioner är därför ett mycket viktigt verktyg för att skriva bra kod.

Innan man kan använda en funktion, så måste funktionen *definieras*. Det gör man genom att använda sig av det reserverade ordet **def** för att ge funktionen ett namn (identifierare) och specificera argument. Nedan är ett exempel på en enkel funktion `f` som tar ett värde `x` som argument och adderar värdet till sig själv:

```
def f(x):
    return x + x
```

Denna funktion skulle matematiskt kunna skrivas som $f(x) = x + x$. Notera att instruktionen **return** tydligt talar om att "här är resultatet". När man väl definierat funktionen, så kan den kallas på följande sätt:

```
f(2.0)
```

4.0

Lägg märke till att Python inte bryr sig om vilken typ argumentet har, så länge alla operationer och uttryck i funktionen går att genomföra. Ovan används ett flyttal, men vi kan också göra:

```
f(2)
f('hello')
```

```
4
'hellohello'
```

Precis som matematiska funktioner, så kan funktioner i Python ha flera argument:

```
def long_name(course_code, course_name):
    lname = course_code + course_name
    return lname
```

Vad är resultatet av `long_name('DA', '2005')`?

Funktioner behöver inte ta några argument

```
def show_user_menu():
    print('Menu:')
    print('1. Add user')
    print('2. List users')
    print('3. Quit')
```

och kan också anropa andra funktioner:

```
def user_choice():
    show_user_menu()
    return input('Which assignment do you want to do? ')
```

Notera att funktionsanrop alltid har parenteser efter funktionsnamnet, även när funktionen inte tar några argument.

2.5.1 Om funktioner

Exemplen ovan introducerar flera nya syntaktiska koncept i Python. En funktion har en definition (även kallad “kropp”) och ett huvud (även kallad “signatur”).

- Huvudet består av funktionsnamnet och en parameterlista (“funktionens argument”).
 - Syntax: `def funktionsnamn(argument1, argument2, ...)`:
- Ordet parameter har två betydelser.
 - Variabler i huvudet kallas *formella parametrar*. Formella parametrar är platshållare för de värden som kommer att ges vid ett funktionsanrop.
 - De värden man ger vid ett anrop till en funktion kallas *anropsparametrar* eller *argument*.
- Kroppen har en dokumentationssträng (frivillig) följt av kod, och är *indragen*.

Indragningen är viktig i Python! Python är ett “indenteringskänsligt” språk, dvs fel indrag ger syntaxfel. Vi använder här ordet *indentering* när vi menar indrag i källkoden. Indenteringen är det som specificerar det *kodblock* som hör till funktionskroppen. Även om man kan använda sig av vilken indentering man vill i Python (till exempel, två mellanslag eller en tab), så länge man är konsekvent, så är *standard-indenteringen* fyra mellanslag.

Att indenteringen ska vara fyra mellanslag specificeras i Pythons stilguide, känd som [PEP-8](#), som också innehåller mycket annan nyttig information om hur man skriver bra Python-kod.

En modern programmeringseditor sätter in fyra mellanslag när använder tab-tangenten. En liten varning dock för att äldre eller mer primitiva editorer kanske inte hanterar tab-tangenten på ett bra sätt. Det kan ge problem beroende på hur editorn är konfigurerad vilket gör att er kod kan se olika ut på olika datorer.

Funktionskroppen avslutas ofta med en **return**-instruktion, till exempel **return** x. Om **return** saknas så returneras värdet None. Det är bra att veta att **return** kan förekomma fler gånger i en funktion, men om en **return**-sats exekveras så avslutas alltid funktionen.

Det är viktigt att förstå skillnaden på funktioner som returnerar något och de som bara till exempel skriver ut något. Betrakta följande kodsnuitt:

```
def f1(x):  
    return x + x  
  
def f2(x):  
    print(x + x)
```

Skillnaden är att f1 *returnerar* värdet på x + x medan f2 bara *skriver ut* värdet. Om vi nu försöker skriva ut resultatet av f1(2) blir det som vi förväntat oss:

```
print(f1(2))
```

```
4
```

men om vi försöker skriva ut resultatet av f2(2) händer följande:

```
print(f2(2))
```

```
4
```

```
None
```

Anledningen är att 4 först skrivs ut av funktionen f2, men sen när vi skriver resultatet av f2 så skrivs None ut då det är vad f2 faktiskt returnerar! Det är väldigt viktigt att förstå den här skillnaden och när det står att ni ska skriva en funktion som *returnerar* något så ska **return** användas och inte **print**.

2.5.2 Multipla returvärden

Hur gör man om man vill returnera flera värden? Det finns flera dåliga sätt att lösa detta, men ett bra: multipla returvärden.

```
def integer_div(nominator, denominator):  
    q = nominator // denominator  
    r = nominator % denominator  
    return q, r
```

Denna funktion kan man sedan anropa så här:

```
quotient, remainder = integer_div(17, 10)
```

2.5.3 Standardvärden

Det är vanligt att man skriver funktioner med parametrar som nästan alltid sätts till samma värde. Python låter dig ange explicit sådana vanliga värden: dessa kallas *standardvärden* (eng.: *default values*) eller *skönsvärden*.

```
def f(x=1):  
    return x + x  
  
print(f())  
  
print(f(3))
```

```
2  
6
```

I första utskriften kan vi se att `f` har kallats med `x=1`, även om vi inte gett det som argument. När vi på nästa rad kallar `f(3)`, så sätts `x=3`.

Följande funktion testar om en punkt (x, y) ligger inom en cirkel med radie r . Standardvärdet `r=1` gör så att cirkeln är enhetscirkeln om man inte ger tredje argumentet.

```
def in_circle(x,y,r=1):  
    # Compute length of hypotenuse  
    h = (x ** 2 + y ** 2) ** 0.5  
  
    # Is the point within the radius?  
    return h <= r  
  
print(in_circle(1,1,2))  
print(in_circle(1,1))  
print(in_circle(0.5,0.5))
```

```
True  
False  
True
```

2.5.4 Nyckelordsparametrar

Python låter dig namnge parametrar även i anropen. Till exempel så ger `integer_div(denominator=10, nominator=17)` ger samma effekt som anropet `integer_div(17, 10)`.

Vad händer om vi gör:

```
integer_div(denominator=10, nominator=17)
```

Python kopplar identifierarna på de formella parametrarna till identifierarna på argumenten, så ovan resulterar i precis samma output.

Ett exempel på en funktion som vi använt som har fler nyckelordsparametrar är `print`. Den har följande nyckelordsparametrar och standardvärden:

- `sep=' '` — dvs ett mellanslag som separator
- `end='\n'` — “backslash-n” betyder radbrytning.

- `file=sys.stdout` — låt utdata gå till terminalen

Exempel:

```
print('hello', 'you', sep='\n')
```

```
hello
you
```

For att se mer information skriv `help(print)` i Spyder-konsolen (iPython).

2.5.5 Dokumentation av funktioner

Det finns två sätt att dokumentera funktioner i Python, *dokumentationssträngar* (eng: *docstrings*) och *kommentarer*. Dokumentationssträngar är den primära dokumentationsmetoden och består av en sträng överst i funktionskroppen. Kommentarer ges till stöd för framtida läsning av kod.

2.5.5.1 Dokumentationssträngar

En dokumentationssträng ska förklara hur en funktion används. En tumregel är att en annan programmerare ska förstå hur funktionen används utan att behöva studera dess kod.

Även om funktionen har ett beskrivande namn är det inte alltid tydligt vad den faktiskt gör. Om en funktion till exempel heter `compute_integral`, behöver dokumentationssträngen inte upprepa detta, men det kan vara värdefullt att beskriva vilken algoritm som används, vilka antaganden som görs om parametrarna och vad som gäller för returvärdet.

Här är ett exempel på en enkel enradig dokumentationssträng:

```
def square(a):
    """Returns argument a squared."""
    return a*a

print (square.__doc__)

help(square)
```

Av konvention använder man flerradssträngar med citattecken, dvs skriver `""" """`, oavsett om man använder en eller flera rader.

En lite mer ambitiös dokumentationssträng kan se ut så här:

```
def square(x):
    """Description of square function

    Parameters:
    x (int): input number

    Returns:
    int: Square of x
    """

    return x*x
```

```
print(square.__doc__) # This is how docstrings are "stored"

help(square) # Try this!
```

Det speciella med dokumentationssträngar är att de knyts till funktionen och blir automatiskt tillgänglig som 'officiell' dokumentation. Du kan se dokumentationen för all kod i Python-systemet, moduler som du installerar, och den kod du skriver själv med kommandot `help(identifier).{python}` (prova i Spyders console). Om du använder en bra kodeditor så blir dokumentationen tillgänglig som popup-rutor och annat. I Spyder gör Ctrl-i när markören står vid ett funktionsanrop att dess dokumentation visas.

Det finns ingen riktig standard för hur man ska skriva dokumentationssträngar. De är beskrivna så här i [PEP-257](#):

The docstring [...] should summarize its behavior and document its arguments, return value(s), side effects, exceptions raised, and restrictions on when it can be called (all if applicable). Optional arguments should be indicated. It should be documented whether keyword arguments are part of the interface.

Vissa projekt och företag väljer att specificera en slags grammatik för dokumentationen, men för din egen kod har du full frihet att välja struktur på dokumentationen.

2.5.5.2 Kommentarer dokumenterar implementation

I Python är kommentarer inte lika viktiga som i många andra programmeringsspråk. Språket är utformat för att likna "pseudokod", dvs algoritmiska beskrivningar, och många gånger (särskilt med väl valda identifierare) kan man gott säga att Python-kod är självförklarande. Naturligtvis kräver det att man har tänkt igenom hur koden presenteras, och det finns gott om tillfällen när det är bra att använda en kommentar för att förklara vissa designval eller konstiga kodrader.

Ett vanlig nybörjarfel är att man skriver kommentarer överallt, men man ska inte underskatta den som läser din kod. Det här är till exempel en dålig kommentar:

```
n += 1 # Add 1 to n
```

Den kommentaren är helt redundant och ger mer att läsa utan att tillföra något.

Se istället de här två kommentarerna. De förklarar varför man valt att begränsa antalet iterationer när man löser en ekvation $f(x) = 0$ numeriskt och varför man vågar dividera med x utan att verifiera att x inte är nära noll.

```
...
# 10 iterations provides sufficient precision when solving f(x) = 0
x = newton_raphson(f, start_value, maxiter=10)
return magic_value / x # We know that f(x) > 1, so safe to divide
```

2.6 Globala och lokala variabler

Variabler i Python kan vara antingen *globala* eller *lokala*. Namnet syftar till var en variabel är "synlig" för kod, och var variabeln kan modifieras/ändras. I följande exempel är x en global variabel då den har definierats på toppnivå (d.v.s. "längst ut").

```
x = "My global string"
```



```
def foo():
    print("x printed from inside foo:", x)

foo()
print("x printed from outside foo:", x)
```

Funktionen kan inte modifiera x, utan värdet som x innehåller är bara läsbart:

```
x = 42

def foo():
    x = x * 2
    print("x printed from inside foo:", x)

foo()
```

UnboundLocalError: local variable 'x' referenced before assignment

För att kunna ändra på värdet i x, så måste vi säga åt funktionen att den kan ändra på den globala variabeln. Det här gör man med det reserverade ordet **global**.

```
x = 42

def foo():
    global x
    x = x * 2
    print(x)

foo()
```

Globala variabler som ändras av funktioner på detta sätt kan lätt leda till mycket röriga program, det man ibland kallar "spagettikod". Det blir svårt att hålla reda på var värdet på x ändras, så var försiktig om du använder globala variabler på detta sätt. Det finns scenarion när globala variabler är användbara, exempelvis globala räknare. Det är då extra viktigt att ha ett informativt namn på den globala variabeln så att man inte råkar ändra den av misstag!

Lokala variabler är variabler som definieras inuti i en funktion:

```
def foo():
    y = "My local string"
    print(y)

foo()
```

Variabeln y syns inte utanför funktionen (det är därför det kallas för *lokala* variabler):

```
def foo():
    y = "local"

foo()
print(y)
```

NameError: name 'y' is not defined

Var x är synlig är dess *räckvidd* (eng. *scope*). Om x är definierat utanför funktionen och vi sedan skriver

över den i funktionen så är dess värde oförändrat utanför funktionen:

```
x = 5

def foo():
    x = 10
    print("Local x:", x)

foo()
print("Global x:", x)
```

Samma sak händer med funktionsargument:

```
x = 5

def foo(x):
    print("Local x:", x)

foo(12)
print("Global x:", x)
```

2.7 Villkorssatser

Det händer ofta att man vill exekvera ett visst kodblock endast om något uttryck eller något test är sant. Detta görs med så kallade *villkorssatser*, eller "if-satser". I Python görs detta med **if**, **elif** (som står för *else if*) och **else**.

```
answer = input('Write "done" if you are done: ')
if answer == 'done':
    quit()
print('Okay, I guess you were not finished yet.')
```

Obs: lägg märke till dubbla likhetstecken! Det är skillnad på *jämförelse* och *tilldelning* (dvs == och =).

Exemplet ovan kan även skrivas:

```
answer = input('Write "done" if you are done: ')
if answer == 'done':
    quit()
else:
    print('Okay, I guess you were not finished yet.')
```

Notera att print-satsen är indragen lika långt som quit. Precis som för funktioner så gäller det att det indenterade kodblocket exekveras.

Indenteringsfel är svåra att se ibland:

```
answer = input('Write "done" if you are done: ')
if answer == 'done':
    quit()
else:
    print('Okay, I guess you were not...')
    print('.. quite finished yet.') # indentation error
```

Ett till exempel med **else**:

```
namn = input('What is your name? ')
if namn == 'Anders':
    print('Hello Anders!')
else:
    print('Hello, whoever you are!')
```

Man kan använda sig av *nästlade* villkorssatser, dvs en villkorssats inuti en villkorssats:

```
temp = int(input('What is the temperature? '))
if temp < -30:
    print('That is very cold!')
else:
    if temp < 0:
        print('That is pretty cold...')
    else:
        if temp == 0:
            print('Zero')
        else:
            if temp < 30:
                print('That is a comfortable temperature')
            else:
                print('Wow! Super warm')
```

I ovan fall är koden inte särskilt lättläst. Ett bättre alternativ är att använda **elif**:

```
temp = int(input('What is the temperature? '))
if temp < -30:
    print('That is very cold!')
elif temp < 0:
    print('That is pretty cold...')
elif temp == 0:
    print('Zero')
elif temp < 30:
    print('That is a comfortable temperature')
else:
    print('Wow! Super warm')
```

Läsbarhet är viktigt! Det reserverade ordet **elif** är strikt talat ej nödvändigt då det alltid kan definieras med nästlade **if-else**, men koden blir lättare att läsa. Denna typ av funktionalitet kallas ibland *syntaktiskt socker*.

Vad är skillnaden på beteendet för

```
x = int(input('Guess a number! '))
if x == 42:
    print('Correct!')
if x > 42:
    print('Too high...')
else:
```

```
print('Too low...')
```

och

```
x = int(input('Guess a number! '))
if x == 42:
    print('Correct!')
elif x > 42:
    print('Too high...')
else:
    print('Too low...')
```

Vad händer om man ger 42 som svar till input?

Slutligen så kan vi använda oss av villkorssatser för att skriva om xor-uttrycket från [Logiska operatorer](#) och definiera en funktion:

```
def xor(x, y):
    if x and y:
        return False
    elif not x and not y:
        return False
    else:
        return True

print(xor(True, True))
print(xor(True, False))
print(xor(False, True))
print(xor(False, False))
```

```
False
True
True
False
```

En kortare version kan med det logiska uttryck vi skrev tidigare:

```
def xor(x, y):
    return not ((x and y) or (not x and not y))
```

Vilken version av funktionen xor är lättast att läsa?

2.8 Uppgifter

1. Vilket eller vilka av alternativen nedan ska man tänka på när man väljer ett namn på en funktion eller variabel?
 - a) Namnet ska vara kort
 - b) Namnet ska vara beskrivande
 - c) Namnet ska vara unikt i funktionen och/eller skriptet
 - d) Namnet ska vara skrivet med stora bokstäver
 - e) Inget av ovanstående

2. Skriv kod som läser in ett heltal från användare och skriver ut Odd om talet är udda och Even om talet är jämnt.
3. Gör sanningstabeller för de booleska operatorerna and och or (d.v.s. fyll i ? i tabellen nedan):

x	y	x and y	x or y
True	True	?	?
True	False	?	?
False	True	?	?
False	False	?	?

4. Försök, utan att köra koden, utvärdera dessa fyra uttryck för alla möjliga booleska värden på x, y och z:

```
(not x) or (not y)
x and (y or z)
(x != z) and not y
x and z
```

Kör sedan koden för att bekräfta dina svar.

5. Skriv en Python-funktion som motsvarar den matematiska funktionen $f(x, y) = x/y$. Vi vill att funktionen ska säga till om man delar ett tal skilt från 0 med 0. I detta fall så ska funktionen skriva ut `The result is infinite...`, men *inte* returnera något explicit värde. Om man delar 0/0, så ska funktionen skriva ut `Indeterminate form...` och inte heller returnera något.

6. Låt:

```
x = True
y = False
```

Vilka uttryck nedan får värdet `True`? Varför?

```
(x and y) or (not x and not y)
(x or y) and (not x or not y)
x or y or not x or not y
not(not x and not (x and y))
```

7. Definiera nand, nor och xnor som Python-funktioner.

Tips: https://en.wikipedia.org/wiki/Logic_gate#Symbols.

8. Prova funktionerna nand, nor och xnor med argument som *inte* är av typen bool. Är resultaten som du väntade dig? Om inte, varför?

Kapitel 3

Listor och iteration

Det här kapitlet handlar om:

- Listor
- Iteration med **for** och **while** loopar

Listor är viktiga i programmering då de låter oss samla flera element tillsammans. Iteration handlar om att automatisera upprepning och är ett fundamentalt koncept i så kallad *imperativ* programmering. Två viktiga konstruktioner som vi ska prata om i det här kapitlet är **for** och **while** loopar. Listor och loopar är väldigt användbara tillsammans, exempelvis kan man skriva ett program som läser in tal från användaren, sparar dem i en lista och beräknar medelvärde av de inmatade talen.

3.1 Listor

Python har stöd för listor av element med olika typ. Dessa skrivs inom hakparenteser (“[” och “]”) och elementen i listan separeras med komma:

```
mylist = ['hej', 2, 'du']
```

Pythons lista kan jämföras med typen *array* i andra språk, men den är mindre beräkningseffektiv. Dock är Pythons listor mer lättanvända. Detta är typiskt för scriptspråk: det är enklare att använda datastrukturer, på bekostnad av effektivitet.

3.1.1 Listindexering

Man kan hämta ut elementen ur en lista genom `mylist[i]` där `i` är ett heltal:

```
print(mylist[0])
print(mylist[1])
print(mylist[2])
```

```
hej
2
du
```

Viktigt: listor börjar indexeras från 0!

Man kan även använda negativa värden för att hämta ut element från slutet av listan.

```
print(mylist[-1])
print(mylist[-2])
print(mylist[-3])
```

```
du
2
hej
```

Om man använder ett index som är för stort får man ett felmeddelande:

```
print(mylist[3])
```

```
IndexError: list index out of range
```

Fråga: vad händer om man indexerar med ett för stort negativt värde (t.ex. `mylist[-4]`)?

Man kan lägga till element i slutet av en lista med `append`.

```
mylist.append(True)
print(mylist)
```

```
['hej', 2, 'du', True]
```

Man kan enkelt tilldela ett nytt värde till ett element i en lista.

```
mylist[1] = 5
print(mylist)
```

```
['hej', 5, 'du', True]
```

Vi kan även göra detta genom att indexera från slutet av listan:

```
mylist[-2] *= 3
print(mylist)
```

```
['hej', 5, 'dududu', True]
```

Man kan hämta ut alla element med index $i < j$ genom `mylist[i:j]`. Detta kallas för *slicing*.

```
print(mylist[0:1])
print(mylist[0:2])
print(mylist[1:3])
print(mylist[1:4])
```

```
['hej']
['hej', 5]
[5, 'dududu']
[5, 'dududu', True]
```

Man kan ändra steglängden i slicingen till `k` genom `mylist[i:j:k]`:

```
print(mylist[0:4:2])
print(mylist[1:4:2])
```

```
['hej', 'dududu']  
[5, True]
```

3.1.2 Listfunktioner

Man kan addera listor och multiplicera dem med tal för att upprepa dem:

```
print(mylist + mylist)  
print(2 * mylist)
```

```
['hej', 5, 'dududu', True, 'hej', 5, 'dududu', True]  
['hej', 5, 'dududu', True, 'hej', 5, 'dududu', True]
```

Längden av en lista räknas ut med len:

```
print(len(mylist))  
print(len(5 * mylist + mylist))
```

```
4  
24
```

En väldigt användbar funktion är **in** som testar om ett element finns i en lista:

```
print(5 in mylist)  
print(False in mylist)
```

```
True  
False
```

Jämför med $x \in S$ från matematiken som säger att x finns i mängden S . Funktionen **in** fungerar även på strängar:

```
print('gg' in 'eggs')
```

```
True
```

Det finns även **not in** vilket motsvarar $x \notin S$ från matematiken:

```
print(5 not in mylist)  
print(False not in mylist)
```

```
False  
True
```

Man beräknar minsta och största element ur en lista med min och max:

```
print(min([2,5,1,-2,100]))  
print(max([2,5,1,-2,100]))
```

```
-2  
100
```

Man kan ta reda på vilket index ett element finns på med hjälp av index:

```
print(mylist.index(True))
```


3

Försöker man hitta index för något som inte finns i listan får man ett felmeddelande:

```
print(mylist.index(42))
```

```
ValueError: 42 is not in list
```

Man kan räkna antalet gånger ett element finns med i en lista med count:

```
mylist.append(5)
print(mylist.count(5))
print(mylist.count(42))
```

2
0

Vi kan ta bort element på ett specifikt index eller en hel slice med **del**:

```
print(mylist)
del mylist[2]
print(mylist)
del mylist[0:2]
print(mylist)
```

```
['hej', 5, 'dududu', True, 5]
['hej', 5, True, 5]
[True, 5]
```

Detta är samma som `mylist[i:j] = []`. Man kan även använda **del** `mylist[i:j:k]` för att få en annan steglängd.

Vi kan även ta bort elementet på index i och returnera det med pop:

```
mylist = ['hej', 5, 'dududu', True, 5]
print(mylist)
x = mylist.pop(2)
print(mylist)
print(x)
```

```
['hej', 5, 'dududu', True, 5]
['hej', 5, True, 5]
dududu
```

Detta fungerar även med negativa värden (dvs, genom att man indexerar från slutet). Ger man inget argument till pop så är det samma som `pop(-1)`, dvs sista elementet tas bort.

```
x = mylist.pop()
print(mylist)
print(x)
```

```
['hej', 5, True]
5
```

Man kan stoppa in ett element x på ett givet index i i en lista med `insert(i, x)`:

```
mylist.insert(1, 'du')
print(mylist)
```

```
['hej', 'du', 5, True]
```

För att ta bort första elementet som är lika med något x använder man `remove(x)`:

```
mylist.append(5)
print(mylist)
mylist.remove(5)
print(mylist)
```

```
['hej', 'du', 5, True, 5]
['hej', 'du', True, 5]
```

Slutligen kan man använda `reverse` för att vända på en lista:

```
mylist.reverse()
print(mylist)
```

```
[5, True, 'du', 'hej']
```

3.1.2.1 Sammanfattning av användbara listfunktioner

- `mylist.append(x)`: lägg till x i slutet av mylist.
- `len(mylist)`: beräkna längden av mylist.
- `x in mylist`: testa om x finns i mylist (returnerar en bool).
- `x not in mylist`: testa om x inte finns i mylist (returnerar en bool).
- `min(mylist)` och `max(mylist)`: hämta ut minsta respektive största element i mylist.
- `mylist.index(x)`: räkna ut första index där x finns i mylist.
- `mylist.count(x)`: räkna hur många gånger x finns i mylist.
- `del mylist[i]`: ta bort elementet på index i ur mylist (utan att returnera det).
- `mylist.pop(i)`: ta bort elementet på index i ur mylist (och returnera det).
- `mylist.insert(i,x)`: sätt in x på index i i mylist.
- `mylist.remove(x)`: ta bort första förekomsten av x ur mylist.
- `mylist.reverse()`: vänd mylist baklänges.

För mer listfunktioner och detaljer om listor se: <https://docs.python.org/3/tutorial/datastructures.html#more-on-lists>

<https://docs.python.org/3/library/stdtypes.html#sequence-types-list-tuple-range>

<https://docs.python.org/3/library/stdtypes.html#lists>

3.1.3 Praktisk funktion för att skapa listor: range

Ett sätt att skapa heltalslistor är att använda `range`:

```
print(list(range(1,4)))
print(list(range(3)))      # samma som range(0,3)
print(list(range(4,2)))
print(list(range(0,10,2)))
print(list(range(5,0,-1))) # iterera baklänges
```

```
[1, 2, 3]
[0, 1, 2]
[]
[0, 2, 4, 6, 8]
[5, 4, 3, 2, 1]
```

Obs: vi måste konvertera resultatet till en lista med hjälp av `list(...)` då `range` inte ger en lista direkt, utan ett “löfte” om en lista. Dvs, `range` returnerar ett objekt som skapar en lista vartefter man behöver en. Detta beteende kallas för *lat evaluering* och är smidigt då man inte behöver skapa hela listan på en gång. Detta har fördelen då hela listan inte behöver sparas i minnet på datorn på en gång vilket kan bli långsamt för stora listor. Mer om detta senare.

Om man försöker skriva ut `range(0, 10)` utan att konvertera till en lista händer följande:

```
print(range(0, 10))
```

```
range(0, 10)
```

3.1.4 Listors egenskaper

Listor har många styrkor, bland annat:

1. Enkla och effektiva att iterera över (mer om detta i nästa stycken).
2. Enkla att utöka.
3. Omedelbar åtkomst om man har index.
4. Listelementen kan ha olika typ.

Egenskap 3 är speciell i Python. Andra språk, som t.ex. C och Java, skiljer på *array* och *lista*. En array är som en vektor och har en fix storlek. I dessa språk är listor istället en speciell datastruktur där elementen är “länkade” så att man bara har omedelbar tillgång till det första (och kanske sista) elementet. Det gör att man inte automatiskt får omedelbar tillgång till alla element. Men i Python har man alltså direkt åtkomst till alla element genom listindexering vilket gör att dom påminner lite om arrayer.

En svaghet med listor är att de är ineffektiva att *leta* i (dvs operatören `in` kan vara långsam på stora listor). En annan svaghet är att oväntade saker kan hända om man har en lista som innehåller element av olika typ och försöker hämta ut största/minsta värde (se uppgift 2).

3.2 for loopar

for används för att iterera över uppräknings av olika slag, framförallt listor. Dessa uppräknings kallas *sekvenser* i Python och även strängar är sekvenser (av tecken).

```
for x in [1, 2, 3]:
    print(x)
```

```
1
2
3
```

Obs: notera att `print` satsen är indragen! Allt som kommer efter **for** och som är indenterat (med lika många mellanslag) kommer att köras i loopen. Som vi såg i förra kapitlet är Python indenteringskänsligt, dvs det är viktigt att man indenterar kodblock lika mycket och detta gäller även för **for** loopar. Detta

skiljer sig från språk som C och Java där man istället grupperar kodblock inom måsvingar (dvs “{” och “}”) och rader avslutas med semikolon “;”. Skulle man ha indenterat något fel i Python får man felmeddelandet:

```
IndentationError: expected an indented block
```

Ett kodblock kan innehålla flera instruktioner:

```
s = 'A number: '  
  
for x in [0,1,2]:  
    print(x)  
    s += str(x)  
  
print(s)
```

```
0  
1  
2  
A number: 012
```

Vi kan även iterera över en lista som ligger i en variabel:

```
mylist = [112, 857, 'hej']  
  
for x in mylist:  
    print(x)
```

```
112  
857  
hej
```

Eftersom strängar också är sekvenser kan man lätt iterera över bokstäverna i en sträng:

```
for c in 'DA2005':  
    print(2*c)
```

```
DD  
AA  
22  
00  
00  
55
```

3.2.1 Funktion som returnerar konsonanterna i en sträng

Låt oss nu implementera en lite mer komplicerad funktion som returnerar en strängs konsonanter. Innan man börjar skriva koden bör man fundera hur funktionen ska fungera. Givet en sträng s så ska den iterera över alla bokstäver och testa om de är konsonanter. Om bokstaven var det så ska den läggas till en sträng med alla konsonanter vi hittat i s och annars ska inget hända. Detta kan implementeras på följande sätt:

```
def consonants(s):
    cs = 'BCDFGHJKLMNPQRSTVWXZbcdfghjklmnpqrstvwxyz'

    out = ''

    for c in s:
        if c in cs:
            out += c

    return out

print(consonants('Hey Anders!'))
```

Hndrs

Notera hur vi definierar en variabel `out` vilken används för att spara de konsonanter vi hittat i `s`. Detta är ett väldigt vanligt sätt att programmera på och något vi kommer se mer av i kursen.

3.2.2 Dela upp en sträng i ord: `split`

Låt oss skriva en funktion som tar en sträng och bryter upp den i smådelar när man hittar ett blanksteg som separator. Det vill säga vi ska skriva en funktion `split(s)` som tar in en sträng `s` och returnerar alla ord i strängen. Detta kan göras genom att man har två variabler, en med de ord vi hittat och en med alla bokstäver i ordet vi håller på att bygga upp. För varje bokstav i `s` testas om den är ett mellanslag eller inte. Om bokstaven är ett mellanslag så lägger vi det inlästa ordet i listan, annars så lägger vi till bokstaven till ordet och tittar på nästa bokstav.

```
def split(s):
    out = []
    term = ''

    for c in s:
        if c == ' ':
            out.append(term)
            term = ''
        else:
            term += c

    # add the final word unless it is empty
    if term != '':
        out.append(term)

    return out

print(split('Hey Anders! How are you?'))
```

['Hey', 'Anders!', 'How', 'are', 'you?']

Säg nu att vi vill generalisera detta till många olika möjliga separatorer, men att mellanslag ska vara standardvärdet, då skriver vi istället följande kod:

```
def split(s, sep = ' '):
    res = []
    term = ''
    for c in s:
        if c in sep:
            res.append(term)
            term = ''
        else:
            term += c

    if term != '':
        res.append(term)

    return res

print(split('Hey Anders! How are you?'))
print(split('Hey Anders! How are you?', '!'))
```

```
['Hey', 'Anders!', 'How', 'are', 'you?']
['Hey Anders', ' How are you?']
```

Vi kan nu alltså anropa funktionen med ett *eller* två argument.

Obs: det finns redan en variant på `split` inbyggd i Python!

3.2.3 Flödeskontroll: `pass`, `continue`, `break` och `return`

Man kan kontrollera och avsluta iteration med `pass`, `continue`, `break` och `return`.

Den enklaste av dessa är `pass` som inte gör någonting:

```
for i in range(4):
    if i == 1:
        pass
        print('passed 1')
    else:
        print(i)
```

```
0
passed 1
2
3
```

Obs: `pass` operationen kan verka oanvändbar då den inte gör någonting, men ett vanligt användningsområde är när man strukturerar upp sitt program men inte har skrivit klart allt. Man kan till exempel definiera en funktion där kroppen bara är `pass` eller ha en `if` sats där något fall inte är skrivet än. Detta gör att man kan testa all annan kod i programmet och sedan skriva klart de delar där man använt `pass` efter att man försäkrat sig om att allt annat fungerar.

En `continue` instruktion avslutar nuvarande iteration av loopen och går vidare till nästa:

```
for i in range(4):
    if i == 1:
        continue
    print('passed 1')
    else:
        print(i)
```

```
0
2
3
```

En **break** instruktion avslutar loopen helt:

```
for i in range(4):
    if i == 2:
        break
    else:
        print(i)
```

```
0
1
```

Slutligen, **return** används i en loop när man vill returnera något från en funktion. Till exempel:

```
def first_even_number(l):
    for number in l:
        if number % 2 == 0:
            return number

print(first_even_number([1,45,24,9]))
```

```
24
```

3.2.4 Pythons for är annorlunda

Det kan vara bra att vara medveten om att Pythons **for** loop är mer generell än i de flesta andra språk. Till exempel förväntas looperna gå över heltalsvärden i många äldre språk. Pythons konstruktion kan sägas vara modern: den bygger på att man konstruerar en datastruktur som stödjer iteration. Det är fullt möjligt, och inte så svårt, att skriva sina egna itererbara datatyper för speciella behov. Som vi sett går det att iterera över både listor och strängar, men vi kommer att se att även andra typer i Python stödjer det.

3.2.5 Ett vanligt problem med listor i Python: muterbarhet

Ett vanligt problem med listor i Python som vi kommer se flera gånger i kursen är att man kan råka ändra på listan som man itererar över utan att vilja det. Detta är en väldigt vanlig källa till buggar, både för nybörjare och mer erfarna programmerare.

Låt oss säga att vi vill köra följande program som är tänkt att lägga in de ord som är längre än 6 bokstäver längst fram i listan:

```
animals = ['cat', 'dog', 'tortoise', 'rabbit']

for x in animals:
```

```

if len(x) > 6:
    animals.insert(0, x)

print(animals)

```

Men kör man detta kommer Python bara loopa och inte komma ur loopen! För att avsluta loopen trycker man CTRL-C. Problemet är att vi ändrar på `animals` listan samtidigt som vi loopar över den, så **for** loopen kommer aldrig avslutas.

För att undvika detta bör man itererar över en *kopia* av listan. Vi kan göra detta genom att använda `.copy()`:

```

animals = ['cat', 'dog', 'tortoise', 'rabbit']

for x in animals.copy():
    if len(x) > 6:
        animals.insert(0, x)

print(animals)

['tortoise', 'cat', 'dog', 'tortoise', 'rabbit']

```

Här används `animals.copy()`, istället för bara `animals`, så att vi får en kopia av listan och undviker den oändliga loopen.

Detta problem med listor kan komma upp när man programmerar och inte är försiktig med att kopierar saker. Detta kan leda till väldigt mystiska buggar som den oändliga loopen ovan. Det som gör att Pythons listor beter sig så här är att de är *muterbara*. Vi kommer prata mer om muterbarhet senare i kursen, men i dagsläget behöver ni bara komma ihåg att man kan behöva använda `.copy()` på en listvariabel för att undvika att råka ändra på den när man inte vill det.

3.3 while loopar

Ibland finns det inte någon självklar struktur att iterera över, men man vill ändå repetera ett kodblock flera gånger. Då finns **while** loopen. Den fungerar så att man kör ett kodblock tills något booleskt test är uppfyllt. Om vi kör följande exempel:

```

i = 10

while i > 0:
    print(i)
    i -= 1

print('Done!')

```

```

10
9
8
7
6
5

```



```
4
3
2
1
Done!
```

Ett vanlig användningsområde för **while** loopar är att loopa tills man fått rätt indata från en användare. Programmet nedan kommer läsa in indata från användaren tills man skriver exit:

```
loop = True

while loop:
    print("Write 'exit' to quit")
    answer = input('Write something: ')
    if answer == 'exit':
        print('Goodbye!')
        loop = False
    else:
        print('Let us keep going!')
```

```
Write 'exit' to quit
Write something: hey
Let us keep going!
Write 'exit' to quit
Write something: ho
Let us keep going!
Write 'exit' to quit
Write something: exit
Goodbye!
```

3.3.1 while loopar över listor

Listor är speciella och kan användas som booleska uttryck. Den tomma listan motsvarar False och alla andra listor motsvarar True.

```
list = []

if list:
    print('Not empty!')
else:
    print('Empty!')

list2 = ['Hey']

if list2:
    print('Not empty!')
else:
    print('Empty!')
```

```
Empty!  
Not empty!
```

Denna egenhet är till för följande *idiom* där man loopar över en lista och plockar bort dess element:

```
mylist = ['Ho', 'ho', 'ho!']  
  
while mylist:  
    x = mylist.pop()  
    print(x)  
  
print(mylist)
```

```
ho!  
ho  
Ho  
[]
```

När loopen är klar är listan tom och man behöver alltså inte jämföra listan med []. Detta är alltså ett till exempel där Python har förändrat en lista i en loop.

3.3.2 Ta bort blanksteg i slutet av en sträng

Så här kan man ta bort blanksteg i slutet av en sträng med hjälp av en **while** loop och slicing:

```
def remove_trailing_space(s):  
    i = len(s)  
    while i > 0 and s[i-1] == ' ':  
        i -= 1  
    return s[0:i]  
  
print(remove_trailing_space('SU '))
```

```
SU
```

Funktionen fungerar på så sätt att man börjar med en räknare *i* som sätts till längden på listan. Sen räknar vi hur många tecken från slutet av listan som är blanksteg och när vi är klara returnerar vi en slice av de ursprungliga listan där vi tagit bort alla blanksteg i slutet.

3.3.3 Största gemensamma delare: GCD

Den *största gemensamma delaren* (eng.: *greatest common divisor*, GCD) av två tal är det största heltal som delar dem. Exempelvis är 3 GCD av 15 och 12.

Euklides algoritm för att beräkna GCD för två tal *a* och *b* lyder så här:

- Antag $a \geq b$,
- låt *r* vara resten av heltalsdivision $a // b$,
- om $r == 0$, då är $\text{GCD}(a, b) = b$,
- annars kan man använda att $\text{GCD}(a, b) = \text{GCD}(b, r)$.

Lägg märke till att det är en villkorad iteration, men ingen tydlig struktur att iterera över. Så vi bör använda en **while** loop för att implementera den:

```
def GCD(a,b):
    if b > a:
        print('a must be bigger than b in GCD(a,b)')
        return None

    r = a % b
    while r != 0:
        a = b
        b = r
        r = a % b
    return b

print('GCD: ' + str(GCD(15,12)) + '\n')
print('GCD: ' + str(GCD(12,15)))
```

GCD: 3

b must be bigger than a in GCD(a,b)
GCD: None

3.4 Uppgifter

1. Vad är resultatet av `[1,2] + ['a', 'b']`?
2. Vad händer om man använder `min/max` på listor med element av olika typ?
3. Skriv klart funktionen

```
def vowels(s):
    vs = 'AEIOUYaeiouy'
    pass
```

vilken tar in en sträng `s` och returnerar dess vokaler.

4. Modifiera funktionen i uppgift 3 så att den istället bara returnerar konsonanter utan att ändra på strängen `vs`. Skriv sen en funktion `vowels_or_consonants` med en extra parameter vilken bestämmer om vokaler eller konsonanter ska returneras. Parametern ska ha ett standardvärde så att vokaler returneras om användaren inte kallar på funktionen med två argument.
5. Vad händer om man inte har blocket med `if` term `!= ''` i funktionen `split(s)` i avsnitt avsnitt 3.2.2? Hitta på ett exempel med en sträng `s` där utdata blir fel om man inte har det. Vidare, hur beter sig vår funktion `split` för strängar med flera mellanslag i rad? Skriv sen en korrekt version av funktionen.
6. Testa följande kodsnuitt:

```
list = [1,2,3]
list2 = list
list2.reverse()
print(list)
```

Fungerar den som ni trodde? Varför inte? Hur kan man fixa det?

7. Skriv om koden

```
mylist = ['Ho', 'ho', 'ho!']

while mylist:
    x = mylist.pop()
    print(x)

print(mylist)
```

så att listan `mylist` inte är tom när man är klar.

Tips: använd `.copy()`. Hur kan man göra om man inte får använda `copy`?

8. Testa att använda **continue**, **break** och **pass** i **while** loopen nedan. Vad händer?

```
n = 10
while n > 0:
    n -= 1
    if n == 7:
        # test continue, break or pass here
    print(n)
```

9. Skriv en **while** loop som läser in tal från användaren och sparar dem i en lista. Detta ska göras tills användaren skriver in 0 och då ska loopen avslutas och listan skrivas ut.

10. Skriv funktionen

```
def naive_gcd(a,b):
    pass
```

som beräknar GCD av `a` och `b` genom att testa alla tal mellan 1 och `min(a,b)` för att se vilket som är det största talet som delar båda.

11. Skriv funktionen `portkod()` som läser in en siffra (1–9) i taget från användaren. Efter varje siffra är inläst ska funktionen testa om de fyra senaste siffrorna matchar med korrekt kod. Om korrekt kod ges ska funktionen skriva ut `Door unlocked!` och returnera `True`. Välj vad som är korrekt portkod själv, t.ex. 1337.

Tanketips: Vilken loop ska vi använda?

12. Om vi har `lst = [1, 2, 3]` så kan man stoppa in ett element mellan 1:an och 2:an med instruktionen `lst.insert(1, 'x')`. Resultatet är att listan blir `[1, 'x', 2, 3]`.

Ett annat men ekvivalent sätt är att skriva `lst[1:1] = ['x']`.

- Vad är det som händer i den instruktionen?
- Vilket sätt tycker du är tydligast?

Kapitel 4

Uppslagstabeller, filhantering och mer om loopar

En “datastruktur” är en typ för att samla data. Istället för att ha ett tal i en `float` eller en sträng i en `str` så håller en datastruktur reda på många olika data på ett logiskt sätt. Vi har redan sett ett antal sätt i kursen, som till exempel listor. I det här kapitlet ska vi titta på *uppslagstabeller* som har likheter med listor men kan indexeras med andra typer än heltal.

Vi vill också kunna spara data till en fil på datorn eller läsa in data från en fil. Vi ska i kapitlet även titta på hur detta kan göras. Slutligen kommer vi även titta lite mer på vad man kan göra med loopar i Python.

4.1 Uppslagstabeller

Uppslagstabeller (eng. *dictionaries*) är en datastruktur som blivit en arbetshäst för scriptspråk. De är enkla att använda och bra till mycket. Uppslagstabellen är som en lista, men man indexerar med *nycklar* (eng. *keys*) istället för med heltal. Man kan tänka på en uppslagstabell som en mängd av nyckel-värde par (eng. *key-value pairs*). I Python används typen `dict` för uppslagstabeller.

Det finns två typiska sätt i Python att skapa en uppslagstabell. Till exempel kan man skapa en uppslagstabell som har nycklarna 'DA2005' och 'MM2001' med respektive värden 7.5 och 30 genom:

```
hp1 = dict(DA2005=7.5, MM2001=30)
hp2 = {'DA2005': 7.5, 'MM2001': 30}
```

Båda sätten är identiska och `hp1 == hp2`. Lägg märke till att första sättet har samma syntax som att anropa en funktion med nyckelparametrarna DA2005 och MM2001, medan det andra sättet använder sig av en *sträng* för nyckeln och värdet separerat med ett kolon.

För att komma åt ett värde används `[]`-syntax med *nyckelvärdet*:

```
print('Number of ECTS points for DA2005:', hp1['DA2005'])
```

```
Number of ECTS points for DA2005: 7.5
```

Lägg märke till att själva nyckeln också kan vara en variabel. I både `hp1` och `hp2` är nycklarna strängar, dvs av typen `str`. Ett exempel som illustrerar denna poäng:

```

DA2005 = 33
MM2001 = 'hello'
hp3 = {DA2005: 7.5, MM2001: 30}
print('Number of ECTS points for DA2005:', hp3[DA2005])
print('Number of ECTS points for DA2005:', hp3[33])
print('Number of ECTS points for MM2001:', hp3[MM2001])
print('Number of ECTS points for MM2001:', hp3['hello'])

```

```

Number of ECTS points for DA2005: 7.5
Number of ECTS points for DA2005: 7.5
Number of ECTS points for MM2001: 30
Number of ECTS points for MM2001: 30

```

Här har antingen värdet av till exempel MM2001 använts som nyckel direkt, eller så har identifieraren använts. Om man skulle försöka använda sig av en *sträng* av värdet "MM2001" här, så får man ett fel:

```
print('Number of ECTS points for MM2001:', hp3['MM2001'])
```

```
KeyError: 'MM2001'
```

Det här har att göra med att alla objekt i Python som inte kan ändras kan användas som nycklar. Uppslagstabeller implementeras med en teknik som kallas hashtabeller och innebär att man konverterar nyckeln till ett heltal (det "hashas", vilket avser att man "hackar upp" och blandar bytes till ett heltal) som används för uppslagning. Om nyckel kan ändras, "muteras", då finns det risk att man inte hittar tillbaka till det man stoppat in i tabellen. Generellt gäller att *icke-muterbara* värden, till exempel strängar, tal, tupler etc., kan användas som nycklar. Det här utesluter muterbara typer som till exempel listor.

Använder man sig av dict() för att skapa en uppslagstabell, så tolkas nyckelparametrarna alltid som strängar.

För mer detaljerad dokumentation av uppslagstabeller i Python se:

- <https://docs.python.org/3/tutorial/datastructures.html#dictionaries>
- <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>

4.1.1 Viktiga egenskaper

- En uppslagning är billig (dvs snabb).
- Det är enkelt att ändra innehåll.
- Uppslagstabeller är enkla att iterera över (men du får inte ändra i en uppslagstabell under iteration).

4.1.2 Andra namn

- Hashtabell
- Hash map

I "traditionella" språk (C, C++, Java, m.fl.) har de inte samma särställning som i scriptspråken. Scriptspråken har gjort uppslagstabeller lättanvända.

4.1.3 Fler exempel

Tomma uppslagstabellen kan skrivas {}, detta är syntaktiskt socker för dict().

```
empty = {}  
print(empty)  
print(dict())
```

```
{}  
{}
```

Som vi noterat ovan kan man skapa uppslagstabeller på följande sätt:

```
test1 = {'hopp': 4127, 'foo': 4098, 'hej': 4139, 12: 'hej'}  
test2 = dict([('hej', 4139), ('hopp', 4127), ('foo', 4098)])
```

```
print(test1)  
print(test2)
```

```
{'hopp': 4127, 'foo': 4098, 'hej': 4139, 12: 'hej'}  
{'hej': 4139, 'hopp': 4127, 'foo': 4098}
```

Observera att nycklar och värden inte behöver vara av samma typ!

Man kan uppdatera uppslagstabeller på liknande sätt som listor, och lägga till nya nyckel-värde par genom [-] och ta bort med **del**:

```
hp = {}  
hp['DA2005'] = 7  
print(hp['DA2005'])
```

```
hp['DA2005'] += 0.5  
print(hp['DA2005'])
```

```
hp['MM2001'] = 30  
hp['UG1001'] = 1  
print(hp)
```

```
del hp['UG1001']  
print(hp)
```

```
7  
7.5  
{'DA2005': 7.5, 'MM2001': 30, 'UG1001': 1}  
{'DA2005': 7.5, 'MM2001': 30}
```

4.1.4 Iterera över uppslagstabeller

Vi kan använda **for** för att iterera över nycklarna:

```
for key in hp:  
    print(key)
```

```
DA2005  
MM2001
```

Vi kan även använda `.keys()`:

```
for key in hp.keys():  
    print(key)
```

```
DA2005  
MM2001
```

och `.values()`:

```
for val in hp.values():  
    print(val)
```

```
7.5  
30
```

Vi kan även extrahera alla par av nycklar och värden med `.items()`:

```
for key, val in hp.items():  
    print('Number of ECTS points for ' + key + ': ' + str(val))
```

```
Number of ECTS points for DA2005: 7.5  
Number of ECTS points for MM2001: 30
```

Alla dessa tre funktioner returnerar itererbara objekt med element som består av antingen nycklar, värden eller tupler med nyckel-värde-par.

4.1.5 Funktioner för uppslagstabeller

Vi kan även manipulera uppslagstabeller med liknande funktioner som för listor:

```
print(len(hp))  
hp['foo'] = 20.0  
print(len(hp))  
  
print('DA2005' in hp)  
print('20.0' in hp)
```

```
2  
3  
True  
False
```

En till användbar funktion är `.get()`, som tar två argument: nyckelvärdet och ett värde som returneras om nyckeln inte finns i uppslagstabellen. Om nyckeln finns, returneras värdet som hör till nyckeln.

```
print(hp.get('DA2005', 42)) # key exists  
print(hp.get('bar', 42)) # key does not exist
```

```
7.5  
42
```


4.2 Filhantering

Filer innehåller information som ligger på hårddisken, till exempel textfiler, bilder eller filmer, etc. Du kommer åt filer genom att interagera med operativsystemet. Detta kan göras direkt från Python.

4.2.1 Läsa från filer

För att till exempel läsa från en fil `data.txt` skaffar vi först ett “handtag” (eng. *handle*). Det gör man med `open()`, som tar ett filnamn som obligatoriskt argument.

```
h = open('data.txt')
```

Ibland lägger man till ett andra argument med strängen “r” som markerar att man tänker läsa (*read*) filen.

```
h = open('data.txt', 'r') # 'r' för "read"
```

För att läsa in hela filens innehåll till en sträng skriver vi sedan:

```
h.read() # whole file in string
```

eller en lista med strängar där varje list-element är en rad från filen:

```
h.readlines() # whole file as list of strings
```

Detta funkar inte bra i många fall med stora filer. Det blir problem om man läser in en fil som är större än datorns RAM-minne.

För att bara läsa in en rad från `h` använder vi:

```
h.readline()
```

Notera att innehållet i en fil läses *en gång*, så har vi redan använt oss av `read` eller `readlines` så kommer `readline` att inte göra något, eftersom handtaget/filpekaren redan har kommit till slutet av filen.

4.2.2 Skriva till filer

Om vi vill skriva till en fil `out_data.txt` måste återigen använda `open` för att skapa ett filhandtag, men nu måste man skicka med strängen “w” som andra argument (w för *write*) för att indikera att man tänker just skriva till filen.

```
output = open('out_data.txt', 'w')
```

Detta *skapar* eller *tömmer* filen `out_data.txt`. **Varning:** Var försiktig då allt i filen `out_data.txt` kommer att försvinna om den redan finns! Om vi vill skriva till `output` (dvs till filen `out_data.txt`) så använder man idag nyckel-parametern `file` till `print`:

```
print('This is my first line', file=output)
print('\n', file=output) # start a new line
print('This is my second line', file=output)
```

I äldre kod kan du komma att se ett annat skrivsätt:

```
output.write('This is my first line')
output.write('\n') # start a new line
output.write('This is my second line')
```

Här används `write` som är en funktion som är associerad med en öppnad fil. Det här skrivsättet används också i många andra programmeringsspråk.

Observera att det inte är möjligt att skriva till en fil som är öppnad för läsning, och inte heller tvärtom.

4.2.3 Stänga filer

När vi är klara med en fil bör vi stänga den:

```
h.close()
output.close()
```

Det är viktigt att stänga filer:

- Antalet öppnade filer är begränsat i ett operativsystem – slösa inte med dem!
- Ett annat program kan vilja skriva till öppnad fil – släpp den så annat program inte behöver vänta!
- Du kan förlora data om du inte stängt filen.

När ett program avslutas så stängs öppna filer automatiskt, men det anses ändå vara god programmering att tydligt stänga en fil.

4.2.4 Ett exempel

Tänk dig en fil `people.txt` med komma-separerad information: efternamn, förnamn, födelseår.

```
Svensson,Lisa,1815
Olsson,Erik,1901
Mörtberg,Anders,1986
```

Följande program skapar en fil `modern.txt` med alla från 1900-talet:

```
h_in = open('people.txt', 'r')
h_out = open('modern.txt', 'w')
for n in h_in:    # loop through the file, row by row
    lname, fname, year = n.split(',')
    if int(year) > 1900:
        h_out.write(fname + ' ' + lname + '\n')
h_out.close()
h_in.close()
```

Öppnar vi nu filen `modern.txt` så kommer den innehålla

```
Erik Olsson
Anders Mörtberg
```

4.2.5 Ett praktiskt idiom: `with open`

Python-konstruktionen `with open` har blivit populär och anses idag vara idiomatisk för att arbeta med filer. Anledningen är att idiomat förbättrar läsbarheten för program och uppmuntrar till god kodstruktur.

Här är ett minimalt exempel:

```
with open('ut.txt', 'w') as out:
    print('hej hopp', file=out)
```

Detta motsvarar

```
out = open('ut.txt', 'w')
print('hej hopp', file=out)
out.close()
```

Det finns flera fördelar med att använda `with open`:

- Ditt program blir med lättläst.
- Det uppmuntrar till god kodstruktur.
- Det anses underlätta felhantering.
- Filer stängs när man lämnar `with`-blocket

I Python kallas `with` för en *kontexthanterare* (eng: *context handler*) och kan tillämpas även för annat än filer.

Exemplet ovan kan skrivas om med `with` på följande sätt:

```
with open('people.txt', 'r') as h_in, open('modern.txt', 'w') as h_out:
    for n in h_in:
        lname, fname, year = n.split(',')
        if int(year) > 1900:
            print(f'{fname} {lname}', file=h_out)
```

4.2.6 Att tänka på kring filhantering

- Läs och skriva filer är långsamt.
 - Undvik att läsa en fil två gånger.
 - Använd filer som mellanlagring bara om du måste.
- En del filer är stora. Undvik därför att läsa in allt om det ej behövs. Att läsa in 10 GB data till en dator med 8 GB RAM kan vara katastrofalt för en beräknings hastighet, och om man ändå behandlar data radvis så kan det gå mycket snabbare att läsa in data rad för rad.
- Undvik att hårdkoda sökvägar. Du ska inte anta att andra användare har filerna på samma plats som du.
- Tillfälliga filer bör ha unika och tillfälliga namn.
 - Använd modulen `tempfile` (<https://docs.python.org/3/library/tempfile.html>) för säkerhet och bekvämlighet!

4.2.7 Tre speciella filer

På Unix-system (det inkluderar olika Linux-varianter och MacOS) så finns alltid tre filer öppna, och det har förts över till PC-system också:

- `sys.stdin`
- `sys.stdout`
- `sys.stderr`

Du kommer åt dessa om du har använt instruktionen `import sys` som betyder “läs in modulen `sys`” (läs mer om `import` i kapitel 7!).

Det som är speciellt med dessa filer är att de inte är knutna till någon slags fil på hårddisken (eller motsvarande), åtminstone inte utan annan programmatisk instruktion. I vanliga fall är `sys.stdin` knutet till ditt tangentbord och om du läser från den filen så är det som att läsa av tecken vartefter du skriver

dem. Filerna `sys.stdout` och `sys.stderr` kan man se som knutna till den terminal eller utdata-enhet som din Python-tolk använder. Dessa två kan styras om, till exempel till en datafil, och det är grunden till att man har *två* utdata-filer: om du skriver varningar och felmeddelanden till `sys.stderr` så beblandas inte dessa med utdata. Du får helt enkelt två skilda kanaler för data och användarkommunikation.

Dessa tre filer är ett arv från 70-talets datorer och hör till Unix-traditionen, men är än idag mycket användbart.

4.3 Mer om loopar: nästling

Syntaxen för en **for**-loop bygger på att den indragna “kroppen” på loopen har en eller flera satser. Man kan förstås använda *fler* loopar:

```
for i in range(3):
    for j in range(3):
        print(i * j)
```

```
0
0
0
0
1
2
0
2
4
```

Märk att man ökar indraget för varje loop man gör. Indraget är viktigt för att avgöra strukturen på programmet och hur det exekveras! Ett exempel:

```
print('-----')
for i in range(1,4):
    for j in range(1,4):
        print(i, j, i * j, sep=' ')
    print('-----')
```

```
-----
1  1  1
1  2  2
1  3  3
-----
2  1  2
2  2  4
2  3  6
-----
3  1  3
3  2  6
3  3  9
-----
```

Notera att `print('-----')` endast körs en gång i den yttre loopen. Vad hade hänt om vi istället haft den lika långt indragen som `print(i, j, i * j, sep=' ')`?

Vi kan även kontrollera att endast beräkna jämna produkter med hjälp av villkorssatser i looparna. Notera indenteringsnivån på de två olika **if**-satserna.

```
for i in range(1,8):
    if i % 2 == 1:
        continue
    for j in range(1,8):
        if j % 2 == 1:
            continue
        print(i * j)
```

```
4
8
12
8
16
24
12
24
36
```

4.4 Algoritmiskt tänkande

När man lär sig programmering är en av de stora utmaningarna att skapa sig en djup förståelse för datorn och programmeringsspråket, för hur annars ska man veta vad man ska instruera datorn?

Språket Python är ett *högnivåspråk*, vilket innebär att datorns tekniska detaljer i stor utsträckning abstraherats bort. Det kan hjälpa att känna till vissa tekniska detaljer, se till exempel avsnittet om *datarepresentation*, men i Python kan du komma långt ändå och väsentligen låtsas som att du arbetar i en abstrakt och matematisk miljö utan större tanke på hur din dator faktiskt fungerar.

Man kommer däremot inte ifrån det *algoritmiska tänkandet*: att förstå hur man går från problembeskrivning till en lösning i form av kod. Detta kan vara utmanande eftersom vi till vardags inte är så noggranna med att beskriva hur man gör saker. När du kommunicerar med andra om vad som ska göras i ett projekt eller uppdrag så är det mycket som lämnas outtalat. Människor delar så mycket erfarenhet och bakomliggande kunskap att alla detaljer inte behöver kommuniceras. Du och din dator delar däremot inte mycket erfarenheter; för att få datorn att göra det du vill måste du förklara dig in i minsta detalj.

4.4.1 Recept är inte algoritmer

Recept brukar användas som en analogi för algoritmer och används ibland för att illustrera algoritmiskt tänkande. Analogin är bra som första förklaring av vad en algoritm är, men den kan också leda fel eftersom recept bygger på mycket bakomliggande kunskap. Om du har lärt ut matlagning till ett barn så har du nog märkt att det krävs lite mer än att bara läsa receptet.

Ta till exempel uppgiften att tillaga pannkakor. Här är ett recept (där första delen om ingredienser är borttagen) som de flesta av oss kan följa utan problem.

1. Vispa ut vetemjölet i hälften av mjölken till en slät smet.
2. Vispa i resterande mjölk, ägg och salt.
3. Smält smör i stekpannan och håll ner i smeten.

4. Grädda tunna pannkakor i stekpannan. Circa 1 dl smet per lagg är lagom.

Redan i första punkten är det en massa detaljer som saknas. Ska jag vispa i mjölet innan det hamnar i mjölken, eller är det bättre att dumpa all mjöl i mjölken och sedan börja vispa? Vilket redskap bör användas för vispningen? Och var ska vi arbeta? Man kan ju tänka sig att lägga mjölet på bänken och blanda ner mjölken, men det verkar opraktiskt. Ska man kanske använda mjölkförpackningen? Nja, det är nog bättre att ta fram en ren bunke att arbeta i, men hur stor bör den vara?

Låt oss försöka förbättra första punkten så att receptet blir mer algoritmiskt:

1. Blanda vetemjöl och mjölk så här:
 - a. Ta fram en bunke som har plats för 2 liter och häll i hälften av mjölken.
 - b. Ta fram en visp.
 - c. Häll i mjölet, en matsked i taget, och vispa 20 sekunder mellan varje matsked.
2. Vispa i resterande mjölk, ägg och salt.
3. Smält smör i en stekpanna och häll ner i smeten.
4. Grädda tunna pannkakor i stekpannan. Circa 1 dl smet per lagg är lagom.

Det som åstadkommit är att en bedömning plockats bort (vad är en "slät smet"?) och detaljeringsgraden har ökat, förhoppningsvis så att viss tvetydighet har försvunnit. Skulle en sjuåring kunna följa steg 1?

Punkt 2 och 3 kan göras mer algoritmiska på liknande sätt, men lägg märke till att punkt 4 har lite större utmaningar. Dels presenteras uppgifterna i fel ordning (smeten måste ju hållas i pannen för att kunna gräddas!), dels lämnas bedömningar till läsaren. Istället för att föreslå 1 dl smet så bör vi noggrant förklara "häll i smet tills att hela stekpannan täcks av ett tunt lager, men inte mer, eller smeten tar slut." Det sista villkoret hanterar att smeten sällan räcker till en täcka stekpannans yta för sista pannkakan. Vi måste också förklara att stekpannan ska vara varm (hur varm?), det behövs en stekspade, smeten gräddas tills att undersidan blivit brunaktig och då vänder man och gräddar andra sidan. Och vad innebär "brunaktig" egentligen?

Som du ser blir det mycket att skriva och detaljeringsgraden kan snabbt bli lite frustrerande, men det kan vara vad som krävs för att beskriva för en robot hur man gräddar pannkakor på "traditionellt sätt".

4.4.2 *Top-down, down-up, och att göra rätt*

När vi förbättrade pannkaksreceptet så började vi med en ganska grov beskrivning av vad som ska göras och förfinade första punkten. Det är ett vanligt arbetssätt, särskilt när man är lite osäker på vad som ska åstadkommas, och kallas för att arbeta uppifrån och ner, eller *top-down*. Tanken är att stegvis bryta ner en uppgift i mindre delar, tills att man når en nivå där det går att formulera kod för att lösa deluppgiften.

Den motsatta ansatsen är att arbeta nerifrån och upp, *bottom-up*, genom att identifiera användbar kod som det sedan går att komponera ihop en större helhet med. Om man ska använda pannkaksexemplet så kan man till exempel säga "jag vet hur man blandar mjöl och mjölk, och det kommer ju behövas, så jag började koda den deluppgiften."

Du behöver inte välja mellan *top-down* och *bottom-up*, båda ansatserna är användbara, och det viktiga är att arbeta med uppgiften. Du ska inte ha förväntningen att man funderar på problemet, ser lösningen, och sen skriver den kod som behövs. Så kan det vara ibland, åtminstone för en erfaren programmerare, men det är inget fel att experimentera sig fram och leta lösningar. Kom dock ihåg att skriven kod både kan och bör modifieras efter behov. Nybörjare kan vara nöjda över att skriven kod fungerar och undviker därför ibland att göra ändringar, men det innebär ofta att man skjuter upp problemen och hamnar i svårare problem senare. Det är lätt hänt att göra ett lite olämpligt designval och när det upptäcks så bör man rätta misstaget. Det kan innebära att revidera sin kod om och om igen. En av de viktigaste

insikterna sen programmeringens barndom är just att vi alla gör fel, oavsett hur väl man studerat en uppgift och hur noggrant man planerat, och att det är bra att revidera kod.

Ett annat triviale faktum, som ändå missas ibland, är att det är avgörande att förstå uppgiften. Det är, till exempel, lätt hänt att uppdragsgivare och programmerare menar helt olika saker när en uppgift diskuteras och oavsett hur snyggt programmeraren löser uppgiften (enligt sin egen definition) kommer uppdragsgivaren att vara missnöjd. För att återkomma till pannkaksexemplet så avser antagligen en uppdragsgivare med amerikansk bakgrund inte samma sak med ordet "pancake" som en svensk programmerare som kanske översätter till "pannkaka". Det ligger långt utanför den här kursen att lösa den utmaningen, men det är värt att påpeka att även om du programmerar för egen del så kan du upptäcka att det finns fler aspekter på en uppgift än du förstått från början.

4.4.3 Arbeta med uppgiften!

Några saker att tänka på, som kan uppfattas som triviala, men är värdefulla att gå igenom, särskilt när man jobbar tillsammans med andra:

1. Vad är indata till beräkningen? Till exempel, om du vill räkna antalet ord i en text, hur är texten representerad? Den kanske ges som en enda lång sträng? Eller kanske som en lista av strängar, där varje sträng är en mening? Det är också bra att gå igenom lite extremfall. För exemplet med att räkna ord är det bra att veta om kan texten vara tom, det vill säga ha längd noll, eller finns det alltid minst ett ord? Om vi går tillbaka till pannkaksalgoritmen, vet vi alltid att vi har tillräckligt med ingredienser? Det vore kanske bra att anpassa receptet efter hur många ägg man faktiskt har?
2. Hur ska resultatet se ut? I allmänhet vill vi skriva funktioner som löser uppgifter, se kapitlet om *programstruktur*, och då ska man utgå ifrån att funktionen ska returnera ett värde, men värdet kan bestå av mycket information. Om du skriver en funktion som räknar antalet ord i en text så kan det räcka med ett heltal som svar, men kanske är det mer användbart att returnera en uppslagstabell som associerar ord med ett heltal som beskriver hur många gånger det återkommer i texten?

Det vi pratar om här är alltså start- och slutpunkt för en beräkning. Men hur reder man ut det som kommer däremellan? Det är där själva algoritmen ligger och det är inte alls självklart att den är lätt att komma på. Vissa algoritmiska uppgifter är helt enkelt mycket svåra att förstå och hitta lösningar på. I den här kursen är uppgifterna relativt enkla, i den meningen att gymnasiekunskaper i matematik är tillräcklig förberedelse. Oavsett svårighetsgrad så finns det sätt att arbeta sig fram till en algoritm.

Det går att arbeta sig fram till algoritmer och de behöver inte alls vara komplicerade för att lösa problem som är svåra att beskriva. Här är några enkla tips för att komma framåt.

- Skapa små exempel. Dessa exempel kan vara bra för att experimentera med när man vill komma på hur en algoritm ska se ut, och de kan vara användbara när man vill testa sin kod.
- Hur löser du *egentligen* en uppgift i huvudet? Vissa saker, som addition, är vi så duktiga på att vi slutat tänka på vilka steg man tar. En noggrann genomgång av de steg som du är bekväm med kan ge dig lösningen.
- Hur skulle du lösa uppgiften med penna och papper? Prova dig fram!

Just att arbeta med uppgiften är viktigt för att förstå problemet och det kan vara kontraproduktivt att börja skriva kod med en gång; om du inte förstår problemet så kommer din kod sannolikt vara fel. Å andra sidan kan kodning vara ett sätt att upptäcka att man inte förstått problemet.

Slutligen, se också till att förstå den kod du själv skriver. Man ska inte önska sig fram till fungerande kod. Om du tänker "den här funktionen borde ge mig svaret" eller "jag tror det här fungerar" så ska du bromsa upp: ord som "borde" och "tror" är varningssignaler. Du ska *veta* att koden fungerar eller åtminstone ha god skäl för att tro att den fungerar. Om det känns skakigt så ska du helt enkelt testa. Verifiera att

funktioner gör det du tror de gör. Resonera för dig själv om vad som borde hända i din kod. Förutom att du fastställer vad som fungerar och inte fungerar så är det ett bra sätt att lära sig programmering.

4.5 Representation av data

Ofta behöver man inte fundera på hur data lagras i en dator. När man jobbar med till exempel heltal så räcker det att ha en abstrakt bild av att tal lagras som nollor och ettor, för vi kan ju lätt se hur man kan skriva $x = 7$ och på olika sätt använda att heltalet 7 “finns” i x . Reglerna för hur nollor och ettor bildar heltal kallas *heltalsrepresentation* och är idag standardiserat (men Python avviker, se senare kapitel).

De inbyggda typerna i Python (och andra programmeringsspråk) är lätta att använda utan att bry sig om deras representation, men för mer problemspecifika data (tex personers adresser) och abstrakta idéer (som polynom och andra matematiska begrepp) måste programmeraren tänka efter och hitta sätt att använda språkets datastrukturer och andra finesser för att hitta en lämplig *datarepresentation*. I det här kapitlet diskuterar vi några exempel på hur man kan arbeta med datarepresentation.

4.5.1 Lärandemål

- Representation av heltal, flyttal, strängar, och boolska värden
- Andra data
 - Hur ska man lagra koordinater?
 - Hur kan man lagra polynom?
 - Lagring av datum och tid

4.5.2 Primitiva datatyper

Många data representeras på ett rättframt sätt i programmeringsspråk eftersom det finns inbyggda typer för dem. De mest grundläggande typerna brukar kallas *primitiva datatyper* och i Python är det `bool`, `int`, `float`, `complex`, och `str`. Olika språk har olika primitiva datatyper: i språket C är `char` en datatyp som representerar ett tecken i exakt en byte, och strängar är (ungefär) listor av `char`-värden. Python gör tvärtom, eftersom även ett enskilt tecken ska lagras i en `str`.

Listor och uppslagstabeller kallas *sammansatta datatyper*, eftersom de i någon mening består av primitiva datatyper.

4.5.3 Datastrukturer

En datastruktur är en konstruktion som kan samla flera primitiva datatyper till en enhet. Du har redan stött på listor och uppslagstabeller som är utformade för att samla olika mängder av data så att delarna blir lätt åtkomliga. Dessa datastrukturer är generella och deras syfte är att göra det lätt arbeta med data av variabel storlek, men man kan också se dem som bekväm teknik att indexera eller namnge delar av data. I ett senare kapitel introducerar vi *tupler* (kapitel 6) samt begreppen *klass* och *objekt* som ger oss ett sätt att skapa egna datastrukturer. Klasser används gärna för att göra program mer lättlästa.

4.5.3.1 Representera abstraktioner

De flesta uppfattar det nog inte som så konstigt att begrepp som *namn*, *adress*, och *text* är lämpliga att lagras som strängar. Det kan däremot vara svårare att se hur mer abstrakta begrepp ska representeras.

Som ett första exempel på abstraktioner kan vi ta punkter i planet. Vi kan vilja lagra positioner på en karta, punkter i en figur, eller liknande, och behöver ett sätt att arbeta med positionernas koordinater.

Man kan tänka sig att införa två variabler per punkt, som x och y och sätta $x=0$ och $y=0$ för origo, eller använda två listor x och y om man har många punkter, så origo kanske råkar hamna i första elementen: $x[0]=0$ och $y[0]=0$. Nackdelen med detta sätt att arbeta är att de två koordinater som behöver hanteras ihop ligger åtskilt, i de två variablerna x och y . Det kan ibland vara opraktiskt, men framförallt kan det också öka risken för att man gör fel.

Ett alternativ kan vara att låta koordinater lagras som par, eller snarare som listor med två element. Vi kan då exempelvis definiera $origin = [0, 0]$ eller $points = [[0,0], [1,0], [2,1]]$ och få värden grupperade på ett naturligt sätt. Det gör det lätt att göra enkla operationer. En tilldelning kan till exempel se ut som $origin = points[0]$.

Ett annat alternativ är att använda en uppslagstabell för punkter och initiera $origin = \{'x': 0, 'y': 0\}$. På det här viset kan vi alltså namnge elementen och det blir kanske lite lättare att läsa vad man avser i programmet. Vi skulle till exempel kunna skriva $p['x'] = p['x'] + \text{delta}$ för att justera x -koordinaten något, och det skulle nog vara mer lättförståeligt än motsvarande $p[0] = p[0] + \text{delta}$ om vi använder listor för punkter.

Det vi diskuterar här är hur punkter i planet representeras. Vi tar ett mycket abstrakt begrepp (punkter), funderar på vilka data man behöver arbeta med (x och y), och bestämmer hur vi kan använda enkla datastrukturer för den faktiska lagringen.

4.5.4 Exempel: personnummer

I många IT-system behöver man hantera personnummer. Hur representeras de bäst? Eftersom ett personnummer kan anges med 10 eller 12 siffror kan det vara lockande att betrakta det som ett heltal. Till exempel kan personnumret 890201-3286 (fiktivt) representeras som 8902013286. Detta är dock ett stort tal, för stort för att rymmas inom ett 32-bitars heltal. Python klarar detta utan problem, men i de flesta andra programmeringsspråk kan det orsaka svårigheter. En lösning är att dela upp personnumret i två delar, exempelvis `[890201, 3286]`. På så sätt rymmer båda delarna inom 32-bitars gränsen.

Nu kan vi ta ett steg tillbaka och fundera över vad vi faktiskt vill göra med personnummer. Oftast är behovet att kunna jämföra dem – till exempel för att besvara frågan “är det här rätt person?” – och det är sällan nödvändigt att utföra aritmetiska operationer på dem. Därför finns det ingen större fördel med att lagra personnummer som heltal.

Om vi däremot vill verifiera att ett personnummer är korrekt inmatat kan de enskilda siffrorna användas för detta ändamål. Vi kan även vilja försäkra oss om att datumdelen i personnumret är ett korrekt datum. Många programmeringsspråk har bra stöd för datumhantering med särskilda datatyper. (I kapitel 7 introducerar vi *moduler*, och Python har modulen `datetime`, som är användbar för arbete med datum.)

Den som har erfarenhet av personnummer kan dra sig till minnes att Skatteverket använder “samordningsnummer” för personer som inte är folkbokförda i Sverige. Samordningsnummer liknar personnummer men är ändå lätt åtskiljbara, eftersom de bygger på födelsedatumet men har en extra justering: man adderar 60 till dagen i födelsedatumet. Om vi exempelvis omvandlar personnumret 890201-3286 till ett samordningsnummer, skulle det bli 890261-3286. Ett sådant datum kan inte representeras med en vanlig datumtyp, eftersom det inte är ett giltigt kalenderdatum.

Kanske är det enklaste att låta personnummer representeras av strängar?

4.5.5 Exempel: polynom

Om man arbetar med polynom (vilket inte bara matematiker kan behöva göra) så behöver man en representation för dem. Det är inte praktiskt att lagra $4x^2 - 3x + 7$ som en sträng, för hur ska man då

göra beräkningar på det? Ett vanligt sätt att arbeta med polynom är att lagra polynomets koefficienter som tal i en lista, och det mest praktiska är att låta koefficienten för termen med grad i vara på plats i i listan. Det kan kännas som baklänges, men en fördel är att polynomets grad är längden på listan minus ett. Dessutom finns det algoritmiska fördelar.

Några exempel gör det tydligare:

Polynom	Python-representation
$3x^2 - 2x + 1$	[1, -2, 3]
x^4	[0, 0, 0, 0, 1]
$4x^2 + 5x^3$	[0, 0, 4, 5]
$5 + 4x + 3x^2 + 2x^3 + x^4$	[5, 4, 3, 2, 1]

En matematiker kan bli orolig för hur vi representerar noll-polynomet. Är det som [] eller [0]? Det är ett specialfall som man kan behöva hantera, till exempel vid beräkning av ett polynoms gradtal, men det är en detalj som vi inte fördjupar oss i här.

Med representation av polynom som koefficientlistor har vi möjlighet att göra olika operationer. Att addera två polynom kommer att hanteras genom att addera elementen i två koefficientlistor, med viss hänsyn till att polynomen kan ha olika gradtal.

Som exempel kan vi titta på derivering av polynom. Vi vill kunna derivera *symboliskt*, utan att beräkna derivatan i en viss punkt, så resultatet ska bli ett nytt polynom. Hur lärde vi oss i skolan? Jo, vi har lärt oss att konstanttermen försvinner, en term ax^b deriveras till abx^{b-1} , och varje term kan deriveras för sig. Vi kan alltså iterera igenom en koefficientlista, hålla reda på graden på termen som koefficienten representerar, och beräkna nya koefficienter som ab . En enkel algoritmidé för att symboliskt derivera ett polynom p givet som en koefficientlista:

- Skippa koefficienten på index 0.
- Håll reda på aktuellt gradtal.
- Lägg de nya koefficienterna i en ny lista.
- Iterera över koefficienterna från index 1 och beräkna de nya koefficienterna.

Vi kan formulera Python-kod nästan direkt från denna algoritmidé:

```
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial[1:]:
        new_coefficient = term * degree
        result.append(new_coefficient)
        degree += 1
    return result
```

Om man testar funktionen så kan det se ut så här:

```
derivative([1, -2, 3])
```

```
[-2, 6]
```

Det innebär alltså att vi har gått från ett abstrakt polynom till en konkret symbolisk representation, deriverat "maskinellt" till ny koefficientlista, som representerar ett annat abstrakt polynom:

$$3x^2 - 2x + 1 \longrightarrow [1, -2, 3] \longrightarrow [-2, 6] \longrightarrow 6x - 2$$

Vi kan konstatera att med rätt representation kan vi gör avancerade beräkningar på abstrakta begrepp.

4.6 Uppgifter

1. Givet de två listorna nedan

```
a = ['a', 'b', 'c']
b = [1, 2, 3]
```

skapa en uppslagstabell `{ 'a': 1, 'b': 2, 'c': 3 }` genom en loop.

Tips: Det finns flera sätt att åstadkomma detta. Ett sätt är att använda `range` för att få ett index som ni sedan kan använda för att hämta ut element i `a` och `b` med.

2. Skriv kod som tar alla par (x, y) av tal mellan 2 och 20 med $x \neq y$ och skriver ut deras gemensamma delare (GCD).

Tips 1: Använd nästlade loopar och `range`.

Tips 2: Använd funktionen `GCD` från förra kapitlet.

Koden ska till exempel skriva ut följande:

```
x y GCD
2 3 1
2 4 2
2 5 1
2 6 2
2 7 1
2 8 2
# osv...
```

3. Gör hela pannkaksreceptet i avsnitt 4.4.1 mer algoritmiskt!
4. I den här kursen använder vi funktioner för att strukturera koden. Vilka funktioner kan du definiera så att pannkaksreceptet i avsnitt 4.4.1 blir lättare att läsa?
5. Skriv ner en algoritm för att addera två heltal givna som två listor med siffror. Du kan ju addera stora tal med papper och penna, men hur kan algoritmen beskrivas i text? Kan du skriva kod för algoritmen? Denna uppgift är svårare än det låter!
6. I uppgift 2 är det onödigt att skriva ut GCD för till exempel $x, y = 2, 3$ och $x, y = 3, 2$. Gör om koden så att x alltid är mindre än y i raderna som skrivs ut.

Tips: I den inre `for` loopen, vad händer om man itererar från nuvarande värde på x ?

7. I kapitel 3 jobbade vi med funktionen `vowels`. Använd en uppslagstabell som har vokalerna i `vs` som nycklar och ett tal som värde (du kan hitta på talen själv). Modifiera funktionen så att den istället byter ut vokalerna mot talen.
8. I kapitel 3 jobbade vi med funktionen `split` (given nedan). Ha strukturen i denna kod som hjälp för att skriva en funktion `annotate_word` som tar en sträng och associerar varje ord med ett tal där talet representerar när ordet sist förekom i texten (räknat från 1). Funktionen ska returnera en uppslagstabell med ord som nycklar och tal som värden.

```
def split(s):
    out = []
    term = ''
    for c in s:
        if c == ' ':
            out.append(term)
            term = ''
        else:
            term += c

    # add the final word unless it is empty
    if term != '':
        out.append(term)

    return out
```

Funktionen ska funka så här:

```
print(annotate_word('hej du hej hej du'))
```

```
{'hej': 4, 'du': 5}
```

9. Hur kan vi enkelt få `annotate_word` i uppgift 8 att istället returnera en uppslagstabell med tal som nycklar och ord som värden?
10. Modifiera `annotate_word` i uppgift 8 så att inte bara sista förekomsten sparas i texten, utan alla förekomster. Vilken datastruktur bör vi ha som *värde* i vår uppslagstabell?

Kapitel 5

Felhantering, sårfall, listomfattning, och felsökning

Det blir fel ibland, även när man kör program, och felen måste då hanteras. När man programmerar är det typiskt status på en funktion man är intresserad av: gick beräkningen bra eller har det uppstått ett fel? Hur fortsätter vi efter felet?

Hittills i kompendiet har vi definierat funktioner som returnerar resultat (värden), samt att något instruktivt meddelande kanske har skrivits ut med hjälp av att anropa `print`. Alternativt har programmet bara kraschat eller fastnat i en oändlig loop.

I detta kapitel kommer vi att undersöka hur osäker kod vid fel kan påkalla, eller "lyfta", ett **sårfall**. Detta kan hanteras på ett lämpligt sätt med hjälp av **felhantering** vilket i Python görs med **try** och **except**.

Vi ska även i slutet av kapitlet titta på ett helt annat koncept: listomfattningar. Detta är ett smidigt sätt att skapa och modifiera listor, som ofta är mycket mer kompakt än att skriva en loop.

5.1 Felhantering och sårfall

Ett naivt sätt att hantera fel i program är att returnera en felkod i form av ett tal eller en sträng. Vi ska nu börja med att titta på hur detta kan se ut, men sen snabbt gå över till att titta på hur fel kan hanteras bättre med hjälp av sårfall.

5.1.1 Felhantering med hjälp av returnerade felkoder

Att förlita sig på returvärden för felhantering ger komplexa program som tenderar att bli svårlästa. Många funktionsanrop måste följas av **if**-satser för att se om ett fel uppstod som man måste anpassa sig efter. Om ett fel uppstod kan det bli knepigt att återgå till en bra plats i programmet. Erfarenheten är att programmerare undviker den ökade komplexiteten i sitt program genom att ignorera de returnerade felkoderna vilket ger instabila program som lätt kraschar. Denna typ av felhantering förekommer ofta i t.ex. C, Fortran och andra äldre programmeringsspråk.

Ett exempel på denna gamla teknik följer nedan där vi anropar en funktion som ska modifiera en lista. Alla returkoder förutom 0 signalerar att något fel har skett.

```
def divide_list(l,x):
    if type(x) != int or type(x) != float:
        return 11
    elif x == 0:
        return -1
    else:
        # check that elements in list are integers
        if not all([ type(item) == int for item in l]):
            return 12
        for i in range(len(l)):
            l[i] = l[i]/x # we are modifying the list items here
        return 0

return_code = divide_list(l,x)
if return_code == -1:
    # do something
elif return_code == 11:
    # do something else...
elif return_code == 12:
    # do something else...
elif return_code == 0:
    # good!
```

Denna kod är ganska rörig och onödigt komplicerad. Många moderna språk stödjer därför specifika sätt att hantera fel på genom så kallade "särfall" (eng: *exceptions*, ibland översatt till "undantag").

5.1.2 Särfall

Python använder sig av de reserverade orden **try** och **except** för att dela in koden i block som vi inte litar på, vi kan kalla dem osäkra passager, och block som exekveras vid ett särfall med syfte att hantera problemet. Här är grundstrukturen:

```
try:
    # unsafe code
    ...
except:
    # action
    ...

# Rest of the code
...
```

Konstruktionen **try** lägger man alltså runt ett kodblock som kan betraktas som en osäker passage (# *unsafe code* ovan) och efter **except** lägger man de åtgärder som man hanterar eventuella problem med. Indenteringen är viktig här, precis som vid funktionsdefinitioner, villkorssatser och kontrollflöden. Blocket som hör till **except**-satsen (# *action*) exekveras endast om ett fel har gett ett särfall i den osäkra passagen under **try**-satsen: om inget fel har uppstått fortsätter koden utan att **except**-blocket körs. Om ett fel uppstår, så kan åtgärderna under **except** antingen avsluta exekveringen av programmet eller så kan felet behandlas på något annat sätt (t.ex. ignoreras, även om man ska vara försiktig med att strunta i fel som ger ett särfall).

5.1.2.1 Exempel: läsa in ett heltal från användaren

Betrakta följande kodsnitt:

```
while True:
    answer = input("Write a number (write 0 to quit): ")
    x = int(answer)
    if x == 0:
        print("Bye bye!")
        break
    else:
        print("The square of the number is: " + str(x ** 2))
```

Vad händer om användaren skriver in något annat än ett tal?

```
Write a number (write 0 to quit): hej
```

```
ValueError: invalid literal for int() with base 10: 'hej'
```

Detta är, som vi sett tidigare, en konsekvens av att resultatet av `input` alltid har typen `str`, och att det inte alltid går att explicit typomvandla från `str` till `int`. Här är `ValueError` ett särfall: värdet av inparametern ('hej') till `int` är felaktigt. Om man istället använder sig av `try/except`-sats så kan felet hanteras på rätt sätt:

```
while True:
    try:
        answer = input("Write a number (write 0 to quit): ")
        x = int(answer)
        if x == 0:
            print("Bye bye!")
            break
        else:
            print("The square of the number is: " + str(x ** 2))
    except ValueError:
        print("You must write a number!")
```

I koden ovan fångas alla särfall av typ `ValueError` som lyfts i `try`-blocket. Om ett annat typ av särfall skulle påträffas skulle programmet avslutas. Man kan även fånga *alla* fel med endast `except`:

```
while True:
    try:
        answer = input("Write a number (write 0 to quit): ")
        x = int(answer)
        if x == 0:
            print("Bye bye!")
            break
        else:
            print("The square of the number is: " + str(x ** 2))
    except:
        print("You must write a number!")
```

För att koden ska bli så överskådlig och tydlig som möjligt, vill man dock helst skriva så lite kod som möjligt innanför `try`-blocket. Genom att vara specifik i sin avsikt kan exemplet ovan förbättras till

följande:

```
while True:
    answer = input("Write a number (write 0 to quit): ")
    try:
        x = int(answer)
    except ValueError:
        print("You must write a number!")
        continue

    if x == 0:
        print("Bye bye!")
        break
    else:
        print("The square of the number is: " + str(x ** 2))
```

Med continue ser vi till att starta om while-loopen, innan x börjar användas. Vi har identifierat att det bara är typkonverteringen som kan orsaka problem.

5.1.2.2 Exempel: beräkna längden av rader i en fil

Låt oss säga att vi vill beräkna längden på alla rader i en fil:

```
def len_file(filename):
    with open(filename, 'r') as h:
        for s in h:
            print(len(s))

len_file("foo.txt")
```

Om filen foo.txt inte finns får vi följande

```
FileNotFoundError: [Errno 2] No such file or directory: 'foo.txt'
```

Det är bättre att skriva:

```
def len_file(filename):
    try:
        with open(filename, 'r') as h:
            for s in h:
                print(len(s))
    except FileNotFoundError:
        print("Could not open file " + filename + " for reading.")

len_file("foo.txt")
```

Fördelen är att vi får ett program som kommunicerar med ett mer lättläst felmeddelande till användare. Det "vanliga" sättet som Python signalerar fel med är mer till för programmerare. Genom att hantera fel som kan uppstå vid till exempel felhantering kan man göra sitt program mer användarvänligt.

5.1.2.3 Olika åtgärder för specifika sårfall

Det är mycket som kan gå fel och olika sårfall kan man vilja hantera på olika sätt. I nästa exempel så görs separata åtgärder för om något går fel vid filinläsningen, om ett tal delas på noll, eller om den inlästa strängen från filen inte kan typomvandlas till ett heltal. Notera att åtgärden inte behöver placeras nära platsen där felet uppstår.

```
def divide_by_elems(filename,x):
    quotients = []
    with open(filename, 'r') as h:
        for n in h:
            frac = x / int(n)
            quotients.append(frac)
    return quotients

try:
    data = divide_by_elems('numbers.txt', 2)
except IOError:
    print("divide_by_elems: A file-related problem occurred.")
except ZeroDivisionError:
    print("divide_by_elems: Division by zero.")
except ValueError:
    print("divide_by_elems: Character could not be converted to int.")

print(data)
```

Om koden ovan skulle köras med en fil `numbers.txt` som innehåller följande:

```
1
2
0
4
3
0
```

så genereras och fångas ett sårfall. Vilket? Kan du provocera fram de två andra sårfallen?

5.1.2.4 Sårfall för kontrollflöde

Man kan även använda sårfall för kontrollflöde i sitt program. Låt oss säga att vi vill beräkna kvoten mellan varje element i två listor (och ignorera alla element i slutet av den längre listan), men inte krascha om det blir division med noll. Då kan vi skriva följande:

```
def compute_ratios(xs,ys):
    ratios = []
    for i in range(min(len(xs),len(ys))):
        try:
            ratios.append(xs[i]/ys[i])
        except ZeroDivisionError:
            ratios.append(float('NaN')) # NaN = Not a Number
    return ratios
```

```
print(compute_ratios([2,3,4],[2,0,5]))
print(compute_ratios([2,3,4,0],[2,0,5]))
```

I detta fall hade vi dock lika gärna kunnat använda en **if**-sats och testat om `ys[i] == 0`. Mer om detta senare.

5.1.3 Allmänt om särfall

Det anses vara idiomatisk Python att använda sig av **try-except**. Det är inte ovanligt att när man läser om Python-kod ser *EAFP*, som står för *It's Easier to Ask Forgiveness than Permission*. Det här kontrasteras mot *Look Before You Leap (LBYL)*. Det är inte menat att man ska överanvända **try-except**: särfall som uppstår kan ibland ge den tydligaste informationen. Varje möjliga särfall i kod är inte menat att behandlas. Men osäkra passager där man har en tydlig åtgärd i åtanke är ett bra användningsområde.

Så vad är en osäker passage i ett program? Typiskt är:

- All filhantering.
- Kommunikation med användare.
- Kod som annonserar osäkerhet i dokumentationen ("throws exception if...").
- Kod som man vet eller upptäcker är osäker (t.ex. om man implementerar en algoritm som är instabil).

Det finns flera vanliga åtgärder när man fångat ett särfall:

- **Skriv ut felmeddelande och avsluta:** det gör man till exempel om data saknas eller om felet är så allvarligt att det inte finns rimliga sätt att fortsätta.
- **Skriv ut varning, ignorera felet, och fortsätt:** om man har konstiga indata som man kan bortse ifrån, och beräkningarna kan fortsätta ändå.
- **Hantera felet och fortsätt:** fungerar bra för en del förutsägbara fel, om man t.ex. får en dålig inmatning från en användare kan man ju be om ett nytt försök.
- **Skapa ett nytt särfall:** det är lämpligt om du har kod där flera fel kan uppstå, men där felet kan sammanfattas bättre i ett visst sammanhang. Om en algoritm är instabil så kanske **ArithmeticError** är ett bättre fel än **ZeroDivisionError**, eftersom användaren av algoritmen känner till algoritmens svaghet men kanske inte kan associera det med division med noll.

Det finns också åtgärder man ska undvika:

- **Ignorera inte felet!** Det värsta man kan ha efter **except** är instruktionen **pass**. Som användare blir det väldigt svårt att veta vad och var någonting går fel. Python har lyft ett särfall, men man har som programmerare bestämt att strunta i detta. Resultatet blir ofta att man får konstiga följdfel på en helt annan plats i koden.
- För **allvarliga** fel *räcker inte* en felutskrift. Det är inte snyggt att låta ett program fortsätta om felet är så allvarligt att man vet att det kommer uppstå ett annat fel senare. Vissa fel ska helt enkelt avbryta ett program.

5.1.4 Hantera egna stabilitetsproblem

Det är inte nödvändigtvis en svaghet att problem uppstår. Det är svårt, om inte helt omöjligt, att t.ex.:

- Förutse formatet på *all* möjlig indata.
- Se till att *alla* användare (inklusive du själv) anropar funktioner helt rätt.

I Python finns många specifika särfall definierade (se [dokumentationen](#) för en detaljerad beskrivning):

- `Exception`
- `ArithmeticError`
- `IOError`
- `IndexError`
- `MemoryError`
- `ZeroDivisionError`
- m.m.fl.

I Python är sårfall inte bara en textstrång eller felkod som skrivs, sårfallet är en klass (klasser kommer att behandlas i mer detalj senare i kompendiet). Det gör det möjligt att lätt skapa egna sårfallsvarianter, som är baserade på de existerande sårfallen.

Man kan även lyfta ett generellt sårfall med det reserverade ordet **raise** (det är härifrån termen “lyfta” kommer). I t.ex. Java och C++, åstadkoms detta med hjälp av `throw`. Om vi t.ex. vill ta ut första elementet ur en lista och kasta ett fel om listan är tom kan vi skriva:

```
def head(xs):
    if not xs:
        raise Exception("head: input list is empty")
    else:
        return xs[0]

try:
    print(head([]))
except Exception as e:
    print(str(e))

print(head([]))
```

```
head: input list is empty
# specific information about where the exception was caught
Exception: head: input list is empty
```

Notera att koden fortsätter efter första gången `head` anropades i **try-except**-blocket: endast strängen med felmeddelandet skrivs ut. Man kan “spara” sårfallet i en variabel som man vidare kan använda (för t.ex. information, eller om man vill lyfta sårfallet efter man utfört en viss operation som att spara något till en fil) genom **except Exception as e**, där `e` är identifieraren för det fångade sårfallet. Strängen med felmeddelandet fås genom `str(e)`. Andra gången `head` anropas så lyfts sårfallet som stöts på i funktionen: samma information skrivs ut igen men man ser att det är **Exception** som sköter sårfallet snarare än **except**-blocket och programmet avslutas.

Nyckelordet **raise** är användbart för att skriva mer detaljerade felmeddelanden, eller när ett “fel” uppstår som inte täcks av Pythons felhantering. Ett exempel på ett sådant fel är om en algoritm som vi skrivit måste bibehålla ett värde på en parameter `h` större än `0.01` om algoritmen ska fungera (tänk t.ex. steglängd på x-axeln för att hitta nollställe till polynom). Vi kan då skriva något i stil med

```
def my_algorithm(x, h):
    while True:
        if h < 0.01:
            raise ArithmeticError('Stepsize h should not be smaller than 0.01.\
The value of h was:', h)
```

```
# check polynomial root
...

# update h
...
```

5.1.5 Vad du skriver påverkar hur du jobbar med särfall

Man kan säga att det finns olika scenarier för användning av särfall.

- **Applikationsprogrammering:** Om du skriver ett program som använder kod från olika moduler, såväl egna och som andras, så kommer det finnas funktioner och metoder som genererar särfall. Det blir då viktigt att använda **try-except** för att ta om hand om de särfall som kan uppstå. Programmet ska inte avslutas på grund av problem som kunnat förutses och som det kanske finns enkla lösningar till. Man kan också vilja undvika att användaren av ditt program (du själv eller andra) ska behöva se och tolka felmeddelanden.

Några exempel:

- Om ett särfall uppstår på grund av dålig indata, då ska man tala om det på ett tydligt sätt. Att släppa igenom ett felmeddelande från ett särfall är inte användarvänligt, eftersom den egentliga orsaken ofta inte alls förklarar varför särfallet uppstått.
- Om ett särfall uppstår på grund av, exempelvis, numerisk instabilitet eller osäkerheter i omgivningen (som att en fil med ett visst namn redan finns) så vill man antagligen hantera det på ett bra sätt istället för att slänga ett särfall i ansiktet på användaren.
- **Stödprogrammering:** Det är mycket vanligt att man skriver kod som är stöd till det program man verkligen vill skriva. Om ditt mål är att skriva en egen variant av Matlab så behöver du många funktioner som gör beräkningar; om du vill analysera data lagrade i ett komplicerat format kanske det är bra att ha en modul som just hanterar data så att du senare kan fokusera på beräkningarna. Dessa stödrutiner kan stöta på problem som måste signaleras till användaren och det görs lämpligen genom att använda **raise** till att skapa ett särfall.

Python talar om *att* ett fel uppstått. Du som programmerare kan tala om *varför* ett fel har uppstått.

5.1.6 Vanliga misstag

5.1.6.1 if-sats istället för särfall för flödeskontroll

Det kan vara lockande att använda **except** för flödeskontroll, men ofta är det bättre att istället använda **if**-satser. Till exempel i denna kod:

```
def elem_access(xs,i):
    try:
        if i >= len(xs):
            raise IndexError
    except IndexError:
        print("Index is out of range.")
    return xs[i]
```

Det är ju ingen poäng med att först skapa ett särfall och sedan genast fånga det. Ett annat misstag i just detta exempel är att om det uppstår ett fel, så *fortsätter* programmet efter en felutskrift, här med att försöka komma åt element i trots att i är för stort. Om man fångar ett särfall så ska man hantera det på

något sätt. Det är ofta rimligt att avsluta programmet med ett felmeddelande och `quit()`. I det här fallet, när `elem_access` verkar vara en hjälpfunktion, är det bättre att bara skapa ett särfall vid problem och låta användaren av hjälpfunktionen bestämma vad man ska göra.

```
def elem_access(xs,i):    # Bättre version
    if i >= len(xs):
        raise IndexError
    return xs[i]
```

5.1.6.2 Överanvändning

Använd inte särfall för att tackla att kod inte fungerar som du hade tänkt dig.

I exemplet nedan skapas en ny uppslagstabell `d` för nycklar `e`, tagna från en lista, genom att anropa en funktion som använder informationen i uppslagstabellen `data`. Tydligt har programmeraren märkt att alla värden från `selected` inte finns med i `data` och därför skyddat sig genom att fånga de `KeyError`'s som kan uppstå:

```
data = {1 : "Make bed",
        2 : "Shower",
        4 : "Eat breakfast"}
selected = [1,3,2]

d = {}
for elem in selected:
    try:
        d[elem] = perform_task(data[elem])
    except KeyError:
        print()          # Bad line of code!
```

Det är bättre att använda en `if`-sats för att undvika felen:

```
data = {1 : "Make bed",
        2 : "Shower",
        4 : "Eat breakfast"}
selected = [1,3,2]

d = {}
for elem in selected:
    if elem in data:
        d[elem] = perform_task(data[elem])
```

5.2 Listomfattningar

Listomfattning (eng: *list comprehension*) är en teknik för att skapa listor som också är populär bland funktionella programmeringsspråk och därför ofta tas upp i det sammanhanget. Listomfattning är praktiskt för att initiera listor med värden att jobba vidare med. En fördel, som kanske ligger bakom listomfattningars popularitet, är att de kan vara mycket tydliga tack vare sin nära koppling till matematisk notation. Betrakta till exempel mängden av kvadrater av tal upp till 10:

```
[x**2 for x in range(10)]
```

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Man kan också filtrera bort elementen — inte alla behöver presenteras. Till exempel alla udda heltal upp till 20 kan skrivas så här:

```
[x for x in range(20) if x % 2 == 1]
```

```
[1, 3, 5, 7, 9, 11, 13, 15, 17, 19]
```

(Man kan korta ner det uttrycket lite till, kan du se hur?)

Vi kan även anropa på funktioner som konverterar element enligt något system

```
def transform(x):
    if x % 2 == 0:
        return "Even"
    if x % 3 == 0:
        return True
    else:
        return x
print([transform(x) for x in range(20)])
```

```
['Even', 1, 'Even', True, 'Even', 5, 'Even', 7, 'Even', True]
```

Sammanfattning: en listomfattning har tre delar:

- resultatdelen, med ett uttryck som ger elementen i resultatet,
- generatordelen, där **for** används för att plocka fram värden till resultatdelen, och
- predikatdelen, som bestämmer vilka element som ska presenteras.

Mer komplicerat exempel med nästlad loop:

```
[(a,b) for a in range(2,8) for b in range(2,8) if a % b == 0]
```

```
[(2, 2), (3, 3), (4, 2), (4, 4), (5, 5), (6, 2), (6, 3), (6, 6), (7, 7)]
```

Smidigt för att få alla kombinationer av element i två listor. Notera att utdata är en lista av **tupler**, dvs par av element.

En användbar funktion som tar två listor med samma längd och gör en lista av tupler där man parat ihop elementen på samma index med varandra är `zip` (finns dock redan implementerat i python). Man skriver den lätt med en listomfattning.

```
def zip(xs,ys):
    if len(xs) != len(ys):
        print("zip unequal lengths!")
        return []
    return [ (xs[i],ys[i]) for i in range(len(xs)) ]
```

5.2.1 dict-omfattningar

Listomfattning är den traditionella tekniken, men i Python har man generaliserat dessa till uppslagstabeller (och mängder som kommer i senare kapitel). Man kan alltså skapa uppslagstabeller med dict-

omfattning (eng. *dict-comprehensions*). Man måste i dessa fall se till att man skapar både nycklar och värden:

```
my_dict = {x: x ** 2 for x in (2, 4, 6)}  
print(my_dict)
```

```
{2: 4, 4: 16, 6: 36}
```

```
squares = {str(x) + '^2': x ** 2 for x in range(4)}  
print(squares)
```

```
{'0^2': 0, '1^2': 1, '2^2': 4, '3^2': 9}
```

```
lst = ['1', '42', '13']  
my_dict2 = {'key_' + x: int(x) for x in lst}  
print(my_dict2)
```

```
{'key_1': 1, 'key_42': 42, 'key_13': 13}
```

5.3 Felsökning

När man programmerar drabbas man av misstag, missförstånd, och felbedömningar. Det skapar fel i koden, som ibland kallas för “buggar” och är en försvenskning av det engelska ordet “bug” som betyder insekt, men har också använts som benämning på fel i tekniska system redan innan datorer började byggas. Buggar kan få program att krascha, det vill säga att programmet avbryts abrupt eftersom det inte går att fortsätta, eller bara orsaka ett felaktigt resultat som kanske inte ens märks. Man ska därför vara glad över felmeddelanden: det är ju betydligt värre med fel som man inte är medveten om.

Vi kan dela in programmeringsfel som språkliga misstag och körfel (“runtime errors”). Språkliga misstag fångas av kompilatorn och är oftast lätta att förstå när man har lärt sig programmeringsspråket. Det kan vara svårare med körfelen, eftersom man ofta upptäcker symptom, det vill säga konsekvenser av misstag eller problem. Det felmeddelande man får talar alltså inte alltid om vilket det egentliga felet är, utan är ett symptom på det egentliga felet. Som exempel: om ditt program kraschar på grund av att en uppslagstabell saknar den nyckel som används för en uppslagning, så är kanske uppslagningen i sig inte felet, utan det kan vara att fel nyckel används eller så har nyckeln inte har lagts in. Det är sällan rätt att “bota” symptom på fel när man programmerar, utan det är viktigt att lösa det underliggande problemet. I exemplet med uppslagstabeller är det därför inte en lösning att lägga till kod så att inget händer om en nyckel inte hittas i uppslagstabellen, utan man bör istället fastställa varför nyckeln används eller varför nyckeln inte finns inlagd.

5.3.1 Exempel på fel

Fel kommer ofta av missförstånd. Man kanske missförstår en detalj i en algoritm eller har en felaktig idé om hur indata faktiskt ser ut. Ibland har man missförstått det programmeringsspråk man arbetar i och det är lätt hänt eftersom det finns många detaljer att hålla reda på.

Vi kan titta på några exempel på återkommande fel, för att visa vad man kan behöva vara uppmärksam på.

5.3.1.1 Exempel: problem vid sortering

Antag att du behöver sortera en lista med tal och experimenterar med den här snutten kod:

```
lst = [10, 3, 4]
lst = lst.sort()
for elem in lst:
    print(elem)
```

Om man kör det här lilla programmet får man ett felmeddelande vid for-loopen: `TypeError: 'NoneType' object is not iterable`. Varför det? Det är ju uppenbart att det finns element i listan? Felet uppstår på grund av ett missförstånd om hur `sort` fungerar. Uttrycket `lst.sort()` returnerar `None`, men ändrar ordningen på elementen i `lst`, och då blir ju rad två i koden olycklig eftersom `lst` tilldelas `None`.

Lägg märke till att felet görs på rad två, men detekteras på rad tre. Det här är vanligt: när vi observerar fel så är det ofta symptom på fel som gjorts tidigare.

Ett sätt rätta till felet ovan är att byta till funktionen `sorted`:

```
lst = [10, 3, 4]
lst = sorted(lst)
for elem in lst:
    print(elem)
```

Detta fungerar eftersom `sorted` returnerar en ny kopia av listan, men sorterad.

5.3.1.2 Exempel: dåligt returvärde

En av våra tumregler är att man alltid ska returnera ett resultat från en funktion. Man ska ha goda skäl för att inte returnera något. Anledningen är att undvika ett vanligt nybörjarmisstag, som även rutinerade programmerare kan göra.

Antag att vi vill ha en liten funktion som tar fram de tre största värdena ur en lista. Vi provar oss fram och konstaterar att det löses enkelt med två rader kod:

```
lst = [10,3,4,5,1,2]
lst = sorted(lst)
print(lst[-3:])
```

```
[4, 5, 10]
```

Bra, då gör vi det till en funktion:

```
def largest_three(lst):
    lst = sorted(lst)
    print(lst[-3:])
```

Funktionen testas vi, som man bör, men då blir det fel.

```
a, b, c = largest_three([10,3,4,5,1,2])
```

```
[4, 5, 10]
```

Traceback (most recent call last):

File "<stdin>", line 1, in <module>

`TypeError: cannot unpack non-iterable NoneType object`

Resultatet skrivs ut, men tilldelningen av returvärdet fungerar inte, eftersom det inte finns en retur-sats, vilket ger returvärdet `None`. Man ska alltid returnera de intressanta värdena!


```
def largest_three(lst):
    lst = sorted(lst)
    return lst[-3:]
```

5.3.1.3 Exempel: iteratorer är inte listor

Inspirerat av lösningen ovan kanske man tänker sig en liknande uppgift, löst på liknande sätt. Antag att vi vill skriva ut en lista i omvänd ordning och se visa maxvärdet.

```
lst = [10, 3, 4]
lst = reversed(lst)
for elem in lst:
    print(elem)
print('Largest:', max(lst))
```

Nu blir det fel igen, på sista raden: `ValueError: max() arg is an empty sequence`. Varför det? Det blir inget fel om man använder `sorted` istället för `reversed`. Nej, men de två funktionerna har olika typer på sina returvärden. Med `sorted` får vi en lista och den kan man utan problem använda upprepade gånger, men `reversed` returnerar en iterator och dessa används en och endast en gång. När `for`-loopen är klar så är iteratorn ”förbrukad” och `max` har inget att arbeta med.

Det här är ett ganska vanligt misstag. Iteratorer används flitigt i Python, vilket är bra eftersom de är effektiva och minnessnåla, men man får vara uppmärksam på att en iterator behöver omvandlas till en lista ibland.

Hur rättar du felet?

5.3.2 Förstå felmeddelandet

Ett första steg till att bli bra på felsökning är att läsa felmeddelanden noga. Om programmet avslutar med ett fel så brukar felmeddelandet tala om ganska tydligt varför det avslutar. Återigen, detta är ofta bara symptom på det egentliga felet, men symptom är viktiga.

Felmeddelanden i Python har alltid samma struktur.

- Först visas var felet uppstår. Det görs genom att visa ett *stack trace* som anger vilka funktionsanrop som är involverade och i vilka filer och på vilka rader de anropas.
- Sist visas var felet uppstår, med filnamn och rad, och en enkel beskrivning av felet.

Betrakta den här konstiga koden, skriven enbart för att visa upp ett felmeddelande:

```
def f1(x):
    return f2(x)

def f2(x):
    return f3(x)

def f3(x):
    return 17 / x

f1(0)
```

Om man kör programmet anropas alltså $f1(0)$, som anropar $f2(0)$, som i sin tur anropar $f3(0)$ där division med noll uppstår. Det resulterar i det här felmeddelandet (åtminstone i Python 3.9):

```
Traceback (most recent call last):
  File "~/src/kod.py.py", line 11, in <module>
    f1(0)
  File "~/src/kod.py.py", line 3, in f1
    return f2(x)
  File "~/src/kod.py.py", line 6, in f2
    return f3(x)
  File "~/src/kod.py.py", line 9, in f3
    return 17 / x
ZeroDivisionError: division by zero
```

Ganska mycket text för ett enkelt fel, men det är illustrativt. Lägg särskilt märke till att det framgår på vilken rad det blir fel, här rad 9, och att det framgår att symptomet är just division med noll. Editorer som Spyder visar hjälpsamt just den problematiska filen och raden. Och tack vare "Traceback", som ger oss vårt *stack trace*, kan man utläsa hur funktionerna anropat varandra, vilket är ytterligare tips inför felsökningen.

En del felmeddelanden kan vara svåra att läsa, och för en nybörjare så kan viss terminologi vara alltför teknisk, men även om man inte förstår allt så kan man ändå hitta värdefulla ledtrådar.

5.3.3 Leta fel med spårutskrifter

Spårutskrifter är en beprövad och välanvänd teknik för att leta fel. Idén är enkel: lägg in `print`-satser som avslöjar värden på variabler och visar både hur variabelvärden ändras och hur programflödet går. Man kan se det som ett noggrant sätt att leta symptom som kan avslöja vad felet är.

Som exempel kan vi titta på exemplet om symbolisk derivering av polynom från avsnittet om Datarepresentation, men med ett fel insmuget:

```
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial: # Fel!
        new_coefficient = term * degree
        result.append(new_coefficient)
        degree += 1
    return result
```

Antag att vi har observerat att det blir något fel utan att lyckas lista ut vad det beror på. En enkel spårutskrift visar de relevanta variabelvärden i varje iteration:

```
def derivative(polynomial):
    result = []
    degree = 1
    print('degree term new_coefficient') # Praktisk rubrik
    for term in polynomial: # Fel!
        new_coefficient = term * degree
        print(degree, term, new_coefficient) # Spårutskrift
        result.append(new_coefficient)
```

```
degree += 1
return result
```

Denna spårutskrift skulle förhoppningsvis snabbt visa upp att vi av misstag fått med konstanttermen i deriveringen.

När man jobbar med spårutskrifter händer det lätt att man känner ett behov av att krydda med förklarande information. Här ovan var det till exempel bra att lägga in en rubrikrad. Behovet är så pass vanligt att Python har en särskild finess för spårutskrifter lite gömd i f-strängar: om du skriver `f'{x}'` så ersätts `{x}` av värdet på variabeln `x`, men om du skriver `f'{x=}'` så sätts istället `x=` och värdet på variabeln in. Så exemplet ovan kan bättre skrivas så här:

```
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial:    # Fel!
        new_coefficient = term * degree
        print(f'{degree=}, {term=}, {new_coefficient=}') # Spårutskrift
        result.append(new_coefficient)
        degree += 1
    return result
```

Prova! Om vi provkör detta med polynomet $1 + x + x^2$ så ser det ut så här:

```
derivative([1,1,1])
```

```
degree=1 term=1, new_coefficient=1
degree=2 term=1, new_coefficient=2
degree=3 term=1, new_coefficient=3
```

Här ska man se att de finns en term med grad 3, vilket är ju är fel eftersom vi har ett andragradspolynom som indata.

Spårutskrifter är bekvämt och praktiskt att använda, men betraktas ofta som dålig praxis. Nackdelarna med spårutskrifter är att det är arbetsamt och ibland onödigt att lägga in dem, man måste ju oftast ta bort dem också, men de används ändå vitt och brett av erfarna programmerare. I scriptspråk som Python är kostnaden med spårutskrifter ändå ganska låg eftersom man tenderar att arbeta ganska interaktivt med koden, mer än i vissa andra språk, och det är lätt att testa kod.

5.3.3.1 Loggning

Om man finner att spårutskrifterna är användbara, så kanske man entusiastiskt börjar planera för särskilda funktioner för spårutskrifter, avstängbara, där man kan få dem skrivna till en separat fil för spårutskrifter. I så fall har man börjat med loggning och det finns ingen anledning till att skriva egen kod för det. Det är ett så vanligt behov att det finns en standardmodul, med namnet `logging`, för ändamålet. Syftet med modulen är att göra det lätt att logga status på program, inte bara för att leta fel, utan mer allmänt för att hålla koll på vad ett system håller på med. Många datorsystem arbetar över lång tid med viktiga systemtjänster och då blir loggar ett viktigt verktyg för att hålla koll på vad som gjorts i ett system.

Här är ett exempel på hur deriveringskoden kan justeras till att logga status till filen `derivative.log`:

```

import logging
logging.basicConfig(filename='derivative.log', level=logging.DEBUG)
def derivative(polynomial):
    result = []
    degree = 1
    for term in polynomial:
        new_coefficient = term * degree
        logging.debug(f'{degree=}, {term=}, {new_coefficient=}') # Spårutskrift
        result.append(new_coefficient)
        degree += 1
    return result

```

De två första raderna initierar loggningen och man kan se vilken fil som loggmeddelanden hamnar i och vilken *nivå* som ska användas. Utan `level=logging.DEBUG` kommer debug-meddelanden och att vara utan effekt, men utskrifter gjorda med `logging.warning` eller `logging.error`, två andra utskriftsmetoder med högre prioritet kommer däremot skrivas ut.

Om man evaluerar koden ovan och sen `derivative([1, 1, 1])` kommer man återfinna dessa utskrifter i loggfilen:

```

DEBUG:root:degree=1, term=1, new_coefficient=1
DEBUG:root:degree=2, term=1, new_coefficient=2
DEBUG:root:degree=3, term=1, new_coefficient=3

```

I vänsterkanten anges vilket sorts loggmeddelande som getts ("DEBUG") och termen "root" som anger källan till meddelandet, vilket i vårt exempel inte har någon funktion. Modulen stödjer mer avancerad användning än vad vi visar här.

5.3.4 Leta fel med debugger

Programmerarens kanske bästa hjälpmedel är *debuggern*, på svenska även känd som *avlusare*. Det är ett program som används för att köra ditt program på ett kontrollerat sätt, med möjlighet att stega sig fram, rad för rad, och granska variablers värden utan spårutskrifter.

I editorer särskilt utformade för Python, som Spyder och PyCharm, är debuggern inbyggd. Andra editorer, till exempel VS Code, har tillägg som erbjuder stöd för debugger. I vissa verktyg finns kontrollerna för debuggern alltid tillgängliga, och i andra måste du särskilt instruera editorn att "nu vill jag debugga". Oavsett vilken editor och debugger som du använder så kan du räkna med att det finns flera återkommande och användbara finesser, och som dessutom återfinns för de flesta programmeringsspråk. Vi ska titta på några av de grundläggande finesserna.

5.3.4.1 Brytpunkter

En brytpunkt markeras ofta med en röd symbol i kodens vänsterkant och den betyder helt enkelt "stanna innan denna rad exekveras". Du kan alltså starta ditt program med debuggern och när körningen kommer till brytpunkten så pausar den. Körningen avbryts inte helt, utan kan återupptas när man är redo för det.

I pausen har du möjlighet att granska vilka värden som de olika variablerna har, vilket förhoppningsvis ger dig ledtrådar om felet. Att arbeta med brytpunkter och inspektion av variabler är förstås ganska likt att använda spårutskrifter, men fördelen är att du inte behöver ta bort spårutskrifterna när du är klar. Om ditt program dessutom har många variabler med lite komplicerade datastrukturer så brukar

brytpunkter vara enklare att arbeta med. En fördel är att du kan undersöka datastrukturer och behöver inte i förväg välja ut vad som ska visas.

5.3.4.2 Stegning

Brytpunkter ger dig ögonblicksbilder av programmet och det är inte alltid tillräckligt för att förklara vad som går fel. Därför är det praktiskt att kunna stega sig fram i programmet, rad för rad, och kontinuerligt inspektera variabler om så behövs eller helt enkelt bara se vilken väg i programmet man råkar ta.

Det finns två sätt att stega och du bör vara beredd att använda båda. I det enkla steget, ofta kallat *step* i en debugger och tillgängligt via en knapp, så är det alltid aktuell rad som exekveras. Om raden innehåller ett funktionsanrop så följer man koden in i funktionen, vilket ibland innebär att man följer körningen in i Pythons inre delar. Det kan vara både förvirrande och tidsödande. Det är sällan man behöver granska Pythons egen implementation för att förstå varför ens program gör fel. I sådana tillfällen är det bra att använda det andra sättet att stega, ofta kallat *next* eller *step over*. Med *next* så instruerar man debuggern att exekvera kodrader och fortsätta till nästa rad. Man hoppar alltså över detaljerna i den funktion som eventuellt anropas.

Det brukar också finnas möjlighet att säga "fortsätt till sista raden på funktionen" (i Spyder: "step return"), som är ett bra sätt att komma framåt på ett snabbare men fortfarande kontrollerat sätt.

5.3.5 Strategier för felsökning

Även om man behärskar tekniker för felsökning så kan det vara svårt eller långsamt att hitta det verkliga misstaget. Det är värt att tänka igenom hur man kan avslöja det underliggande felet.

I avsnittet Defensiv programmering ges tips om hur man kan använda `assert` för att lägga in villkor i sitt program. Villkoren ska beskriva utsagor som borde vara sanna där `assert`-raderna ligger. Om villkoret gjorts falskt av en bug så avbryts programmet.

Du kan också läsa om enhetstestning, som är ett sätt att systematiskt och automatiserat testa att programmets funktioner är korrekta. Om alla funktioner i ett program har enhetstester så får man hjälp att tidigt hitta misstag.

Med eller utan enhetstestning, det kommer alltid vara viktigt att använda bra strategier för att hitta sina fel. Vi ska nu gå igenom några strategier.

5.3.5.1 Avgränsa fel

Om ett program kraschar på rad 917 efter att ha exekverat ett flertal olika funktioner så kan misstaget ligga på just den raden eller raden innan, men man ska inte utgå ifrån att felet är nära rad 917. Fel kan propagera i flera led utan att man lägger märke till det. Det blir då viktigt att kunna lyfta blicken och titta längre tillbaka i programmet. Att då granska först rad 917, sen rad 916, och rad 915, och så vidare längre bakåt, är långsamt. Det är mer effektivt att, om felet inte ligger nära rad 917, försöka göra avgränsningar. Är indata till den senast anropade funktionen korrekta? Hur skapades dessa indata?

När felet är riktigt förvirrande svårt att hitta kan man behöva skapa "kontrollpunkter" i sitt program för att verifiera att beräkningarna är korrekta fram till dem. På så sätt kan man avgränsa och isolera källan till felet.

När man har ett fungerande program som man bygger ut eller förändrar så är förstås den nya koden huvudmisstänkt och ska, oavsett var de ligger, granskas först.

5.3.5.2 Skriv om

En del fel är alldeles framför ögonen, men man ser dem inte pga felaktiga antaganden eller bara ren förvirring. Förvirring och tillfällig blindhet för vissa misstag är att vänta, eftersom man ofta måste hålla många saker i huvudet när man programmerar.

En typ av misstag som uppstår lättare i scriptspråk (i motsats till språk som Java där man måste deklarerat sina variabler) är felstavning. I förenklad form kan det se ut så här:

```
identifier = 7           # A first assignment
idenfifier = identifier * 2 # Updating the assignment
print(identifier)
```

Varför skrivs värdet 7 ut? Ser du misstaget?

Om man kan lokalisera ett fel till en funktion, men ändå inte lyckas identifiera felet, så är en strategi att formulera om funktionen. Bara processen att skriva en funktion på nytt, kanske med andra identifierare eller med en lite annorlunda algoritm, kan bota den tillfälliga blindheten mot enkla misstag som stavfel, eller avslöja underliggande antaganden som man inte först reagerat på.

5.3.5.3 Granska indata

En del problem beror på att indata innehåller konstigheter som det kan vara svårt att lägga märke till. Moderna standarder har botat det här i hög grad, men man kan fortfarande stöta på problem som till exempel beror på att indata har skapats på en Mac men ska analyseras under Windows.

För det mesta kan man förlita sig på att textfiler använder teckenkodningen UTF-8, som är kompatibelt med äldre teckenkodningar som ASCII. Om man jobbar i äldre versioner av Python, version 2.7 och tidigare, så kan man inte förlita sig på att programmet automatiskt klarar UTF-8, och då kan till exempel svenska tecken bli konstiga. Teckenkodning ger alltså sällan problem.

Det är däremot lätt hänt att man gör antaganden om formatet på indata. Om mellanrum är viktigt, kodas det med ett blanksteg ("space") eller med ett TAB-tecken? En sån detalj är lätt att missa även när man tittar på indatafilen i en texteditor. Det kan därför hjälpa att skriva kod som hanterar både blanksteg och TAB-tecken.

Det är mycket vanligt att man flyttar filer mellan datorsystem, och då kan man upptäcka att radbrytningar hanteras olika på olika datorsystem. På Mac och Linux brukar ny rad kodas med ett tecken, '\n' i Python, men på en PC så används '\n\r', vilket läses som "new line and carriage return". På PC ser du alltså spår av de första datorskrivarna och deras koppling till mekaniska skrivmaskiner: först ska man mata fram papper till en ny rad, och sen ska skrivhuvudet flyttas längst till vänster.

Ett sätt att slippa bry sig om dessa detaljer är att förlita sig på etablerat kod för att läsa data, och det kan du läsa om i ett senare kapitel.

5.3.5.4 Välj dina testdata

När man letar fel så ska man välja sina testdata med omsorg.

- Arbeta med små testdata. Om ditt mål att analysera en stor datafil så bör du skapa mindre filer med liknande egenskaper för testning. Det snabbar upp felsökningen.
- Fel uppstår ofta på mer ovanliga indata, och därför är det bra att testa på specialfall som man konstruerar. Om du läser data från fil så bör din kod kunna läsa en tom fil utan att krascha. En funktion som arbetar med strängar bör klara den tomma strängen, '', utan problem. Om du

implementerar en numerisk funktion $f(x)$ så bör den fungera på $x = 0$ (om funktionen är definerad där).

Tänk också på att du antagligen kommer att behöva testa många gånger, och kanske även i framtiden. Det är därför bra konstruera testdata och spara dem för framtida behov.

5.4 Uppgifter

1. Vad händer om vi anropar funktionen `divide_by_elems` (funktionen given i avsnitt 5.1.2.3) med en sträng som argument för `x`? Skriv om funktionen så att om strängen kan göras om till ett heltal ska den fortsätta utan att krascha, annars ska den returnera `None`. Lös detta i funktionen med hjälp av `try` och `except`.
2. Skriv om funktionen `divide_by_elems` så att den inte bryter efter första fel, utan fortsätter att dela talet `x` med rader som kommer efter. Till exempel för filen `numbers.txt` som ovan ska funktionen returnera listan (och eventuellt skriva ut diverse meddelanden från felhanteringen):

```
[2.0, 1.0, 0.5, 0.6666666666666666]
```

3. Givet följande lista

```
xs = [2, -167, 12, 5676, -14, -7, 12, 12, -1]
```

Skapa med hjälp av en listomfattning listan `ys` som innehåller endast de positiva talen i `xs`.

4. Skapa med listomfattning en lista `ys` där alla talen i listan `xs` i uppgift 3 har konverterats till positiva om de är negativa (t.ex. `-167` blir `167`).
5. Givet funktionen nedan

```
def is_prime(x):  
    if x >= 2:  
        for y in range(2,x):  
            if not ( x % y ):   
                return False  
    else:  
        return False  
    return True
```

Skapa en lista med alla primtal under 100 genom att använda en listomfattning.

6. Givet följande lista

```
xs = [2, 3, 5, 7]
```

Använd listomfattning för att skapa en lista `ys` som innehåller produkterna av alla par av element från `xs`, dvs elementen `2 · 2`, `2 · 3`, `2 · 5`, `2 · 7`, `3 · 2`, osv.

7. Skapa med listomfattning en lista `ys` som innehåller produkter från multiplikation av alla parvisa element i `xs` från uppgift 6 utom elementen själva.

Obs: Talet 2 på index 0 i `xs` ska inte tolkas samma som 2 på index 1. Du kan alltså inte testa om talen är lika utan måste testa om index är samma.

Tips: använd `range`.

8. Skriv om labb 1 med exceptions så att programmet inte kraschar om man skriver in något fel.

Kapitel 6

Sekvenser och generatorer

6.1 Sekvenser

Python har stöd för ett antal olika sekvenstyper. Sekvenstyper i Python har gemensamt några vanliga operationer, t.ex. gäller:

- `+`: konkatenering/sammanslagning
- `*`: repetition
- `in` och `not in`: test om ett värde finns/inte finns som element i sekvensen
- `len()`: längden av sekvensen/hur många element
- `min()` och `max()`: minimum och maximum av element (om element-typerna är kompatibla med `<` och `>`)
- `[ : ]`: slicing för åtkomst av element
- m.fl.

Många av operationerna ovan känns förmodligen igen från `list`. Mycket riktigt är listor en av sekvenstyperna i Python. De andra typerna vi kommer att titta på i detta kapitel är tupler, range och strängar, vilka vi alla stött på tidigare i ett fall eller annat.

Sekvenstyper kan delas in i två underkategorier: *muterbara* och *icke-muterbara*. Med muterbarhet så menas att ett skapat objekt kan *ändras* på efter att det skapats. På samma sätt så är icke-muterbara objekt konstanta. Hur ett objekt används, och vilka potentiella sidoeffekter kod kan ha, kan bero på om ett objekt är muterbart eller inte.

För mer detaljerad information, se Python-dokumentationen om [sekvenstyper](#).

6.1.1 Listor

Listor är en sekvenstyp som är *muterbar*. Det är en viktig egenskap hos listor som betyder att vi kan uppdatera deras värden utan att kopiera listan. Vi har redan sett många exempel på vad som kan göras med listor, men för att poängtera vilken effekt det kan ha att listor är muterbara, betrakta på följande funktionsdefinition och kod:

```
def sum_of_increment(l, x):  
    for i in range(len(l)):  
        l[i] += x
```

```

ret_sum = sum(1)
return ret_sum

mylist = [3, 3, 4, 5]
print("mylist is:", mylist)
s = sum_of_increment(mylist, 2)
print("mylist after first call:", mylist)
print("First call returns:", s)
s = sum_of_increment(mylist, 2)
print("mylist after second call:", mylist)
print("Second call returns:", s)

```

```

mylist is: [3, 3, 4, 5]
mylist after first call: [5, 5, 6, 7]
First call returns: 23
mylist after second call: [7, 7, 8, 9]
Second call returns: 31

```

Resultatet blir olika varje gång funktionen kallas, även om funktionsanropet görs med samma variabler! Det här beror på att `mylist` ändras inuti funktionen, vilket går att göra eftersom listor är muterbara. Det här är också möjligt för att Python inte kopierar hela listan till funktionsargumentet (i det här fallet 1). Detta kan ibland vara användbart, men man bör vara medveten om och försiktig med detta, så att det inte får oanade konsekvenser.

För mer om listor och vilka funktioner som list-typen har, se Python-dokumentationen och [mer info om listor](#).

6.1.2 Tupler

Tupler liknar listor, men de är *icke-muterbara*. Så vi kan inte bara uppdatera en tupel som vi vill. Vi har sett tupler förr, även om vi kanske inte explicit skapat variabler av typen tupler: [multipla returvärden](#) från *funktioner* returneras nämligen som en tupel med flera element. Syntaxen för tupler är väldigt likt den som listor, med den skillnaden är att man använder `()` för att skapa en tupel istället för `[]`.

```

mytuple = (3, 3, 4, 5)
myothertuple = 3, 3, 4, 5 # equivalent syntax
print(mytuple)
print(mytuple == myothertuple)
print(mytuple[0])
print(mytuple[3:1:-1])
print(len(mytuple))
print(3 in mytuple)

# function returning multiple values
def integer_div(nominator, denominator):
    q = nominator // denominator
    r = nominator % denominator
    return q, r

divs = integer_div(17, 10)

```

```
print(type(divs))

# tuples are immutable
mytuple[0] = 42

(3, 3, 4, 5)
True
3
(5, 4)
4
True
<class 'tuple'>
TypeError: 'tuple' object does not support item assignment
```

Lägg märke till att slicing av en tupel ger tillbaka en tupel: slicing av en sekvenstyp ger tillbaka samma typ.

Om vi t.ex. skulle köra om kodexemplet från sektionen [Listor](#), med funktionen `sum_of_increment` definierad på samma sätt, fast med en tupel:

```
mytuple = (3, 3, 4, 5)
s = sum_of_increment(mytuple, 2)

TypeError: 'tuple' object does not support item assignment
```

Precis som listor, kan tupler innehålla olika datatyper och vi kan nästla dem:

```
t = (12345, 54321, 'hello!')
mytuple = (t, (1, 2, 3, 4, 5))
print(mytuple)

((12345, 54321, 'hello!'), (1, 2, 3, 4, 5))
```

Tupler kan innehålla *muterbara* objekt:

```
t2 = ([1, 2, 3], [3, 2, 1])
print(t2)

([1, 2, 3], [3, 2, 1])
```

Det sista exemplet `t2`, kan *inte* användas som nyckel i en uppslagstabell. Jämför t.ex.:

```
d = {t: 'hello'} # works
d2 = {t2: 'goodbye'} # TypeError: unhashable type: 'list'
```

Det här är som diskuterats tidigare för att nyckelvärden måste vara konstanta. I detta fall räcker det alltså inte med att själva tupeln är icke-muterbar eftersom den innehåller muterbara objekt (listor), vilket gör att `t2` *inte* kan användas som nyckel.

```
# t2[0] = [1, 2, 4] # throws TypeError, immutable
print(t2)
t2[0][0] = 42
print(t2)
```

```
([1, 2, 3], [3, 2, 1])
([42, 2, 3], [3, 2, 1])
```

6.1.3 range

Pythons range är också en *icke-muterbar* sekvenstyp. Den beter sig som en tupel, men den har inte stöd för vissa operationer, t.ex. + och *.

```
myrange = range(10)
print(myrange[3])
print(myrange[2:9:3])
print(4 in myrange)
print(42 in myrange)
myrange + myrange
```

```
3
range(2, 9, 3)
True
False
TypeError: unsupported operand type(s) for +: 'range' and 'range'
```

Återigen, slicing ger här tillbaka en range-typ.

6.1.4 Strängar

Strängar är även de ett exempel på en sekvenstyp, också *icke-muterbara*. Vi har precis som listor använt strängar flitigt. Det finns väldigt många användbara strängfunktioner; de flesta funktionerna har namn som tydligt säger vad de gör. Här nedan ges bara några exempel för vanligt förekommande strängfunktioner:

```
my_string = "Hello world! This is fun"
print(my_string.index("o"))
print(my_string.count("l"))
print(my_string.upper())
print(my_string.lower())
print(my_string.capitalize())
print(my_string[3:7])
print(my_string[3:7:2])
print(my_string[::-1])
print(my_string.startswith("Hello"))
print(my_string.endswith("asdfasdfasdf"))
print(my_string.split())
print(my_string.split("i"))
```

```
4
3
HELLO WORLD! THIS IS FUN
hello world! this is fun
Hello world! this is fun
lo w
l
```

```
nuf si sihT !dlrow olleH
True
False
['Hello', 'world!', 'This', 'is', 'fun']
['Hello world! Th', 's ', 's fun']
H
```

Glöm inte att man i Spyder lätt i t.ex. konsolen kan se vilka funktioner som är associerade med en sträng (eller generellt ett givet objekt) genom att använda sig av `dir()`. T.ex. kan man:

```
dir(my_string)
```

Detta kan ge lite “för mycket” information. Vill man ha en mer interaktivt sätt, kan man i konsolen skriva `my_string`. (lägg märke till punkten) och sen trycka TAB.

För mer om strängar och fler strängfunktioner, se [Python-dokumentationen](#).

6.2 Generatorer

Generatorer i Python är itererbara objekt som i varje iteration lämnar tillbaka ett värde till användaren. Vad värdet är beror på hur en generator har definierats. Vi har stött på flera itererbara objekt tidigare, t.ex. är alla sekvenstyper som precis beskrivits i [Sekvenser](#) itererbara.

Den största skillnaden mellan generatorer och sekvenstyper (förutom `range`) är när själva beräkningen av elementen sker: detta kan antingen ske *direkt* när man skapar objektet eller först när värdet på elementet *behövs*, d.v.s. när man väl utför en loop. Generatorer är ett exempel på en typ där värdet av ett element beräknas först när det behövs, detta är så kallad *lat evaluering*.

6.2.1 Lat och strikt evaluering

Om alla värden i ett objekt beräknas när objektet skapas kallar man det *strikt* eller *ivrig evaluering/beräkning* (eng. *eager evaluation*). Motsatsen är *lat* eller *selektiv evaluering* (eng. *lazy evaluation*) där värden beräknas när de behövs. Ett exempel med sekvenstyper som visar skillnaden:

```
mylist = [0, 1, 2, 3, 4, 5]
myrange = range(6)
print("mylist: ", mylist)
print("myrange: ", myrange)
```

```
mylist: [0, 1, 2, 3, 4, 5]
myrange: range(0, 6)
```

Här har vi skapat en lista `mylist`, som är ett exempel på strikt evaluering: alla värden som listan innehåller är här beräknade och sparade i minnet som utskriften visar. Utskriften av `range` å andra sidan visar bara just det, att det är ett objekt av typen `range` skapat med inparametrar `0, 6`. `range` är ett exempel på lat evaluering: alla värden som `myrange` innehåller *beräknas* varje gång man t.ex. gör `myrange[0]`. Lat evaluering kommer väl till hands när man använder sig av iteration, då det är mycket mer *minneseffektivt* än att iterera över ett objekt som är helt sparad i minnet.

`range` är lite av ett specialfall jämfört med de andra sekvenstyperna, just eftersom det är lat evaluering. Den finns för att det är en så vanlig struktur att ha en sekvens av ökande tal. Om man vill använda sig av lat evaluering, fast för mer komplicerade strukturer, så kommer generatorer väl till hands.

6.2.2 Generatorfunktioner

I Python kan man skapa en generatorfunktion på ett väldigt likt sätt som man skapar en vanlig funktion. När en vanlig funktion anropas, så exekveras all kod i funktionen fram tills att en `return`-sats stöts på, då ett värde (potentiellt) lämnas tillbaka till användaren och avslutar funktionskörningen. En generatorfunktion returnerar ett värde med `yield`, men funktionens *tillstånd sparas*. Nästa gång funktionen anropas så fortsätter funktionen från där den var sist tills *nästa* gång en `yield`-sats stöts på, vid vilken den lämnar tillbaka värdet och “pausar”.

Vi använder här funktionsanrop för generatorfunktioner lite löst. Egentligen kallas själva generatorfunktionen endast *en gång* och ger då tillbaka en generator. Det är denna generator som när den itereras över varje gång exekverar koden i generatorfunktionen.

Ett exempel visar nog detta bättre:

```
def inf_gen(start=0, step_size=1):
    """Infinitely generate ascending numbers"""
    while True:
        yield start
        start += step_size
```

Detta är en oändlig uppräknings av tal: en “oändlig range”. Hade vi försökt att använda oss av en lista, så inser man snabbt att det är svårt att skapa ett objekt med *oändligt* många element: vi hade fått slut på minne på datorn. Men representeras det som en generator är detta inget problem och vi kan fortfarande iterera över den. För att komma åt underliggande generator som man kan iterera över anropar man generatorfunktionen:

```
my_inf_gen = inf_gen(10, 1)
print(my_inf_gen)
print(next(my_inf_gen))
print(next(my_inf_gen))
print(next(my_inf_gen))
```

```
<generator object inf_gen at 0x7f1d7855be40>
10
11
12
```

Här har vi använt oss av `next`-funktionen med generatoren `my_inf_gen` som argument för att stega fram nästa värde. Det är faktiskt implicit `next` som anropas om man skulle iterera över `my_inf_gen` genom att t.ex. göra `for i in my_inf_gen: print(i)`. Vi kan inte göra det här, eftersom det skulle fortsätta i oändlighet, men testa gärna själv för att se vad som händer. (För att avsluta en oändlig loop kan man trycka CTRL+c)

Generators behöver självklart inte vara oändligt itererbara (som ovan). Som du ser kan du bestämma själv hur komplicerad en generator ska bli med nästan samma syntax som för en vanlig funktion. Iterationen över en generator avslutas när funktionskroppen körs utan att stöta på ett `yield`. Precis som man kan ha flera `return`-satser i en funktion, kan man ha flera `yield` i en generatorfunktion. Skillnaden är att funktionsblocket fortsätter från där det slutade vid senaste `yield`. En viktig detalj, och skillnad mot `range`, är att en generator bara kan itereras över *en gång*.

Vi illustrerar ovan koncept med en enkel generatorfunktion:

```
def my_generator(stop=5):
    i = 0
    j = 0
    while i + j < stop:
        print("i: ", i, "j:", j)
        i += 1
        yield i + j
        print("i: ", i, "j:", j)
        i += 1
        j += 1
        yield i + j
        print("i: ", i, "j:", j)

it = 1
gen = my_generator()
for i in gen:
    print("Iteration: ", it, "gen returns: ", i)
    it += 1

# loop a second time
for i in gen:
    print("Will this be printed??")
```

```
i: 0 j: 0
Iteration: 1 gen returns: 1
i: 1 j: 0
Iteration: 2 gen returns: 3
i: 2 j: 1
i: 2 j: 1
Iteration: 3 gen returns: 4
i: 3 j: 1
Iteration: 4 gen returns: 6
i: 4 j: 2
```

6.2.3 Generatoruttryck

Generatorer kan även skrivas på ett liknande sätt som listomfattningar. Det skapas då ingen lista utan istället får man en generator som man kan arbeta vidare med, framförallt iterera över med olika verktyg. Här skapas en lista och en generator.

```
big_list = [x**2 for x in range(10**8)]
big_generator = (x**2 for x in range(10**8))
print("Size of big_list in memory:", big_list.__sizeof__())
print("Size of big_generator in memory:", big_generator.__sizeof__())
```

```
Size of big_list in memory: 859724448
Size of big_generator in memory: 96
```

Som vi ser har listomfattningen genererat hela listan i minnet på datorn. Detta illustrerar vikten av att använda sig utav generatorer när vi ska iterera över många värden.

6.2.4 Exempel: Generator för alla primtal

Generatorer är smidiga att använda till vissa saker, bland annat oändliga sekvenser.

Låt oss säga att vi vill beräkna alla primtalsdelare till ett tal x . Vi kan då skapa en generator med alla primtal och se hur många gånger varje primtal delar x :

```
def is_prime(x):
    for t in range(2, x):
        if x % t == 0:
            return False
    return True

def prime_divs(x):
    primes = (i for i in inf_gen(2) if is_prime(i))
    ps = []
    for p in primes:
        if p > x:
            return ps
        else:
            while x % p == 0:
                ps.append(p)
                x //= p

print(prime_divs(2))
print(prime_divs(3))
print(prime_divs(24))
print(prime_divs(25))
print(prime_divs(232))
```

```
[2]
[3]
[2, 2, 2, 3]
[5, 5]
[2, 2, 2, 29]
```

6.3 Uppgifter

1. Vi såg att funktionen `sum_of_increment(1, x)` ändrade i vår lista när vi kallade på den flera gånger. Vi såg även att vi inte kunde kalla på `sum_of_increment(1, x)` med en tupel. Kan du av detta dra slutsats när det kan vara bra att använda sig av tupler?
2. Skriv en funktion `is_palindrome` som tar en sträng som input, och returnerar `True` om strängen är ett palindrom (d.v.s. om det är samma sak framlänges som baklänges) och returnerar den omvända strängen om det inte är ett palindrom.

Tips: använd sträng-slicing.

3. Fungerar `is_palindrome` som du skrev i uppgift 2 för listor och tupler? Varför?
4. Skriv en funktion `occurrences` som beräknar hur många gånger bokstäver förekommer i en

sträng. Funktionen ska returnera en uppslagstabell med bokstäverna som nycklar och antal gånger bokstäverna förekommer i strängen. Funktionen ska inte göra skillnad på versaler och gemener, utan räkna dessa gemensamt (för en given bokstav). Alla tecken som *inte* är bokstäver ska räknas under nyckeln 'non_alphas', och tecken som *inte* finns med i strängen ska inte finnas med i uppslagstabellen.

Tips: använd dig av de inbyggda strängfunktionerna. Se till exempel Python-dokumentation för [strängfunktioner](#)

Exempel:

```
s = 'Hello my very happy friend! Is the sun shining?!'
uo = occurrences(s)
print(s)
for k, v in uo.items():
    print(k, ":", v)

s = 'JjJJJjj hHhh !/;:(+[//==*])'
uo = occurrences(s)
print("\n" + s)
for k, v in uo.items():
    print(k, ":", v)
```

```
Hello my very happy friend! Is the sun shining?!
h : 4
e : 4
l : 2
o : 1
non_alphas : 11
m : 1
y : 3
v : 1
r : 2
a : 1
p : 2
f : 1
i : 4
n : 4
d : 1
s : 3
t : 1
u : 1
g : 1

JjJJJjj hHhh !/;:(+[//==*])
j : 7
non_alphas : 17
h : 4
```

5. Använd `occurrences(s)` ovan för att skriva ett program som läser in en fil och skriver ut bokstäverna som nycklar och antalet gånger bokstäverna förekommer för hela filen. För att testa kan ni

använda filen `palindrome.txt` med innehåll:

```
not palindrome line
palindrome line enil emordnilap
no way
hello ! olleh
Tomorrow will be a glorious day!!!
oooooooooooooooooooooooooooo
rust is also a programming language
oooooooooooooooooooooooooooo
```

Tips: läs in hela filinnehållet som en sträng.

6. Skriv en generatorfunktion `fibonacci(stop=x)` som genererar Fibonacci tal (se <https://sv.wikipedia.org/wiki/Fibonaccital>) mindre eller lika med `x`.

Tips: Använd dig utav strukturen i funktionen `my_generator` ovan.

7. I första raden i funktionen `prime_divs` i [Exempel: generator för alla primtal](#) så använde vi oss av ett generatoruttryck för att "filtrera" alla tal ur en generator som var primtal i en ny generator. Man kan för sin filter-generator alltså skriva:

```
filter_gen = (i for i in iterable if function(i))
```

Här är `function` en (användardefinierad) funktion som ger tillbaka `True` eller `False` när `i` ges som argument, och `iterable` är ett itererbart objekt (t.ex. en generator som vi hade i `prime_divs`). Alla `i` som ger `function(i) == True` kommer vara en del av `filter_gen`. Python har redan implementerat detta i funktionen `filter`, som du kan läsa dokumentationen om. En liknande funktion är `map` (se dokumentationen) där generatoren som skapas bör innehålla *värdena* som en funktion `f` *returnerar* när `f` appliceras på alla element i ett itererbart objekt.

Skriv en egen funktion `map_gen` som åstadkommer detta genom att ta en *godtycklig funktion* som argument och ett itererbart objekt, och som *returnerar* en generator.

Exempel:

```
gen = map_gen(int, '123456')
sum(gen)
```

```
21
```

8. Skriv en funktion `palindrome_rows` som tar ett filnamn som argument, och kollar varje rad i filen om det är ett palindrom eller inte. `palindrome_rows` ska resultera i en generator som man kan iterera över när funktionen anropas.

Tips: tänk på att rader läses in *inklusive* ny-rad-tecken. Modifiera `is_palindrome` från uppgift 2 så att den bara lämnar tillbaka `True` eller `False` baserat på om strängen är ett palindrom (alltså inte lämnar tillbaka själva strängen baklänges om det *inte* är ett palindrom).

Exempel med filen `palindrome.txt` ovan:

```
p_rows = palindrome_rows('palindrome.txt')
i = 1
for p in p_rows:
    print("Row", i, "palindrome?", p)
    i += 1
```

```
Row 1 palindrome? False
Row 2 palindrome? True
Row 3 palindrome? False
Row 4 palindrome? True
Row 5 palindrome? False
Row 6 palindrome? True
Row 7 palindrome? False
Row 8 palindrome? True
```

9. Du “vet” att det i en fil `almost_lowercase.txt` som ska bestå av endast gemener och tecken gömmer sig *ett enda* versalt ord, men du vet inte vilket det är. Det här tycker du är extremt jobbigt! Använd dig av `open` tillsammans med strängfunktioner för att *utan en loop* ta reda på vilket ord det är. Du kan anta att ett “ord” avgränsas av mellanrum och att alla övriga ord (utöver det du letar efter) består endast av gemener och/eller tecken.

Tips: någon av `strip`-funktionerna tillsammans med indexering kan komma till användning här.

Exempel på `almost_lowercase.txt`:

```
this is a file that /should/ consist of only lowercase letters.
but it doesn't!
there is a hidden upppercase word right HERE that you are supposed to find.
```

Kapitel 7

Moduler, bibliotek och programstruktur

7.1 Moduler

Många programmeringsspråk har ett “modulsystem”, så även Python. Dessa är smidiga när man skriver större program och egna bibliotek. De kan användas för:

- Kodåtervinning – importera kod som andra programmerare (eller du själv) har skrivit.
- Struktur – det är svårt att arbeta med mycket kod i en fil, så sprid istället ut den över flera filer.
- Samarbete – det är mycket lättare att skriva kod tillsammans som ett team om den är uppdaterad i flera filer så att alla inte behöver ändra i samma fil.

Med hjälp av moduler kan du börja förstå ett stort projekt som bestående av självständiga enheter. Du ska inte behöva förstå alla detaljer i en modul, det ska räcka att förstå dess *gränssnitt*. Gränssnittet är de funktioner och variabler som är tänkta att vara synliga för andra programmare som använder sig av modulen.

Stora program blir snabbt överskådliga. Väl valda moduler kan då bli viktiga för att bättre förstå ditt och andras program.

En teknisk fördel med moduler är att de hjälper dig att hantera *namnrymden* (eng. *name space*). Många kan vilja använda samma identifierare för t.ex. funktioner i ett stort program (exempelvis `sort`). Moduler döljer implementationen och variabel-/funktionsnamn.

Principer:

- Varje Python-fil är en modul.
- Allt som ska användas (från en annan fil) måste *importeras*.

7.1.1 Import av moduler

Man kan importera moduler på ett antal olika sätt:

- **import** modulename: Tillåter att funktioner/variabler används med punktnotation (dvs allt från modulename är importerat “qualified”).

```
import math
math.ceil(3.5) # Avrunda uppåt
ceil(4.4) # Fel!
```

- **from** modulename **import** f1, f2, ...: Tillåter användning av f1 och f2 i ditt program utan punktnotation.

```
from math import ceil
ceil(3.5)
```

- **from** modulename **import** *: Gör allt från modulename tillgängligt utan punktnotation.

Varning: detta anses orsaka dålig läsbarhet och det blir även svårt att veta vilka delar av en modul en annan modul beror på.

Obs: identifierare som börjar med `_` importeras inte. De anses "privata" till modulen.

Notera: vissa moduler bygger på att man `*-importerar`, då de *behöver* (eller är skrivna så att) skriva över inbyggda funktioner i Python för att fungera.

- **import** modulename **as** X: Lättare hantera långa modulnamn när man använder punktnotation. Exempel:

```
import math as M
M.ceil(3.4) # Praktiskt för långa modulnamn
```

- **from** modulename **import** X **as** Y: För att undvika namnkrockar. Exempel:

```
from math import gcd as libgcd

def gcd(a,b):
    r = a % b
    if r == 0:
        return b
    else:
        return gcd(b, r)

for a in range(2,10):
    for b in range(2,a):
        print(gcd(a,b) == libgcd(a,b))
```

- Moduler kan grupperas i *paket* (eng. *packages*), vilka ska förstås som hierarkiskt ordnade grupper av moduler.

```
import os.path as OS
if OS.isfile('data.txt'):
    ...
```

Vilket sätt att importera moduler ska man då använda? Det beror förstås på omständigheterna. Ett praktiskt exempel finns här nedan, där en funktion för att beräkna binomialkoefficienter definieras på fyra olika sätt. Man brukar dock rekommendera att inte använda `from m import *` eftersom risken för namnkrockar ökar och det blir otydligt vad du använder från den importerade modulen.

(a)

```
import math
def binom1(n, k):
    return math.factorial(n) / (math.factorial(k) * math.factorial(n-k))
```

(b)

```
import math as m
def binom2(n, k):
    return m.factorial(n) / (m.factorial(k) * m.factorial(n-k))
```

(c)

```
from math import factorial
def binom3(n, k):
    return factorial(n) / (factorial(k)*factorial(n-k))
```

(d)

```
from math import factorial as fac
def binom4(n, k):
    return fac(n) / (fac(k)*fac(n-k))
```

Vilket alternativ av (a) till (d) är mest lättläst, tycker du? Tips: modulen `math` har en funktion `comb` som är snabbare än den som presenteras här.

7.1.2 Skriva egna moduler

Låt oss säga att vi har följande Python kod i en fil `circle.py`:

```
pi = 3.14

def area(radius):
    return pi * (radius ** 2)

def circumference(radius):
    return 2 * pi * radius
```

Vi kan då importera den i en annan fil:

```
import circle

print(circle.pi)
print(circle.area(2))

import circle as C

print(C.pi == circle.pi)

tau = C.pi * 2

def circumference(radius):
    return tau * radius
```

```
print(C.circumference(2) == circumference(2))
```

Notera 1: filen `circle.py` måste hittas av Python när programmet letar efter potentiella moduler att importera. Den första platsen Python letar på är i samma mapp som det `program/.py`-fil man kör ligger i; det enklaste sättet är därför att ha sin modulfil (här `circle.py`) i *samma* mapp som huvudprogrammet/filen som importerar modulen.

Notera 2: för att använda ett värde på π så importerar man normalt `math.pi`, som kommer med bra precision.

```
from math import pi
print(pi)
```

```
3.141592653589793
```

7.1.3 `main()`-funktioner

Ett vanligt idiom:

```
print("Hello")

def main():
    print("python main function")

if __name__ == '__main__':
    main()

print("__name__ value: ", __name__)
```

Om och endast om man startar programmet med denna fil så anropas `main`. Om man använder filen som en modul så körs inte `main`. Du kan alltså skriva moduler som innehåller exempelprogram och/eller testkod som inte påverkar vad som händer vid `import`. Anledningen att `main` inte körs om man importerar filen som en modul är att variabeln `__name__` sätts till modulnamnet, så `if`-satsen kommer då inte att köras.

7.2 Populära bibliotek och moduler

Python kommer laddat med batterier: det finns massor av bra och användbara moduler som är en del av Python-implementationen. Dessa moduler kallas tillsammans Pythons standardbibliotek. För en lång lista samt dokumentation se: <https://docs.python.org/3/library/>.

- `math` – Matematik: trigonometri, logaritmer, osv.
- `sys` – Avancerad filhantering, med mera...
- `os` – Undvik att vara beroende av Win/Mac/Linux (typ undvik problem med `"/` i filnamn i Linux/Mac vs. `"\"` i filnamn för Windows).
- `argparse` – Används för parametrar när man startar program på kommandoraden.
- `itertools` – Bra för avancerade och snabba iterationer.
- `functools` – Fler högre ordningens funktioner.

- `datetime` – Datum och tidstyper.
- `pickle` – Serialisering av Python objekt.
- `csv` – Det är svårare än du tror att läsa en komma-separerad fil!
- `json` – Spara data i JSON-format för att lätt kunna läsa in igen.
- `matplotlib` – Visualisera data.
- `numpy` och `scipy` – För beräkningar.
- ...: Utforska själv!

7.2.1 JSON och serialisering

Problem: Jobbigt att skriva och läsa filer! Särskilt med stora datastrukturer (tänk: nästlade listor och/eller uppslagstabeller).

Lösning: *serialisering*

Idé: Dumpa datastruktur på fil i standardformat som lätt kan läsas in igen.

Finns standard pythonmoduler för detta (`marshal` och `pickle`).

Bra portabel lösning: JSON – JavaScript Object Notation

```
import json
legends = [ ['Lovelace', 'Ada', 1815]
            , ['Babbage', 'Charles', 1791]
            , ['Beurling', 'Arne', 1905]]

# Skapa datafil:
with open('legends.json', 'w') as outfile:
    outfile.write(json.dumps(legends))

# Senare... Skriv ut födelsesår:
with open('legends.json', 'r') as data_h:
    external_data = json.load(data_h)
    for x in external_data:
        print(x[2])                # Print birth year
```

7.2.2 NumPy: för vektorer och matriser

Med `numpy` (<https://www.numpy.org/>) får du ett paket för vetenskapliga beräkningar i Python. Det kommer med mycket effektiv lagring av vektorer och matriser, m.h.a. “arrays”. Pythons list har gränssnitt som (bl.a.) en array, men är mycket mer flexibelt. Det kostar dock i effektivitet. Arrayer i `numpy` är närmare datorns egen representation, vilket gör beräkningar mer effektiva (bättre användning av minne och beräkningstid). Effektiviteten kommer på bekostnad av flexibilitet, men i många tillämpningar är det mer än väl värt.

Skapa vektorer och matriser med `numpy`-arrayer:

```
import numpy as np          # idiomatisk Python att importera numpy som np

a = np.array([1,2,3,4])    # Från vanlig lista
```



```

b = np.ones((2,2))      # 2x2-matris med ettor, notera tupeln!
c = np.zeros((10,10))   # 10x10-noll-matris
d = np.linspace(0, 1, 16) # 16 element från 0 till 1, flyttal!
e = np.reshape(d, (4,4)) # Ger 4x4-matris med tal från 0 till 1
f = np.identity(2)       # Enhetsmatris, 2x2

print(a)
print('Only ones:', b)
print('Large matrix of zeros:', c)
print(d)
print(e)
print('Unity matrix:', f)

```

```

[1 2 3 4]
Only ones: [[1. 1.]
 [1. 1.]]
Large matrix of zeros: [[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]]
[[0.         0.06666667 0.13333333 0.2         0.26666667 0.33333333
 0.4         0.46666667 0.53333333 0.6         0.66666667 0.73333333
 0.8         0.86666667 0.93333333 1.         ]
 [[0.         0.06666667 0.13333333 0.2         ]
 [0.26666667 0.33333333 0.4         0.46666667]
 [0.53333333 0.6         0.66666667 0.73333333]
 [0.8         0.86666667 0.93333333 1.         ]]
Unity matrix: [[1. 0.]
 [0. 1.]]

```

Notera att np är ett etablerat alias, och import görs oftast som i exemplet, inklusive i numpy-dokumentationen.

Tips: prova exempelvis `help(np.linspace)` för att se hur den fungerar.

Man har naturligtvis tillgång till operationer på matriser, som t.ex. matrismultiplikation:

```

m = np.array([[1,2], [3,4]])
v = np.array([1, 0.5])
product = np.matmul(m, v)

```

```

[[2. 5.]]

```

numpy betraktar generöst en array som en vektor på ett sätt som passar in. Här ovan tolkas alltså `v` som en kolonnvektor, och om man istället skriver `np.matmul(v, m)` tolkas den som radvektor.

Det finns en särskild datastruktur för matriser definierad i numpy, men den bör man idag undvika eftersom den planeras att plockas bort: rekommendationen är att alltid använda `np.array`.

numpy är en del av scipy (<https://www.scipy.org/>) som innehåller en massa olika paket för att göra vetenskapliga beräkningar och hantera “stor” data med Python.

7.3 Tumregler för programstruktur och bra kod

Det är svårt att skriva bra kod. Med “bra” kod menas kod som är lätt att läsa, förstå sig på, använda och underhålla. Genom att skriva bra kod kan vi även undvika att introducera buggar. Det är dock svårt att säga exakt vad som gör kod bra, men förhoppningsvis har ni fått se lite exempel på både bra och mindre bra kod under kursens gång. Många programmerare har olika definitioner på vad som gör kod bra (på samma sätt som om det är väldigt subjektivt vad som är “bra konst” eller “bra litteratur”).

Man kan hitta en massa specifika Python-tips på: <https://docs.python-guide.org/writing/style/>. Det finns en stilguide, med det stela namnet [PEP-8](#), som är skriven av Guido van Rossum, skaparen av Python. När du börjar programmera i andra språk är det en bra idé att se vilka principer som gäller där.

Det finns verktyg för automatisk granskning av kod. Ett sådant finns på nätet: <https://www.pythonchecker.com/>. Det finns också IDE:er med liknande stöd, och bibliotek som `pylint` som man kan köra i ett terminalfönster.

7.3.1 Programmeringsriktlinjer

Här kommer en lista (i ingen specifik ordning) över saker som vi tycker gör kod bättre:

- Undvik långa rader, mindre än 79 tecken per rad är bra. Om en rad är för lång är det väldigt svårt att läsa den. [PEP-8](#) kräver denna begränsning.
- En kodsnuitt per rad. Följande kod är dålig:

```
print('one'); print('two')

if x == 1: print('one')

if <complex comparison> and <other complex comparison>:
    # do something
```

Följande kod är mycket bättre:

```
print('one')
print('two')

if x == 1:
    print('one')

cond1 = <complex comparison>
cond2 = <other complex comparison>
if cond1 and cond2:
    # do something
```

- Eftersträva att skriva korta funktioner. De flesta funktioner bör få plats på en halv laptopskärm med teckenstorlek 12. I vissa fall kan de självklart bli längre, men tänk då noga på om det går (eller

blir tydligare) att faktorisera ut koden i mindre funktioner först. Om funktioner är väldigt långa är det svårt att få överblick och hålla reda på allt funktionen gör.

- Dela upp kod i funktioner som gör **en** sak och gör det bra. Detta gör det mycket lättare att läsa, testa och förstå koden.

Exempel: “läs in flera datamängder, gör beräkningar på varje datamängd, skriv ut en sammanfattning” bör delas upp på följande sätt:

- En funktion för inläsning av datamängder.
 - En funktion som gör beräkningen på en datamängd.
 - En huvudfunktion som anropar de två andra funktionerna och skriver ut sammanfattningen.
- Funktioner bör returnera något. Det blir då möjligt att testa dom och det är lättare att förstå vad de gör eller förväntas göra. Det finns dock tillfällen då en funktion inte behöver returnera något (utskrifter, listmanipulation, osv), men förväntningen bör ändå vara att funktioner returnerar något. Ett vanligt nybörjarfel är att man använder globala variabler för att hantera dataflödet i sitt program och då “behövs inte” returvärden. För nybörjaren kan alltså brist på returvärden vara en indikation på dålig programstruktur.
 - Varje funktion ska ha en dokumentationssträng.
 - Separera **beräkning** från **interaktion**: funktioner som gör beräkningar bör inte ha print/input. Funktioner som hanterar interaktion bör inte göra komplexa beräkningar. Interaktion är såväl enkla utskrifter som frågor till användaren. Man vill ofta återanvända beräknande funktioner, men inte deras diagnostiska utskrifter.
 - Tänk först på hur programmet bör struktureras och vilka abstraktioner som bör användas innan ni börjar koda. Gruppera funktionalitet och attribut i klasser¹—på så sätt uppnås abstraktion och det blir lättare att ändra i programmet senare.
 - Dela upp i små enheter, d.v.s. dela upp i lämpliga funktioner och klasser. Detta kallas för *modulär* kod då varje logisk del i koden kan studeras som en egen liten modul som sedan kombineras med andra moduler för att skapa komplexa program. Det gör programmet mycket mer överskådligt och lättförståeligt.
 - Returnera **inte** värden av typ `int` eller `bool` som strängar, d.v.s. som `'2'` eller `'True'`. Det gör det svårare att testa och använda funktionerna senare. Det är även lätt att introducera buggar på så sätt, exempelvis:

```
def f(x):  
    return '2'  
  
if f(1) == 2:  
    print("This is not printed!")  
else:  
    print("This is printed!")
```

- Returnera inte “magiska” värden för att markera fel eller problem. Använd hellre särfall. Exempelvis: “return 1000000” är dålig markör för “konvergerade inte”. **Undantag**: None kan vara ett acceptabelt magiskt värde.
- Multipla returvärden är praktiska! Bättre än strängar som innehåller flera värden.

¹Klasser introduceras i kapitel 9.

- Variabler/konstanter är bättre än hårdkodade litteraler. Till exempel om vi skriver ett grafiskt program och vill ha ett fönster med storlek 600 x 600, då är det bättre att först skriva

```
width , height = 600 , 600
```

och sen använda width/height istället för att upprepa 600 överallt. På så sätt blir det lättare att ändra programmet i framtiden om vi vill ha andra dimensioner på fönstret. Det blir också lättare att läsa koden om det står width istället för ett godtyckligt nummer.

- Undvik globala variabler då de ofta leder till “spagettikod” (d.v.s. rörig kod där vitt skilda delar av programmet kan påverka varandra). Man ska kunna förstå en funktion utan att titta på andra delar av ett program. Det är direkt olämpligt att manipulera globala variabler i funktioner (d.v.s. använda nyckelordet **global**).
- Inkludera tester för att både testa och specificera hur program är tänkta att fungera. Detta är lättare att göra om programmet är uppdelat i mindre funktioner som gör **en** sak och **returnerar** den.
- Enkel kod är lättare att läsa. Undvik därför att skriva onödigt komplicerade kodsnuttar. Till exempel är “**if** b == **True**.” mer komplicerat än bara “**if** b:”. Det är som med vanlig svenska: det är dålig stil att stapla många små ord på varandra. På samma sätt som enkla meningar blir lättare att förstå, så är enkel kod lättare att förstå.

Här är ett exempel på ett onödigt komplicerat program:

```
def make_complex(args):
    x, y = args
    d = dict()
    d['x'] = x
    d['y'] = y
    return d
```

Mycket bättre:

```
def make_complex(x,y):
    return {'x': x, 'y': y}
```

Ett bra knep är att alltid läsa igenom sin kod flera gånger och reflektera över hur den kan förenklas.

- Var försiktiga med muterbara datatyper (många buggar uppstår då listor råkas tömmas genom exempelvis `pop()`!). Speciellt farligt är det att ha muterbara typer som globala variabler eller klassattribut.
- Dela upp större projekt i logiska enheter i olika filer/mappar.
- Det ska framgå av identifieraren vad en variabel lagrar eller vad en funktion är till för. Som exempel, om du har raden

```
x = 17 # critical value, abort if exceeding
```

så är det nog bättre att byta x mot något annat, kanske så här:

```
critical_value = 17 # abort if exceeding
```

Ett till exempel: istället för

```
nc = 1.7 # normalization constant
```

bör man skriva:

```
normalization_constant = 1.7
```

Här behövs det då ingen kommentar.

- Det finns tillfällen när funktionsnamnen får vara oinformativa: t.ex. `main`, pga konventioner
- Undvik generiska identifierare. Använd identifierare som är beskrivande.
 - *Dåliga*: `start`, `compute`, `f`, `foo`
 - *Bra*: `remove_outliers`, `newton_raphson`, `compute_integral`
- Kod ändras. Om funktionens syfte ändras så bör du byta identifierare! Om din funktion heter `print_integral` men inte gör några utskrifter så är det förvirrande.
- Kommentarer är bra, men ska inte ersätta enkel kod med bra namn på identifierare. Om koden behöver kommenteras för att vara förståelig så betyder det troligtvis att man gör något komplicerat. Bra användning av kommentarer är att förklara varför man använder en specifik datastruktur (t.ex. varför man använder ett set istället för `list`, etc.) eller hur det är tänkt att en funktion ska användas.

Kommentarer som inte bidrar något mer än vad koden redan säger bör undvikas. Till exempel är följande en helt redundant kommentar:

```
rows = [] # Create an empty list
```

Alla som kan lite Python förstår ju att kodraden skapar en tom lista, så varför skriva det i en kommentar?

7.3.2 Tecken på mindre bra kod

- “Copy-paste-kod”: det är lätt att introducera buggar om man kopierar rader och klistrar in dem igen. Programmet blir även svårsläst om det blir väldigt långt. Kan man introducera en loop eller funktion istället?
- Är det svårt att skapa bra tester? Då har du nog inte delat upp ditt program i små delar som gör **en** specifik sak. Tänk över designen och introducera bättre funktioner och abstraktioner.
- Djup indentering: om man har väldigt nästlade **if**-satser så är koden svårsläst och bör delas upp i mindre delar. En vanlig orsak till djup indentering är att man har flera olika logiska fall—skapa då funktioner för de olika fallen. På så vis blir fallanalysen tydligare och de olika funktionsanropen indikerar vad som sker i fallen.

Här är ett exempel på “dålig” kod:

```
# Print the prime numbers from a collatz sequence
# for 100 consecutive starting numbers
for i in range(1,100):
    n = i
    primes = []
    while n != 1:
        is_p = True
        for x in range(2, n):
            if not (n % x):
                is_p = False
```

```

        if is_p:
            primes.append(n)
        if n % 2 == 0:
            n = n//2
        else:
            n = 3*n + 1
    print("Start number :", i, "Primes in Collatz sequence: ", primes)

```

Detta kodblock är indenterat i fyra nivåer. Det är inte så farligt, men man kan strukturera det bättre:

```

def collatz_primes(n):
    """Computes prime numbers from
    the collatz sequence for an integer n.
    Parameters:
    n (int): input number
    Returns:
    primes: list of prime numbers in collatz sequence for n
    """
    primes = []
    while n != 1:
        is_p = True
        for x in range(2, n):
            if not (n % x):
                is_p = False
        if is_p:
            primes.append(n)
        if n % 2 == 0:
            n = n//2
        else:
            n = 3*n + 1
    return primes

for i in range(1,100):
    primes = collatz_primes(i)
    print("Start number :", i, "Primes in Collatz sequence: ", primes)

```

Här har vi brutit ut en särskild funktion, `collatz_primes(n)`, som används i en `for`-loop, och den funktionen har bara tre indenteringsnivåer (`while`-slingan, `for`-slingan och `if`-satsen).

Vi kan ta det ett steg till och lägga uppdateringen av variabeln `n` och beräkningen om det är ett primtal i egna funktioner:

```

def collatz_update(n):
    if n % 2 == 0:
        return n//2
    else:
        return 3*n + 1

def is_prime(n):

```

```

if n < 2:
    return False
if n == 2:
    return True

for y in range(2,n):
    if not (n % y):
        return False
return True

def collatz_primes(n):
    """Computes prime numbers from
    the collatz sequence for an integer n.
    Parameters:
    n (int): input number
    Returns:
    list: list of prime numbers in collatz sequence for n
    """
    primes = []
    while n != 1:
        if is_prime(n):
            primes.append(n)
        n = collatz_update(n)
    return primes

for i in range(1,100):
    primes = collatz_primes(i)
    print("Start number :", i, "Primes in Collatz sequence: ", primes)

```

Vilken version tycker du är lättast att läsa och förstå?

- Saknas det parametrar och **return** i funktioner som ska göra beräkningar? Då gör de troligtvis en massa konstigheter. Funktioners syfte är vanligen att ta indata och beräkna något. Beräkningen behöver vara inte vara matematisk utan kan vara enkel behandling av data. Det är mindre vanligt att funktioner helt saknar parametrar, men det finns förstås undantag, till exempel för utskrift av en meny eller initialisering av datastrukturer. Hos nybörjare är det dock inte ovanligt att man finner det bekvämt att lägga data i en eller flera globala variabler som funktioner sedan arbetar med. En sån konstruktion gör det svårt att se hur funktioner beror på varandra; man måste läsa funktionerna noggrant för att identifiera programflödet. Man kan sen inte heller använda funktionen i en annan modul om den inte tar någon parameter.

Koden nedan fungerar om vi lägger `is_prime` i annan modul `m` (d.v.s. fil) och kallar på den med `m.is_prime(x)`.

```

def is_prime(x):
    if x >= 2:
        for y in range(2,x):
            if not (x % y):
                return False
    else:

```

```

        return False
    return True

def main(x):
    print("The number is prime:", is_prime(x))

x = int(input("Provide a number to analyze: \n"))
main(x)

```

Koden nedan är sämre skriven och fungerar endast om vi har den globala variabeln `x` i samma fil. Detta gör att vi inte kan importera funktionen i en annan fil.

```

def is_prime():
    if x >= 2:
        for y in range(2,x):
            if not (x % y):
                return False
    else:
        return False
    return True

def main(x):
    print("The number is prime:", is_prime())

x = int(input("Provide a number to analyze: \n"))
main(x)

```

7.4 Uppgifter

1. Importera biblioteket `random` (se <https://docs.python.org/3/library/random.html>) för att generera 10 slumpnässiga *heltal* mellan 1 och 100 och 10 slumpnässiga *flyttal* mellan 1 och 100.

Tips: Läs dokumentationen. Vilka funktioner är lämpligast att använda?

2. Betrakta koden nedan som läser in ett tal och avgör om det är ett perfekt tal eller ett primtal. Skriv om koden så att den får bättre struktur.

Tips: Dela upp koden i funktioner. Vilka bitar i koden tenderar att bli naturliga funktioner? Vad bör funktionerna ta för parametrar och returnera för att vara återanvändbara utanför modulen? Ha gärna en huvudfunktion `main(x)` som tar talet `x` som parameter.

```

# checks if a number is (1) perfect (2) prime
x = int(input("Provide a number to analyze: \n"))
sum = 0
for i in range(1, x):
    if (x % i == 0):
        sum = sum + i

if x >= 2:
    prime = True

```



```

    for y in range(2,x):
        if not (x % y):
            prime = False
    else:
        prime = False

    if sum == x:
        print("The number is perfect")
    else:
        print("The number is not perfect")
    if prime:
        print("The number is prime")
    else:
        print("The number is not prime")

```

3. Lägg till i ditt omstrukturerade program från uppgift 2 valet att låta användaren bestämma om hen ska ange ett tal eller om programmet ska slumpa fram ett tal. Var i programmet bör vi placera denna nya kod?
4. Nedan kod läser in en DNA-sträng (t.ex. 'AGCTAGCGGTAGC') och letar först upp den vanligast förekommande strängen av längd k. Sedan skriver programmet ut alla parvisa avstånd mellan delsträngen på vår DNA-sträng. Till exempel för k = 3 returnerar koden nedan [4, 6] då det är 4 respektive 6 nukleotider mellan startpositionerna för den vanligast förekommande delsekvensen 'AGC'.

Uppgift: Skriv om programmet så att det blir mer läsbart. Vilka bitar i koden utgör lämpliga funktioner? Se till så att de både tar lämpliga parametrar och returnerar lämpliga datastrukturer. Funktionerna ska kunna kallas på individuellt från annan modul.

```

# Given a sequence as input find distances between the copies of
# the most frequent substring

seq = input("Enter a DNA string: ")
k = 3
substrings = {}
for i in range(len(seq) - k + 1):
    substring = seq[i:i+k]
    if substring in substrings:
        substrings[substring] += 1
    else:
        substrings[substring] = 1

max_count = 0
most_freq_substring = ""
for substring, count in substrings.items():
    if count > max_count:
        most_freq_substring = substring
        max_count = count

```

```

positions = []
for i in range(len(seq) - k + 1):
    substring = seq[i: i+k]
    if substring == most_freq_substring:
        positions.append(i)

distances = []
for p1,p2 in zip(positions[:-1], positions[1:]):
    distances.append(p2 - p1)

print(distances)

```

5. † Du har fem nummer i en lista [13, 24, 42, 66, 78] och vill beräkna alla möjliga sätt ett nummer kan skrivas som en “direkt” sammanslagning av 2 element ur listan; med en “direkt sammanslagning” av två element (heltal) menas här t.ex. 13, 78 -> 1378 och 66, 42 -> 6642. Dvs, du vill ta reda på alla permutationer av två element ur listan så och slå samman varje permutation till ett tal.

Tips: Titta på standardmodulen `itertools` för permutationerna. För att slå samman heltal, försöka att först explicit typomvandla talen i listan till `str`, sen konkatenera permutationerna m.h.a strängfunktionen `.join()` och till sista typomvandla tillbaka till `int`.

Utdata bör bli (om alla tal sparats i en lista som man printar):

```
[1324, 1342, 1366, 1378, 2413, 2442, 2466, 2478, 4213, 4224, 4266, 4278, 6613,
6624, 6642, 6678, 7813, 7824, 7842, 7866]
```

6. Skriv om laboration 1 (temperaturkonvertering) så att funktionerna hamnar i separat modul med namnet `temp_functions.py` och dokumentera funktionerna med en dokumentationssträng (se kapitel 2). Huvudprogrammet ska importera funktionerna i `temp_functions.py` samt hantera fel med `try/except`.

Kapitel 8

Funktionell programmering

I detta kapitel ska vi titta på *funktionell* programmering i Python. Detta sätt att programmera på bygger på att man anropar, sätter ihop och modifiera funktioner. På detta sätt kan vi göra mycket av vad vi redan sett i de andra kapitlen, men endast genom att hantera funktioner. T.ex. kan vi byta ut loopar mot *rekursiva* funktioner, dvs funktioner som anropar sig själva.

8.1 Rekursion

Funktioner kan anropa sig själva och då kallas de *rekursiva*. Det finns många exempel på detta i matematiken, men även inom programmering kan rekursion vara användbart då det ofta leder till eleganta och korta program. Det kan dock vara svårt att börja “tänka rekursivt” och se hur man kan lösa olika problem med hjälp av rekursion. Ett bra sätt att lära sig använda rekursion är att ta program man skrivit med hjälp av loopar och göra om dem med rekursion istället.

8.1.1 Fakultet

Ett klassiskt exempel på en rekursiv funktion från matematiken är fakultet. Detta skrivs $n!$ och beräknas genom att man tar produkten av alla tal från 1 till n . Exempel:

$$5! = 5 * 4 * 3 * 2 * 1 = 120$$

Detta kan beskrivas rekursivt, dvs i termer av sig själv, genom:

$$\begin{aligned} 5! &= 5 * 4! \\ &= 5 * 4 * 3! \\ &= 5 * 4 * 3 * 2! \\ &= 5 * 4 * 3 * 2 * 1! \\ &= 5 * 4 * 3 * 2 * 1 * 0! \\ &= 5 * 4 * 3 * 2 * 1 * 1 \\ &= 120 \end{aligned}$$

Detta är en rekursiv beräkning med *basfall* $0! = 1$. En matematiker skriver detta som:

$$n! = \begin{cases} 1 & \text{om } n \text{ är } 0 \\ n * (n - 1)! & \text{annars} \end{cases}$$

I Python skriver vi detta genom:

```
def fac(n):  
    if n == 0:  
        return 1  
    else:  
        return n * fac(n - 1)
```

Fallet $n == 0$ kallas *basfall*. Det är här som rekursionen stannar och börjar nystas upp. Det andra fallet kallas *rekursivt steg* och går ut på att göra ett "enkla" anrop, där problemet i någon mening har gjorts mindre.

Denna funktion anropas precis på samma sätt som vanligt:

```
print(fac(5))
```

```
120
```

8.1.2 Fibonacci-talen

Varje kurs som tar upp rekursion måste ha med Fibonacci-talen som exempel. Detta är en talserie som ser ut på följande sätt:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Det n :te Fibonacci-talet, $F(n)$, är summan av de två tidigare Fibonacci-talen. Denna sekvens har två basfall, $F(0) = 0$ och $F(1) = 1$. Därför kan vi skriva

$$F(n) = \begin{cases} 0 & \text{om } n \text{ är } 0 \\ 1 & \text{om } n \text{ är } 1 \\ F(n - 1) + F(n - 2) & \text{annars} \end{cases}$$

Detta är tydligt en rekursiv definition och vi kan implementera den genom:

```
def fib(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    else:  
        return fib(n-1) + fib(n-2)  
  
for i in range(10):  
    print(fib(i))
```

```
0  
1
```

```
1
2
3
5
8
13
21
34
```

Du kan jämföra denna implementation av en funktion som beräknar Fibonacci-tal med den i uppgift 6 i [Kapitel 6](#), som använder sig av en generator.

8.1.3 Största gemensamma delare, rekursivt

Minns [Euklides algoritmen](#) för största gemensamma delare (GCD, för "Greatest Common Divisor") för två tal a och b från avsnitt 3.3.3:

- Antag $a \geq b$,
- låt r vara resten av heltalsdivision $a // b$,
- om $r == 0$, då är $\text{GCD}(a, b) = b$,
- annars kan man använda att $\text{GCD}(a, b) = \text{GCD}(b, r)$.

En enkel iterativ implementation:

```
def gcd(a,b):
    if b > a:
        print("a must be bigger than b in gcd(a,b)")
        return None

    r = a % b
    while r != 0:
        a = b
        b = r
        r = a % b
    return b

print("GCD: " + str(gcd(15,12)) + "\n")
print("GCD: " + str(gcd(12,15)))
```

```
GCD: 3
```

```
a must be bigger than b in gcd(a,b)
```

```
GCD: None
```

Det är inte helt lätt att se hur denna implementation följer av algoritmen. En rekursiv implementation är mycket närmare:

```
def gcd(a,b):
    if b > a:
        print("a must be bigger than b in gcd(a,b)")
        return None
```

```

r = a % b
if r == 0:
    return b
else:
    return gcd(b,r)

print("GCD: " + str(gcd(15,12)) + "\n")
print("GCD: " + str(gcd(12,15)))

```

GCD: 3

a must be bigger than b in gcd(a,b)
GCD: None

8.1.4 Loopar genom rekursion

Ovan såg vi ett exempel på ett program vi redan skrivit med loopar som kan ersättas med rekursion. Detta kan alltid göras, både för **for** och **while** loopar.

I GCD-exemplet var det en **while** loop som byttes ut mot en rekursiv definition. Nedan är en enkel funktion som använder en **for** loop för att beräkna summan av alla tal upp till och med x:

```

def f(x):
    s = 0

    for i in range(x+1):
        s += i;

    return s

```

Detta kan skrivas rekursivt på följande sätt:

```

def f_rec(x):
    if x == 0:
        return 0
    else:
        return x + f_rec(x-1)

```

Tanken är här att vi anropar `f_rec` rekursivt med mindre och mindre tal tills vi kommer till basfallet när `x` är 0. Detta representerar loopen i `f`. Vi kan testa så att funktionerna returnerar samma resultat genom:

```

for i in range(5):
    print("f(" + str(i) + ") = " + str(f(i)))
    print("f_rec(" + str(i) + ") = " + str(f_rec(i)))

```

```

f(0) = 0
f_rec(0) = 0
f(1) = 1
f_rec(1) = 1
f(2) = 3
f_rec(2) = 3
f(3) = 6

```

```
f_rec(3) = 6
f(4) = 10
f_rec(4) = 10
```

8.1.5 Rekursion över listor

Vi har sett hur man kan skriva rekursiva funktioner över tal. Detta går även att göra över andra datatyper. Det generella tankesättet är att man måste ha ett eller fler basfall och att de rekursiva fallen reducerar indata på något sätt så att man till slut kommer till ett basfall. För tal använde vi ofta 0 som basfall och minskade indata när vi gjorde rekursiva anrop. Vill man skriva rekursiva funktioner över listor använder man istället ofta [] som basfall och tar bort ett element från början eller slutet i det rekursiva anropet.

Vi kan på detta sätt skriva vår egen len funktion genom:

```
def length(xs):
    # obs: lists behave as boolean values ([] is False)
    if xs:
        xs.pop()
        return (1 + length(xs))
    else:
        return 0

print(length([1,341,23,1,0,2]))
```

```
6
```

Varning: den här funktionen ändrar xs! Testa följande:

```
mylist = [1,2,3,6,7]

print(mylist)

print(length(mylist))

print(mylist)
```

```
[1, 2, 3, 6, 7]
5
[]
```

Då length har anropat pop upprepade gånger har mylist blivit tom! För att det inte ska bli så här måste vi anropa length med en *kopia* av mylist:

```
mylist = [1,2,3,6,7]

print(mylist)

print(length(mylist.copy()))

print(mylist)
```

```
[1, 2, 3, 6, 7]
5
[1, 2, 3, 6, 7]
```

Summan av alla element i en lista kan beräknas på följande sätt

```
def sum(xs):
    if xs:
        x = xs.pop() # pop returns the element
        return (x + sum(xs))
    else:
        return 0

print(sum([1,3,5,-1,0,2]))
```

```
10
```

och produkten på detta sätt:

```
def prod(xs):
    if xs:
        x = xs.pop() # pop returns the element
        return (x * prod(xs))
    else:
        return 1

print(prod([1,3,5,-1,2]))
```

```
-30
```

8.1.5.1 Sortering av listor

En snabb och bra sorteringsalgoritm är *quicksort*. Den fungerar på följande sätt:

1. Välj ett element i listan ("pivotelementet").
2. Dela upp listan i två dellistor: en med alla element som är mindre än pivotelementet och en med alla element som är större än pivotelementet.
3. Sortera dellistorna genom rekursion. Basfallet är tomma listan vilken redan är sorterad.
4. Sätt ihop listorna med pivotelementet i mitten.

För att implementera detta i Python skriver vi först två hjälpfunktioner för att hämta ut alla element som är mindre och större än ett givet element:

```
# All elements less than or equal to v in xs
def smallereq(xs,v):
    res = []
    while xs:
        x = xs.pop()
        if x <= v:
            res.append(x)
    return res
```



```
# All elements greater than v in xs
```

```
def greater(xs,v):  
    res = []  
    while xs:  
        x = xs.pop()  
        if x > v:  
            res.append(x)  
    return res
```

```
print(smaller([2,4,5,6,7,8],5))  
print(greater([2,4,5,6,7,8],5))
```

```
[5, 4, 2]  
[8, 7, 6]
```

Vi kan sedan implementera quicksort på följande sätt:

```
def quicksort(xs):  
    if xs == []:  
        return xs  
  
    p = xs.pop()  
    s = quicksort(smaller(xs.copy(),p))  
    g = quicksort(greater(xs.copy(),p))  
    return (s + [p] + g)
```

```
print(quicksort([8,3,3,2,4,5,8,5,6,7,8]))
```

```
[2, 3, 3, 4, 5, 5, 6, 7, 8, 8, 8]
```

Vi kan även skriva smaller och greater direkt med listomfattningar och få en ännu kortare implementation:

```
def quicksort(xs):  
    if xs == []:  
        return xs  
    else:  
        p = xs.pop()  
        s = quicksort([ x for x in xs if x <= p ])  
        g = quicksort([ x for x in xs if x > p ])  
        return (s + [p] + g)
```

```
print(quicksort([8,3,3,2,4,5,8,5,6,7,8]))
```

```
[2, 3, 3, 4, 5, 5, 6, 7, 8, 8, 8]
```

8.1.6 Effektiv rekursion

En fara med att skriva rekursiva funktioner är att de kan bli väldigt långsamma jämfört med iterativ kod skriven med loopar. Exempelvis om vi utvidgar definitionen av fib(5) för hand så ser vi följande:

```
fib(5) = fib(4) + fib(3)
       = (fib(3) + fib(2)) + (fib(2) + fib(1))
       = ((fib(2) + fib(1)) + (fib(1) + fib(0))) + ((fib(1) + fib(0)) + fib(1))
       = (((fib(1) + fib(0)) + fib(1)) + (fib(1) + fib(0)))
         + ((fib(1) + fib(0)) + fib(1))
       = (((1 + 0) + 1) + (1 + 0)) + ((1 + 0) + 1)
       = 5
```

Problemet är att vi beräknar samma tal många gånger, så vi får väldigt långsam kod. Testar man följande så ser man att det tar längre och längre tid att skriva ut utdata:

```
for x in range(35):
    print(fib(x))
```

```
0
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
987
1597
2584
4181
6765
10946
17711
28657
46368
75025
121393
196418
317811
514229
832040
1346269
2178309
3524578
5702887
```

Det sista talet här tar flera sekunder att skriva ut.

Detta kan lösas genom att skriva en smartare version av fib:

```
def fib_rec(n, a=0, b=1):
    if n == 0:
        return a
    elif n == 1:
        return b
    else:
        return fib_rec(n-1, b, a+b)
```

Tanken här är att n är en räknare för indexet på det Fibonacci tal vi håller på att beräkna, a är värdet 2 steg tidigare i sekvensen och b är värdet 1 steg tidigare. På detta sätt behöver vi bara ett rekursivt anrop och koden blir mycket snabbare. Exempelvis skrivs följande ut på nolltid:

```
print(fib_rec(100))
```

```
354224848179261915075
```

Denna funktion kommer fungera på följande sätt för att räkna ut fib_rec(5) (dvs. fib_rec(5,0,1)):

```
fib_rec(5) = fib_rec(5,0,1)
           = fib_rec(4,1,1)
           = fib_rec(3,1,2)
           = fib_rec(2,2,3)
           = fib_rec(1,3,5)
           = 5
```

På detta sätt har vi alltså undvikit den kombinatoriska explosionen och fib_rec är väldigt mycket snabbare än fib.

Vill man prova med större tal går det också:

```
print(fib_rec(1000))
```

```
434665576869374564356885276750406258025646605173717804024817290895365554179490518904
038798400792551692959225930803226347752096896232398733224711616429964409065331879382
98969649928516003704476137795166849228875
```

Vill man prova med väldigt stora tal måste man göra följande då Python har en inbyggd gräns för hur många rekursiva anrop som tillåts:

```
import sys

sys.setrecursionlimit(100000)

print(fib_rec(10000))
```

Beräkningen av detta väldigt stora tal går på nolltid, även fast det består av över 2000 siffror. Så rekursiva funktioner kan vara väldigt snabba om man skriver dem på rätt sätt.

Python är dock inte alltid jättesnabbt när det kommer till att exekvera funktioner skrivna med rekursion. Så det är oftast bättre att skriva en funktion med hjälp av for eller while om man behöver snabb kod.

8.1.7 Diskussion om rekursion

- Princip: bryt upp i mindre instanser, lös dessa.
- Rekursion är att “ta hjälp av en (osynlig) kompis”
- Rekursion är en effektiv algoritmisk teknik
 - Rekursion är dock inte Pythons starkaste sida. Många programmeringsspråk konverterar internt (utan att programmeraren behöver göra något) viss rekursiv kod till while-loopar. Det är möjligt när det rekursiva anropet är sist i funktionen, vilket kallas “svansrekursion”, och ger snabbare och mer minnessnål exekvering.
- Ofta användbar teknik i datastrukturer (t.ex. sökträd).

Principen för ett funktionsanrop är som följer.

- När ett funktionsanrop görs läggs anropsparametrarna på den minnesarea som kallas *stacken*.
- En funktion börjar exekvera genom att plocka upp anropsparametrarna från stacken till lokala variabler.
- När **return** exekveras så läggs returvärdet på stacken.
- Den kod som gjort ett funktionsanrop kan alltså hämta resultatet på stacken när funktionen är klar.

Det är denna princip som gör att rekursion i praktiken är enkel i datorer.

8.2 Anonyma och högre ordningens funktioner

När vi har pratat variabler och värden har vi hittills framförallt pratat om heltal, flyttal, strängar, listor, med mera. Det finns goda skäl att lägga *funktioner* till uppräknningen av värden att arbeta med, vilket handlar om möjligheten att skapa nya funktioner när man behöver dem, skicka funktioner som argument, returnera funktioner, och kunna tillämpa operatorer på funktioner. När ett programmeringsspråk har de möjligheterna säger man att funktioner behandlas som *first class citizens* (eller *first class objects* eller *first class functions*).

Gamla språk i samma tradition som Python, så kallade *imperativa språk*, var dåliga på att hantera funktioner, men i Lisp och andra språk för *funktionell programmering* så har funktioner varit *first class citizens* sedan 50-talet. Idag kan man säga att det är en vanlig finess som många språk stödjer, däribland Python.

8.2.1 Anonyma funktioner

Begreppet *anonyma funktioner* med hjälp av *lambda-uttryck* finns i många språk idag. Man säger att funktionerna är *anonyma* eftersom man inte ger dem ett namn, utan bara skapar dem. Tag kodsnutten

```
double = lambda x: 2 * x

print(double(3))
```

6

Definitionen `double` är en funktion som tar ett argument, `x`, och beräknar `2 * x`. Vi kan även få en funktion av två argument genom:

```
f = lambda x, y: 2 * x + y

print(f(3,4))
```

Detta är ekvivalent med följande definition:

```
def f(x,y):
    return 2 * x + y
```

Man ska dock inte se lambda-uttryck som bara ett alternativ till vanliga funktionsdefinitioner; de är snarare komplement till **def** och dess främsta användningsområde är för att skapa tillfälliga små funktioner att använda tillsammans med *högre ordningens funktioner*.

Anledningen att dessa introduceras med **lambda** är från den så kallade λ -kalkylen. Detta är en beräkningsmodell uppfunnen av logikern Alonzo Church på 1930-talet (https://en.wikipedia.org/wiki/Lambda_calculus). I denna modell är allting uppbyggt från funktioner och variabler (så t.ex. heltal representeras även de som funktioner), och man kan på detta sätt representera alla beräkningar som kan göras med en **Turingmaskin** genom att bara använda lambda-uttryck.

Anmärkning: En Turingmaskin är en matematisk beräkningsmodell/koncept, som används för att förstå algoritmer och deras begränsningar. Datorer som används idag är inte uppbyggda enligt Turingmaskin-konceptet, men termen Turingkomplett om en formell definition av regler, som t.ex. utgör ett programmeringsspråk, är sådan att vilken Turingmaskin som helst kan representeras med instruktioner enligt dessa regler.

8.2.2 Högre ordningens funktioner

En *högre ordningens funktion* tar en funktion som argument och är ett djupare begrepp än det kanske först låter som. Poängen med högre ordningens funktioner är att de kan vara ett sätt att generalisera funktionalitet. Genom att identifiera vanliga mönster för beräkningar och låta dessa implementeras med högre ordningens funktioner kan man förenkla, korta ner, och kanske även snabba upp sin kod. Några exempel på beräkningar man kan vilja göra är:

1. Ta en lista med tal och returnera en lista med dess kvadrater.
2. Ta en lista med icke-tomma listor och returnera en lista med det första elementet från varje lista.
3. Ta en lista med strängar och räkna ut deras längder.

Dessa tre beräkningar har gemensamt att man ska utföra något på varje element i en lista. Det är inte svårt att skriva en **for**-loop för detta, men funktionen `map` är mer kompakt, känns igen av andra programmerare, och är redan implementerad (se också uppgift 7 i [kapitel 6](#)). Det första argumentet till `map` är en funktion som sedan appliceras på alla element i listan. Denna funktion kan antingen ges genom `lambda` eller genom namn på vilken Pythonfunktion som helst.

Ovan uppgifter kan därför lätt lösas med några små enkla uttryck:

```
# 1
print(list(map(lambda x: x**2, [1,2,3,4])))

# 2
print(list(map(lambda t: t[0], [[1,1], [0,2,1], [3,1,53,2]])))

# 3
print(list(map(len, ["hej", "hopp"])))
```

```
[1, 4, 9, 16]
[1, 0, 3]
[3, 4]
```

Obs: map returnerar en generator, så vi måste lägga in `list` för att få ut en lista.

8.2.2.1 Inte bara för listor

Observera också att map och liknande funktioner inte är låsta till listor, utan kan tillämpas på allt som i Python är definierat som *itererbart*. Det inkluderar till exempel strängar, tupler och uppslagstabeller, men kan också definieras av programmeraren själv.

För att beräkna längden på varje rad i filen `people.txt` med innehåll:

```
Anders
Christian
Kristoffer
Lars
```

kan vi köra:

```
print(list(map(len, open('people.txt'))))
```

```
[7, 10, 11, 5]
```

Notera att newline tecknet `\n` räknas med och är av längd 1.

8.2.2.2 Högre ordningens funktion: filter

En annan typ av funktionalitet som vi ofta vill använda är att välja ut element som uppfyller något kriterium. Till det finns funktionen `filter`. Den tar ett predikat, dvs en funktion som returnerar sant eller falskt, och en itererbar datastruktur och returnerar en generator (som du kan iterera över eller plocka fram som en lista med `list`) med de element som predikatet är sant för (`True` eller motsvarande, tex icke-noll). Exempel:

```
print(list(filter(lambda x: x % 2, range(10))))
print(list(filter(lambda x: x, [[1], [2,3], [], [], [4,5,6]])))
```

```
[1, 3, 5, 7, 9]
[[1], [2, 3], [4, 5, 6]]
```

8.2.3 Egna högre ordningens funktioner

Funktionerna `map` och `filter` har ingen särställning i Python, de är helt enkelt bra verktyg som många finner praktiska att använda. Det är inte så svårt att skriva dessa funktioner själv. Däremot kan det ju vara så att du hittar egna beräkningsbehov som kan generaliseras.

Till exempel kan man beräkna summan och produkten av elementen i två listor med hjälp av slicing på följande sätt:

```
def sum_rec(xs):
    if xs == []:
        return 0
```

```

    else:
        return (xs[0] + sum_rec(xs[1:]))

def prod_rec(xs):
    if xs == []:
        return 1
    else:
        return (xs[0] * prod_rec(xs[1:]))

print(sum_rec([1,2,3,4]))
print(prod_rec([1,2,3,4]))

```

```

10
24

```

Den enda skillnaden mellan dessa är att en använder + och 0, och den andra * och 1. Vi kan alltså generalisera dessa till en funktion som tar in en funktion f och ett element x :

```

def foldr(xs,f,x):
    if xs == []:
        return x
    else:
        return f(xs[0], foldr(xs[1:], f, x))

sum = lambda xs: foldr(xs, lambda x,y: x + y, 0)
prod = lambda xs: foldr(xs, lambda x,y: x * y, 1)

print(sum([1,2,3,4]))
print(prod([1,2,3,4]))

```

```

10
24

```

Vi kan nu även lätt skriva en funktion som använder exponentiering som operation:

```

exps = lambda xs: foldr(xs, lambda x,y: x ** y, 1)

print(exps([2,3,4]))

```

```

2417851639229258349412352

```

Vi kan även skriva en funktion för att slå ihop listor av listor:

```

join = lambda xs: foldr(xs, lambda x,y: x + y, [])

print(join([[1], [2,3], [], [], [4,5,6]]))

```

```

[1, 2, 3, 4, 5, 6]

```

Högre ordningens funktioner är alltså mycket smidiga för att skriva väldigt generella funktioner och minimera duplikation av kod!

Obs: det finns en funktion som heter reduce som fungerar exakt som foldr, men med argumenten i en

annan ordning. Den ligger i biblioteket `functools` och importeras på följande sätt:

```
import functools

print(functools.reduce(lambda s, x: s+x, [1,2,3], 0))
```

6

8.3 Uppgifter

1. Skriv en rekursiv funktion `rec_prod_m(n,m)` som tar två tal n och m och beräknar $n * (n-m) * (n-2m) * \dots$ tills $n-km$ har blivit negativt.

Tips: Funktionen kan skrivas genom att modifiera funktionen `fac` något.

2. Vad händer om vi ändrar till `return 0` i basfallet för `prod`? Varför?
3. Deklarera fem funktioner `f1, ..., f5` med hjälp av `lambda` som utför addition, subtraktion, multiplikation, division, och exponentiering. Till exempel ska `f1(4,7)` returnera 11. Använd dessa funktioner för att skriva uttrycken

```
(6*(2+3)/5)**2
(10 - 2**3)*5
```

4. Använd `map` och `lambda` för att skriva funktion `get_evens(xs)` som tar en lista med tal `xs` och konverterar jämna tal till `True` och udda tal till `False`.
5. Studera koden nedan. Hur många anrop till `f` görs totalt om vi anropar `f` med 3? Hur många blir det för 4? Varför?

```
def f(n):
    if n <= 1:
        return 1
    else:
        return f(n-1) + f(n-2) + f(n-3)
```

6. Collatz-sekvensen med start i n får man med följande regler:

- Om $n = 1$ är vi klara.
- Om n är jämnt, fortsätt med $n/2$.
- Annars, fortsätt med $3n + 1$.

Collatz-sekvensen med start i $n = 10$ är: 10, 5, 16, 8, 4, 2. Om man startar med $n = 9$ får man en längre sekvens: 9, 28, 14, 7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2.

Skriv en rekursiv funktion `collatz(n)` returnerar en lista med Collatz-sekvensen med start i n .

Exempel:

```
>>> collatz(10)
```

```
>>> collatz(10)
[10, 5, 16, 8, 4, 2]
```

7. Använd funktionen `foldr` för att skriva en funktion `joinstrings` som konkatenerar en lista med strängar.

8. Skriv `fac` och `fib` med hjälp av loopar istället.
9. Vad blir fel om vi inte kopierar `xs` i quicksort?
10. † Pascals triangel ser ut så här:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

osv... Skriv en funktion `pascal(n)` som returnerar den `n`:e raden. Den ska fungera på följande sätt:

```
for i in range(1,8):
    print(pascal(i))
```

```
[1]
[1, 1]
[1, 2, 1]
[1, 3, 3, 1]
[1, 4, 6, 4, 1]
[1, 5, 10, 10, 5, 1]
[1, 6, 15, 20, 15, 6, 1]
```

Kapitel 9

Objektorientering 1: Klasser

9.1 Objektorientering

Objektorientering (förkortas OO) är ett programmeringsparadigm eller programmeringsstil där man samlar data och datastrukturer med de funktioner som kan tillämpas på dem. I Python är i princip allt implementerat som objekt. Detta gäller såväl heltal, flyttal, strängar, listor, uppslagstabeller som funktioner. Vad menas då med att knyta ihop datastrukturer med “funktioner som kan tillämpas på dem”? Ta till exempel listor:

```
l = [2,7,2]
print(l.append(1))
```

```
[2, 7, 2, 1]
```

I objektorienterade termer har vi på första raden skapat ett objekt `l` av typ `list`. Vi har på andra raden använt funktionen `append` som är *specifik* för datatypen `list`. I objektorientering benämner man funktioner som är knutna till objekt för *metoder*. De anropas med hjälp av punktnotationen `object.method(<parameters>)`. Det är en notation som du vid det här laget kanske känner igen? Ta till exempel strängar och operationer vi gör på dem:

```
s = 'Hej'
print(s.lower())
```

```
hej
```

Här har vi skapat ett objekt `s` av typ `str` och använt metoden `lower` (specifik för datatypen `str`).

9.1.1 Varför objektorientering?

Objektorientering lämpar sig bra att använda när man vill:

- Knyta samman mycket data som relaterar till varandra (hör till samma objekt).
- Kunna definiera egna datatyper.

Säg till exempel att vi vill hålla data associerat till en bil (kanske för ett datorspel) i minnet i ett program. Det kan vara däckbredd, färg, antal dörrar, nuvarande bensinnivå, hastighet, vinkel på framhjulen och

en mängd annan data. Det skulle då vara smidigt att ha all data för denna bil “på ett ställe”. Om vi hade ett bil-objekt `b` skulle vi komma åt och kunna ändra alla *attribut* i detta objekt med

```
b.tire_width()
b.color()
b.increase_speed(2)
```

Det är naturligt att tänka på objekt som “fysiska ting” (t.ex. en bil), men det finns även naturliga objekt som vi inte kan ta på. Vi kommer strax visa hur vi implementerar en kurs (t.ex. DA2005) där vi kan samla användarinformation, resultat, med mera. Slutligen ska vi även titta på hur vi kan implementera vår egen datastruktur för att representera heltalsmängder.

Två fördelar med att skriva objektorienterad kod är:

1. Det uppmuntrar god struktur genom att samla data och relaterad funktionalitet på samma ställe.
2. Det är bra för abstraktion, eftersom det blir lättare att gömma detaljer.

9.1.2 Hur skapas objekt?

Hur skriver vi då objektorienterad kod? För att skapa egna objekt (tänk till exempel på en bil eller en kurs som ett objekt) så måste vi ha en specifikation för objektets data, metoder och allmänna struktur. En sådan specifikation heter *klass* i objektorienteringens terminologi. Vi kan skapa en klass som paketerar data och funktionalitet (du kan tolka “klass” som “klassificering”, som minnestrick). En klass är att jämföra med en typ och ett objekt är en *instans* av en klass. Till exempel är `[1, 2, 3]` en instans av klassen `list`. Varje instans kan ha olika *attribut* kopplade till den för att representera dess nuvarande tillstånd. Ett sätt att förstå attribut är att tänka på dem som ett objekts “egna” variabler. Instanser kan även ha *metoder*, att tänka på som funktioner definierade i klassen, för att modifiera dess tillstånd. Ett exempel på en metod för listor är `pop()`, som man kan tillämpa på ett objekt `xs` med `xs.pop()`.

Vi har i kursen använt många exempel på klasser: heltal, listor, uppslagstabeller, med mera. Vi ska nu lära oss att skriva våra egna.

9.1.3 Att definiera en klass

Ett minimalt exempel är för en tom klass:

```
class Course:
    pass
```

Vi har döpt klassen till `Course`. Konventionen är att klassnamn börjar med stor bokstav.

Klassdefinitionen ovan gör det möjligt att *instansiera* klassen till *objekt*. Man kan sedan knyta olika *attribut* till objektet.

```
k = Course()          # An object is created, an instance of the class Course
k.code = 'DA2005'     # An attribute is assigned to the object
k2 = Course()         # A new object is created
print(k.code)
```

```
DA2005
```

Om man försöker använda attributet `code` för objektet `k2` så får man ett felmeddelande eftersom vi inte har tillskrivit `k2` något sådant attribut:

```
print(k2.code) # This object does not have an attribute "code"
```

```
AttributeError: 'Course' object has no attribute 'code'
```

Här är `code` alltså en variabel som tillhör objektet/instansen `k`. Vi kommer ofta säga "instansattribut" för denna typ av variabler.

9.1.4 Metoder

En *metod* är en funktion definierad i en klass. I `Course` (nedan) är alla **def**-satser metoder. När man definierar metoder följer man samma konventioner som för funktioner, det vill säga identifierarna skrivs med små bokstäver och om man använder flera ord så separeras de med `_`.

```
class Course:

    def __init__(self, code, name, year=2024):          # Constructor
        self.participants = []
        self.code = code
        self.name = name
        self.year = year

    def number_participants(self):
        return len(self.participants)

    def new_participant(self, name, email):
        self.participants.append((name, email))
```

Ignorera identifieraren `self` en liten stund.

Den första metoden, `__init__`, har en särställning och kallas *konstruktor*. En konstruktor är den metod som initierar ett nytt objekt. Konstruktorn anropas bara vid instansiering (alltså då vi skapar objektet). Metoder med identifierare på formen `__fkn__` används av Python-systemet, så man bör vara försiktig med hur dessa används. Konstruktorn heter alltid `__init__`, så när vi kör

```
da2005 = Course('DA2005', 'Programmeringsteknik')
```

anropas konstruktormetoden `Course.__init__` och resultatet sparas i variabeln `da2005`. Objektet i `da2005` är då en *instans* av `Course`-klassen. Det är vidare endast `__init__` som anropas vid instansiering. De övriga metoderna finns tillgängliga att använda när vi anropar dem.

Vi kan modifiera ett objekt genom att anropa dess metoder. Detta gör man genom punkt-notation (precis som vi använt `xs.pop()` och andra listmetoder för att modifiera listor).

```
da2005.new_participant('Johan Jansson', 'j.jansson@exempel.se')
print(da2005.number_participants())
```

```
1
```

9.1.5 Self

Vad är då uttrycket `self` som finns med framför alla attribut och som parametrar till metoder? Variabeln `self` i `Course`-klassen representerar objektet som metoden anropas från (man måste inte kalla det argumentet för `self`, men det är en konvention bland Python-programmerare att göra så). I exemplet

ovan refererar `self` i `new_participant` till objektet `da2005`. Detta kan vara svårt att greppa när man först lär sig om klasser och objektorientering. Som illustration kan du tänka dig att raden

```
da2005.new_participant('Johan Jansson', 'j.jansson@exempel.se')
```

egentligen motsvarar:

```
Course.new_participant(da2005, 'Johan Jansson', 'j.jansson@exempel.se')
```

dvs att vi plockar fram metoden `new_participant` från klassen `Course` och i första parametern anger vilket objekt som ska användas.

Obs: Detta är inte korrekt syntax i Python, utan endast en illustration av konceptet att metoden `new_participant` tar objektet `da2005` som parameter för att kunna modifiera det.

9.1.6 Klass- och instansattribut

Attribut är ett annat sätt att benämna variabler som är specifika för objekt eller klasser. Det finns två olika typer av attribut; *instansattribut* och *klassattribut*. Generellt sett används *instansattribut* för att hålla data som är unik för varje instans (dvs, specifika för objektet) och *klassattribut* används för attribut och metoder som delas av alla instanser av klassen:

```
class Course:

    university = 'Stockholm University' # class attribute shared by all instances
                                         # (1) Occurs outside of constructor
                                         # (2) Has no 'self', i.e., specific object,
                                         # associated to it

    def __init__(self, code, name, year=2024):
        # instance attributes below, unique to each instance
        self.participants = []
        self.code = code
        self.name = name
        self.year = year
```

Exempelanvändning:

```
da2005 = Course('DA2005', 'Programmeringsteknik')
da2005 = Course('DA4007', 'Programmeringsteknik II', year = 2025)

print(da2005.university)           # shared by all courses
print(da4007.university)           # shared by all courses
print(da2005.code, da2005.year, da2005.name) # unique to da2005
print(da4007.code, da4007.year, da4007.name) # unique to da4007
print(Course.university)           # you can also access a class
                                     # attribute from the class itself
```

```
Stockholm University
Stockholm University
DA2005 2024 Programmeringsteknik
DA4007 2025 Programmeringsteknik II
Stockholm University
```

Vi kan även skriva över nuvarande värden på attribut genom

```
da2005.year = 2025
print(da2005.year)
```

```
2025
```

På samma sätt som vi ovan skrivit över instansattributet `year` i `da2005`, men inte i `da4007`, kan vi även skriva över metoder i ett objekt (kom ihåg att funktioner ej har någon särställning utan är som andra objekt i Python).

Varning: Muterbara data kan ha överraskande effekter om de används som klassattribut!

Om vi till exempel hade lagt listan `participants` som en klassvariabel kommer den delas av *alla* kurser:

```
class Course:

    university = 'Stockholm University' # class attribute shared by all instances
    participants = []                  # class attribute shared by all instances

    def __init__(self, code, name, year=2024):
        # instance attributes below, unique to each instance
        self.code = code
        self.name = name
        self.year = year

    def new_participant(self, name, email):
        self.participants.append((name, email))

d = Course('da2005', 'Programmeringsteknik')
e = Course('da4007', 'Programmeringsteknik (online version)')

d.new_participant('Johan Jansson', 'j.jansson@exempel.se')
print(d.participants) # shared by all courses
print(e.participants) # shared by all courses

[('Johan Jansson', 'j.jansson@exempel.se')]
[('Johan Jansson', 'j.jansson@exempel.se')]
```

9.1.7 Privata och publika metoder och attribut

I objektorienterad programmering skiljer man ofta på *privata* och *publika* metoder och attribut. De publika metoderna utgör *gränssnittet* (eng. *interface*) för en klass och är de metoder som användare av klassen ska använda. De privata metoderna och attributen är för internt bruk. I många programmeringsspråk finns det stöd för att kontrollera åtkomst av metoder och attribut så att man inte av misstag börjar förlita sig på annat än klassens gränssnitt. På detta sätt uppnår man *abstraktion*¹ med hjälp av objektorientering:

Python fokuserar på enkelhet och har istället en konvention: identifierare för attribut/metoder som börjar med understreck `_` ses som *privata*. De bör alltså bara användas internt.

Vi har nedan lagt till en privat metod `_check_duplicate` i vårt kursexempel. Det är vanligt att studenter registrerar sig med olika email-adresser. Metoden kontrollerar om identiska namn förekommer i del-

¹Wikipedia: [Abstraction in object oriented programming](#).

tagarlistan när vi lägger till en deltagare och skriver i så fall ut en varning. Vi kan då vidta lämpliga åtgärder (t.ex. implementera en metod `remove_participant` för att ta bort dubbla deltagare).

```
import warnings # included in Python's standard library

class Course:

    def __init__(self, code, name, year=2024): # Constructor
        self.participants = []
        self.code = code
        self.name = name
        self.year = year

    def _check_duplicate(self, name, email):
        for p in self.participants:
            if name == p[0]: # check if identical name
                # raise warning and print message if identical name:
                # uses string formatting, see e.g.:
                # https://www.w3schools.com/python/ref_string_format.asp
                warnings.warn(
                    'Warning: Name already exists under entry: {}'.format(p))

    def number_participants(self):
        return len(self.participants)

    def new_participant(self, name, email):
        self._check_duplicate(name, email)
        self.participants.append((name, email))
```

Vi kan nu se vad som händer om vi registrerar studenter med samma namn:

```
da2005 = Course('DA2005', 'Programmingsteknik')
da2005.new_participant('Johan Jansson', 'j.jansson@exempel.se')
da2005.new_participant('Johan Jansson', 'j.jansson@gmail.com')
```

Den tredje raden kommer att generera en “varning” som skrivs ut

```
UserWarning: Warning: Name already exists under entry:
('Johan Jansson', 'j.jansson@exempel.se')
```

I detta exempel används metoden `_check_duplicate` för internt bruk i klassen `Course`. Det är alltså inte tänkt att användaren av klassen ska anropa denna metod. Metoden är inte med i *gränssnittet* och ökar således inte storleken (komplexiteten i användning) av vår klass. Syftet med privata metoder är att kunna definiera metoder som är praktiska att använda internt i klassen, men som man vet kan vara olämpliga eller ologiska som gränssnittsmetod. I exemplet ovan är det en praktiskt hjälpmetod med användbar funktionalitet när vi lägger till deltagare. Notera att även om det är markerat med `_` att metoden är privat, så kan vi fortfarande anropa den med följande explicita metदानrop,

```
da2005._check_duplicate('Johan Jansson', 'j.jansson@exempel.se')
```

men det betraktas som dålig programmering.

9.1.8 Ordlista

- *Objekt*: ett element av någon typ med olika attribut och metoder.
- *Klass*: en typ innehållande attribut och metoder (t.ex. `list`, `dict`, `Course` från exemplet ovan, etc.).
- *Instansiering*: då ett objekt skapas från en viss klass.
- *Metod*: en funktion definierad i en klass (ex. `pop`, `insert`).
- *Konstruktör*: metod för att instansiera ett objekt av klassen (ex. `dict()`). Varje klass har en konstruktör `__init__` som antingen är implicit eller explicit definierad.
- *Klassattribut*: variabel definierad utanför konstruktorn i en klass (delas av alla instanser av klassen).
- *Instansattribut*: variabel definierad i konstruktorn för klassen (eller satt med `self` i klassen). Tillhör en specifik instans av klassen.

9.2 Modularitet genom objektorientering

I exemplet `Course` ovan har vi använt en lista med tupler för att representera deltagarlistan `participants`. En mer objektorienterad lösning hade varit att ha en separat klass för deltagare:

```
class Participant:

    def __init__(self, name, email):
        self._name = name
        self._email = email

    def name(self):
        return self._name

    def email(self):
        return self._email
```

Vi måste nu modifiera metoderna i `Course` så att de hanterar deltagare representerade som `Participant` istället för tupler av strängar. Exempelvis:

```
class Course:

    # old code unchanged, except for _check_duplicate which
    # has to be modified to take a Participant

    def new_participant(self, p):
        self._check_duplicate(self, p)
        self.participants.append(p)
```

Vi måste nu skapa deltagare genom:

```
d = Participant('Johan Jansson', 'j.jansson@exempel.se')
da2005.new_participant(d)
da2005.new_participant(Participant('Foo Bar', 'foo@bar.com'))
```

Det kan tyckas konstigt att skriva om sin kod så här, men fördelen är att eventuella framtida förändringar kommer bli mindre komplexa. Exempelvis, låt oss säga att vi vill separera förnamn och efternamn för deltagare. Då räcker det att skriva:


```

class Participant:

    def __init__(self, fname, lname, email):      # Changed!
        self._fname = fname                      # Changed!
        self._lname = lname                      # Changed!
        self._email = email

    def name(self):
        return self._fname + ' ' + self._lname   # Changed!

    def email(self):
        return self._email

```

Koden för Course behöver inte alls ändras eftersom den bara använder Participant-gränssnittet. På detta sätt leder objektorientering till mer modular kod, dvs varje klass kan ses som en liten kodmodul i sig och interna modifikationer påverkar ingen annan kod i programmet så länge gränssnittet är oförändrat.

9.3 Ett till exempel på objektorientering: talmängder

Python har en typ för mängder, liknande uppslagstabeller, men nycklarna är inte associerade med några värden. För dokumentation se:

<https://docs.python.org/3/tutorial/datastructures.html#sets>

<https://docs.python.org/3/library/stdtypes.html#set-types-set-frozenset>

Syntaxen för att skapa en mängd (**set**) liknar den för listor, men med {} istället för []. Några exempel:

```

basket = set(['apple', 'banana', 'orange', 'pear'])
# the above is equivalent to
basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
print(basket)
print('orange' in basket)
print('crabgrass' in basket)

a = set([1,2,5])
b = set([3,5])
print(a)
print(a - b) # difference
print(a | b) # union
print(a & b) # intersection
print(a ^ b) # numbers in a or b but not both

```

```

{'pear', 'apple', 'orange', 'banana'}
True
False
{1, 2, 5}
{1, 2}
{1, 2, 3, 5}
{5}
{1, 2, 3}

```

9.3.1 Egen implementation

Här kommer ett till större exempel taget från kursboken (Introduction to Computation and Programming using Python av John V Guttag, sida 111), där vi implementerar vår egen mängd-datatyp `IntSet` med hjälp av listor. Vi kan representera mängder av heltal som listor, med invarianten (en garanterad egenskap) att det inte finns några dubletter.

```
class IntSet:
    '''An IntSet is a set of integers'''

    def __init__(self):
        '''Create an empty set of integers'''
        self.vals = []

    def insert(self,e):
        '''Assumes e is an integer and inserts e into self'''
        if e not in self.vals:
            self.vals.append(e)

    def member(self,e):
        '''Assumes e is an integer.
        Returns True if e is in self, and False otherwise'''
        return e in self.vals

    def delete(self,e):
        '''Assumes e is an integer and removes e from self
        Raises ValueError if e is not in self'''
        try:
            self.vals.remove(e)
        except:
            raise ValueError(str(e) + ' not found')

    def get_members(self):
        '''Returns a list containing the elements of self.
        Nothing can be assumed about the order of the elements'''
        return self.vals
```

Vi bibehåller *invarianten* att ett `IntSet` ej innehåller några dubletter i alla funktioner.

Lite tester:

```
x = IntSet()
print(x)
x.insert(2)
x.insert(3)
x.insert(2)
print(x)
print(x.member(5))
print(x.member(3))
```

```
x.delete(2)
print(x.get_members())
```

```
<__main__.IntSet object at 0x7febb7002040>
<__main__.IntSet object at 0x7febb7002040>
False
True
[3]
```

De två första print-anropen skriver ut information om objektet; var det är definierat och på vilken minnesadress det sparats på. Om man kör denna kodsnuitt igen kommer troligtvis objektet ligga på ett annat ställe i minnet.

9.3.2 Integrera din klass i Python-systemet

Visst vore det bra att kunna lägga till egna datastrukturer som är lika integrerade i Python-systemet som list, dict, set, och så vidare? Det är möjligt och till och med att betrakta som god programmering. Det man skulle behöva är att metoder som Python tillåter på inbyggda objekt/klasser också är tillämpbara på dina egna klasser. Här beskriver vi hur.

9.3.2.1 str, len och andra anpassningar

Vi kan lägga till metoder `__str__(self)` och `__len__(self)` som gör att vi kan använda oss av Python-systemets syntax `str()`, och `len()`. På detta sätt kan vi få mängder att skrivas ut snyggare än ovan och ett exempel på hur `__str__` metoden kan se ut är

```
def __str__(self):
    result = ''
    for e in self.vals:
        result += str(e) + ', '
    return '{' + result[:-1] + '}'
```

Detta gör att följande skrivs ut istället:

```
x = IntSet()
print(x)
x.insert(2)
x.insert(3)
x.insert(2)
print(x)
```

```
{ }
{2,3}
```

9.3.2.2 Integrering av jämförelseoperatorer

Låt oss säga att vi vill kunna jämföra `IntSet` med `<=` och att `x <= y` ska tolkas som att "x är en delmängd till y". Då kan vi lägga till följande metod till `IntSet` klassen:

```
def __le__(self, other):
    for x in self.vals:
        if not other.member(x):
```

```
        return False
    return True
```

Med tester:

```
print(x <= x)
empty = IntSet()
print(empty <= x)
print(x <= empty)
```

```
True
True
False
```

Vi kan på detta sätt definiera de olika standardoperationerna i Python (+, <, =, ...) för våra egna klasser. Vi kan även *operatoröverlagra* andra operatörer som **in**, **and** med flera. För dokumentation kring namn på standardoperationer för metoder se <https://docs.python.org/3/library/operator.html>. Till exempel kan vi döpa om metoden `member` i vår `IntSet`-klass till `__contains__`. Då kan vi använda oss av operatören **in** och skriva `5 in x` istället för att skriva `x.member(5)`.

9.4 Uppgifter

1. Utöka klasserna `Course` och `Participant` med metoden `__str__` så att vi kan använda oss av `str()` för att få en fin textrepresentation av objekt från de klasserna. Du kan använda dig av exempelkoden nedan (i avsnitt 9.4.1). Det är valfritt hur strängen ser ut, men kurskod och kursnamn ska synas för `Course`-objekt och namn och email ska förekomma i `Participant`-objekt.
2. Lägg till attributen `teacher` av typ `bool` till klassen `Participant`. Skriv ut lärarens namn på något speciellt sätt så att man från `Participant`-listan ser vem det är.
3. Lägg till metoder så att operatörerna `==` och `!=` finns för klassen `IntSet`. Operatören `==` ska returnera **True** om mängderna innehåller samma element, annars **False**. Operatören `!=` är negationen (logiska motsatsen) av `==`.

Tips: läs dokumentationen <https://docs.python.org/3/library/operator.html>.

4. Implementera union, differens och snitt för `IntSet`. Snitt kan du implementera på valfritt sätt, men implementera union genom att operatoröverlagra `+` och differens genom att operatoröverlagra `-`. Metoderna skall returnera ett nytt `IntSet` objekt. Till exempel:

```
x = IntSet()
x.insert(2)
x.insert(3)
x.insert(1)

y = IntSet()
y.insert(2)
y.insert(3)
y.insert(4)
print(x == y, y != x)
print(x.intersection(y))
print(x + y, y - x)
```

```
False True
{2,3}
{4,2,3,1} {4}
```

5. Implementera en klass Dog med funktionalitet som möjliggör gränssnittet nedan.

```
d = Dog('Fido')
e = Dog('Buddy')
d.add_trick('roll over')
e.add_trick('play dead')
d.add_trick('shake hands')
print(d.name, d.tricks)
print(e.name, e.tricks)
```

```
Fido ['roll over', 'shake hands']
Buddy ['play dead']
```

6. Implementera metoderna `__str__` och `__gt__` i klassen Dog så vi kan skriva `str(d)` och jämföra `d > e` (> baseras på hur många trick de kan). Exempel nedan (givet koden vi körde i uppgift 5)

```
print(str(d))
print(d > e)
```

```
Fido knows: roll over, shake hands
True
```

9.4.1 Exempelkod för uppgifterna

```
import warnings # included in Python's standard library

class Course:

    def __init__(self, code, name, year=2024): # Constructor
        self.participants = []
        self.code = code
        self.name = name
        self.year = year

    def _check_duplicate(self, p):
        for other in self.participants:
            if p.name() == other.name(): # check if identical name
                # raise warning and print message if identical name:
                warnings.warn(f'Name already exists under entry:' + str(p))

    def number_participants(self):
        return len(self.participants)

    def new_participant(self, p):
        self._check_duplicate(p)
        self.participants.append(p)
```

```
class Participant:

    def __init__(self, fname, lname, email):
        self._fname = fname
        self._lname = lname
        self._email = email

    def name(self):
        return self._fname + ' ' + self._lname

    def email(self):
        return self._email
```

Kapitel 10

Objektorientering och arv

Arv är en central princip i objektorientering (OO) som handlar om att på ett strukturerat sätt kunna återanvända kod och funktionalitet i redan definierade klasser med möjligheten att *lägga till* eller *ändra* på den existerande klassen. Ofta så finns redan en klass som nästan gör det man vill, och det är då en god idé att låta *ärva* från den klassen.

Ett enkelt exempel är vi har en klass A som vi vill använda funktionaliteten från, men ändra på vissa delar:

```
class A:
    """This is a base class
    """
    class_integer = 10

    def __init__(self):
        self.vals = [5]

    def power_sum(self):
        return sum([i**2 for i in self.vals]) + A.class_integer

class B(A):
    """This is a sub class
    """
    def __init__(self, x, y):
        self.vals = [x, y]

    def print_result(self):
        print(self.power_sum())
```

Här är A en *basklass* (eng. *base class*) för B och B är *subklass* (eng. *sub class*) till A. Man kan också säga att A är *superklass* (eng. *super class*) till B. I flera programmeringsspråk används ofta de engelska termerna *parent class* och *child class* för bas- och subklass.

Lägg märke till att **class**-definitionen av B tar A som ett argument, vilket indikerar att B ärver från A. Med ett arv i detta fall så menas att A:s funktionalitet *automatiskt* kommer att vara implementerad i B,

så länge man inte aktivt definierar om något. I det här fallet har klass-konstruktorn för B, `__init__`, definierats om för att ta *två* istället för *ett* argument (x och y). Utöver detta så används funktionaliteten från A: t.ex. finns i ovan exempel inget skäl att definiera om `power_sum` i B.

Användningen är ganska rättfram:

```
a = A()
print(a.power_sum())    # method defined in A

b = B(10, 20)
print(b.power_sum())    # calling method in B inherited from A
b.print_result()        # uses method specific to B
```

```
35
510
510
```

Obs: i Python ärver man inte instansattribut automatiskt! Det görs i många andra programmeringsspråk.

```
class C(A):
    def __init__(self, y):
        self.vals.append(y)
```

```
c = C(10)
```

```
AttributeError: 'C' object has no attribute 'vals'
```

För att se till att C ärver även instansattribut från A, får vi ändra konstruktorn lite genom att anropa `super`, som ser till att metoden som följer anropas på i basklassen:

```
class C(A):
    def __init__(self, y):
        super().__init__() # call constructor of super class
        self.vals.append(y)

class D(B):
    def __init__(self, y):
        super().__init__(5, 10) # call constructor of super class
        self.vals.append(y)
```

```
c = C(10)
d = D(10)
print(c.power_sum())
d.print_result()
c.print_result()        # raises an Error
```

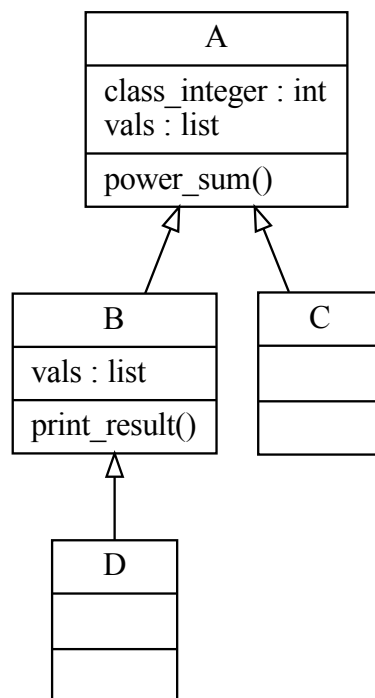
```
135
235
AttributeError: 'C' object has no attribute 'print_result'
```

Felet uppstår på grund av att `print_result()` inte är definierad för C, som ärvt från A. Klass D däremot har ärvt metoden från B.

10.1 Klassdiagram

Ett klassdiagram visualiserar hur klasser hänger ihop. Det innehåller ingen information om objekt. Man kan ange att en klass ärver från en annan (relationen “is-a”, dvs “är en”) och att en klass innehåller objekt från en annan klass (“aggregation” och “komposition”).

Det finns många sätt att rita klassdiagram och informell notation används ofta. Det finns professionella verktyg, som ofta bygger på industristandarden UML, som genererar klassdiagram från kod. T.ex. är fig. 10.1 genererad m.h.a. Python-biblioteket pyreverse (som är en del av kodanalysverktyget pylint).



Figur 10.1: Klassdiagram för A, B, C och D

Ett *objektdiagram* liknar till stor del ett klassdiagram, men förklarar *tillståndet* i ett program vid en viss tidpunkt: t.ex. skulle ett objektdiagram av ovan visa att `a.vals` innehåller `[5]`, `b.vals` innehåller `[10, 20]`, `c.vals` innehåller `[5, 10]` och `d.vals` innehåller `[5, 10, 10]`. Klassdiagrammet i fig. 10.1 förklarar relationer mellan klasser oavsett tidpunkt.

10.2 Substitutionsprincipen

Det är erkänt svårt att bestämma bra klasser. Många böcker har skrivits på ämnet, med många olika tillvägagångssätt för att bestämma vilken klass-struktur man ska ha. Det här kan bli väldigt komplicerat

fort, men en väldigt bra tumregel, som är känd som [Liskov's substitutionsprincip](#) (efter MIT professorn och Turingprisvinnaren Barbara Liskov), är:

- Allt som gäller för en klass ska gälla för dess subclasser.

Principen ger *god design*. Det kanske låter som en självklar princip, men det är lätt att vilja plocka bort funktionalitet i subclasser och det finns många exempel som bryter mot principen.

10.3 Funktioner som ger information om klasser

Två användbara inbyggda funktioner när det gäller klasser är `isinstance` och `issubclass`. Den första funktionen testar om ett givet *objekt* är en *instans* av en klass, och den andra om en *klass* är en subclass av en annan *klass*, dvs om en klass ärver från en annan.

Om vi använder oss av klassdefinitionerna (A, B, C, D) och instanserna (a, b, c, d) ovan, så kan vi t.ex. göra:

```
isinstance(a, A) # a is an instance of the class A
isinstance(d, A) # d is an instance a class that inherits from A
isinstance(d, C) # d is not an instance of a class that inherits from C

issubclass(A, A)
issubclass(D, A)
issubclass(D, C)
```

```
True
True
False

True
True
False
```

Lägg märke till att det inte bara är den *direkta* basklassen som kollas med funktionerna, utan även vidare tillbaka i "arvsledet".

I programmeringsspråk som stödjer multipelt arv (som Python, mer om det i kapitel 11) så kallas ett sådant "arvsled" *method resolution order* (förkortas vanligtvis som *MRO*). MRO specificerar i vilken ordning basklasserna (om flera) till en klass Python söker efter metoder. I Python har klasser en metod `.mro` som visar klassens MRO. T.ex:

```
A.mro()
D.mro()

[__main__.A, object]
[__main__.D, __main__.B, __main__.A, object]
```

Här kan vi se att D ärver från B (som är den första klassen efter D som söks för om en metod finns D) och vidare från A. Den sista elementet i listan, `object`, är den *fundamentala byggstenen* för typer i Python och är därför sist i listan och både A och D har detta som en del av sin MRO. När det gäller Python får man ofta höra "Everything is an object in Python":

```
isinstance(a, object)
isinstance(print, object) # function print is an object
issubclass(str, object)
issubclass(int, object)
```

```
True
True
True
True
```

10.4 Exempel: Geometriska figurer

Som ett exempel på arv ska vi nu implementera klasser som representerar geometriska objekt (figurer) som samlar vanliga/typiska operationer som man kan vilja göra på dessa figurer. Meningen är att med dessa klasser:

- Illustrera värdet av arv
- Illustrera *överlagring av metoder* (eng.: *method overriding/overloading*)
- Illustrera värdet av *inkapsling* (eng.: *encapsulation*, samling av data-attribut och de metoder som agerar på datat)
- Rita klassdiagram

Vi vill skriva kod som implementerar Python-objekt som representerar olika geometriska figurer som består av (raka) sidor. För att följa Liskovs substitutionsprincip, så vill vi ha en basklass som representerar det mest allmänna fallet: en polygon. Subklasserna ska lägga till data och funktionalitet som representerar en specifik typ av geometrisk figur.

Vi börjar med att definiera basklassen Polygon, som är menad att vara en klass som täcker det allmänna fallet men med vissa begränsningar.

```
class Polygon:

    __name__ = "Polygon"    # Klass attribut

    def __init__(self, n_sides, sides=None):
        self.n_sides = n_sides
        if sides:
            self.sides = sides
        else:
            self.sides = [1 for _ in range(n_sides)]
            # Standardvärde sätter alla sidor till längd 1
        self.check_sides()

    # "Private" methods
    def _get_side(self, side_no):
        s = float(input("Enter side " + str(side_no + 1) + ": "))
        return s

    # "Public" methods
    def check_sides(self):
```

```

    # Check that all sides have positive lengths
    if (not self.sides or
        not all(map(lambda x: x > 0, self.sides))):
        raise ValueError("Invalid side lengths")

    # Check that the number of sides are correct
    if self.n_sides != len(self.sides):
        raise ValueError(
            "Wrong number of sides specified in the sides list"
        )

    def set_sides_from_input(self, n_unique_sides = None):
        if n_unique_sides and self.n_sides % n_unique_sides:
            raise ValueError(
                "The total number of sides must be an integer"
                " multiple of the number of unique sides"
            )

        if not n_unique_sides:
            n_unique_sides = self.n_sides

        print("Please enter " + str(n_unique_sides) +
              " unique side lengths for a " + self.__name__)
        unique_sides = [self._get_side(i) for i in range(n_unique_sides)]
        self.sides = unique_sides * (self.n_sides // n_unique_sides)
        try:
            self.check_sides()
        except ValueError as e:
            print(e)
            self.set_sides_from_input() # Recursive call

    def display_sides(self):
        for i in range(self.n_sides):
            print("Length of side", i + 1, ":", self.sides[i])

```

Klasskonstruktorn tar två argument, ett obligatoriskt argument `n_sides` som specificerar hur många sidor polygonen skall ha och ett nyckelordsargument `sides` som har standardvärdet `None`. I konstruktorn så sätts instansattributen till de värden som de formella parameterarna har; om `sides` inte har specificerats så sätts instansattributet till en lista med `n_sides` stycken 1:or, dvs att alla sidor har längd 1. För att en polygon ska vara giltig måste några enkla krav vara uppfyllda, dessa kollas genom att anropa på hjälpmetoden `check_sides` som lyfter ett särfall om något krav inte är uppfyllt.

Det första kravet som kollas i `check_sides` är att alla sidor är positiva. Det görs med `map` och en anonym funktion (`lambda`) och om kravet inte är uppfyllt lyfts ett särfall. Den första delen av villkoret för `if`-satsen i `check_sides` kontrollerar att `sides` är satt, dvs om `sides == None` så exekveras satsen *oavsett* om det andra villkoret är uppfyllt eller ej. Denna sats är av värde då `map` skulle ge ett fel om `sides == None`, eftersom `None` inte går att iterera över. Genom att utvärdera `if not sides` först så exekveras `if`-satsen om `sides == None` (och `map` exekveras aldrig) *eller* så är `sides` satt (vi antar att det är en lista) och `map` kan iterera över `sides`. Det andra kravet som sedan kollas är att antalet sidlängder i listan `sides` faktiskt

stämmer överens med antalet sidor som polygonen ska ha (`n_sides`).

Huvudmetoden i `Polygon` är `set_sides_from_input`, som när den anropas först skriver ut ett meddelande och sedan med hjälp av en listomfattning läser in `n_unique_sides` antal sidlängder. Vi vill här ge möjligheten att kunna specificera att en polygon har `n_sides` *antal* sidor men bara har ett visst antal unika sidlängder, till exempel skulle `n_unique_sides=1` betyda att alla sidor i polygonen är lika långa. Vi vill dock begränsa hur en användare kan specificera antalet unika sidor i en polygon: det totala antalet sidor måste vara en heltalsmultipel av antalet unika sidor. Vi kollar att detta är uppfyllt genom att se om villkoret `n_sides % n_unique_sides` utvärderas till `True`, dvs om det finns en rest efter divisionen (ett tal skiljt från 0) så lyfter vi ett `ValueError` (kom ihåg att alla tal som är skilda från 0 utvärderas till `True` i Python). Inläsningen av *varje enskild* sidlängd görs av den privata metoden `_get_side`. Eftersom vi redan sett till att `n_unique_sides` har ett tillåtet värde, så kan vi multiplicera listan med (unika) inlästa sidlängder med `n_sides // n_unique_sides` för att slutligen få en lista som innehåller *alla* sidlängder (dvs `len(sides) == n_sides`): i fallet att vi har färre unika sidlängder än totalt antal sidlängder, så repeteras listan med de unika sidlängderna ett lämpligt antal gånger. Slutligen försäkrar vi oss att alla sidlängder är tillåtna med den privata metoden `check_sides`. Om en `ValueError` lyfts i `check_sides` så printas felmeddelandet i `error` och `set_sides_from_input` anropas på nytt.

Metoden `display_sides` skriver ut alla sidlängder i `sides`.

Nu när vi har en basklass, så kan vi använda arv för att definiera subklasser som representerar specifika polygoner. Till exempel kan en triangel-typ representeras så här:

```
class Triangle(Polygon):

    __name__ = "Triangle"

    def __init__(self, sides=None):
        super().__init__(3, sides)

    def check_sides(self):
        a, b, c = sorted(self.sides)
        triangle_condition = c < (a + b)
        if not triangle_condition:
            raise ValueError(
                "The sum of the shortest sides must be greater than "
                "the length of the longest side in a "
                + self.__name__
            )
        super().check_sides() # Villkoren för en triangel är striktare än en polygon

    # method specific to triangles
    def get_area(self):
        a, b, c = sorted(self.sides)
        # calculate the semi-perimeter
        s = (a + b + c) / 2
        return (s * (s - a) * (s - b) * (s - c))**0.5
```

Här ser vi att `Triangle` ärver från `Polygon` och att vi i `Triangles` konstruktor anropar *basklassens* konstruktor med två argument (`3, sides`). Att vi explicit anropar konstruktorn med `super` gör att `Polygon`

instantieras med `n_sides=3`, `sides=sides` och att en instans av `Triangle` ärver både instansattribut och metoder från `Polygon`. Vi överlagrar klassattributet `__name__` så att namnet för klassinstansen ges som `Triangle`. Vi överlagrar även `check_sides` så att den kollar att sidlängderna kan sättas ihop till en riktig triangel utöver att kolla kraven som gäller för alla polygoner. Vi *utökar* basklassens funktionalitet genom att lägga till en metod `get_area` som beräknar en triangelns area.

På ett liknande sätt kan vi definiera en rektangel. Rektanglar har fyra sidor, och max två *unika* sidlängder. Det här löses enkelt genom att definiera en klass `Rectangle` som ärver från `Polygon` som anropar basklassens konstruktor med argument korrekta argument, och att sedan överlagra `check_sides` och `set_sides_from_input` så att vi inte kan ha mer än två unika sidlängder.

```
class Rectangle(Polygon):

    __name__ = 'Rectangle'

    def __init__(self, sides=None):
        super().__init__(4, sides)

    def check_sides(self):
        a, b, c, d = self.sides
        rectangle_condition = a == c and b == d
        if not rectangle_condition:
            raise ValueError(
                "Opposite sides must be of equal length in a "
                + self.__name__
            )
        super().check_sides()
        # Villkoren för en rektangel inkluderar villkoren för en polygon

    # method specific for rectangles
    def get_area(self):
        return self.sides[0] * self.sides[1]

    # method specific for rectangles
    def get_diagonal(self):
        return (self.sides[0]**2 + self.sides[1]**2)**0.5

    def set_sides_from_input(self, n_unique_sides=2):
        if n_unique_sides > 2: # En rektangel har 2 eller 1 unika sidlängder
            raise ValueError(
                "A " + self.__name__ + " can't have more than 2 unique side lengths"
            )
        return super().set_sides_from_input(n_unique_sides)
```

`Rectangle` definierar också metoder som är specifika för rektanglar: `get_area` och `get_diagonal`.

Slutligen definierar vi ett specialfall av en rektangel, nämligen en kvadrat `Square`. En kvadrat är en rektangel, så både area och diagonal beräknas på samma sätt som i `Rectangle`. Det är därför lämpligt att ärva funktionaliteten av `Rectangle`.

```

class Square(Rectangle):

    __name__ = 'Square'

    def __init__(self, sides=None):
        super().__init__(sides)

    def check_sides(self):
        a, b, c, d = self.sides
        square_condition = a == b == c == d
        if not square_condition:
            raise ValueError(
                "All sides must have equal length in a "
                + self.__name__
            )
        super().check_sides()
        # Villkoren för en kvadrat inkluderar villkoren för en rektangel

    def set_sides_from_input(self):
        return super().set_sides_from_input(n_unique_sides=1)
        # En kvadrat har exakt 1 unik sidlängd

    def get_corner_coordinates(self):
        s = self.sides[0]/2
        return ((-s, -s), (-s, s), (s, s), (s, -s))

```

Skillnaden mellan en kvadrat och en rektangel är att det bara finns *en unik* sidlängd. För att åstadkomma detta så överlagrar vi `set_sides_from_input` igen och den här gången sätter att `n_unique_sides` alltid är lika med ett. Vi överlagrar även `check_sides` och lägger till det striktare villkoret att alla sidor måste vara lika långa. Slutligen utökar vi kvadrat med en metod som returnerar koordinaterna för kvadratens hörn.

Man kan nu använda sig av ovan definierade klasser:

```

t = Triangle()
t.input_sides()
print('The area of the triangle is:', t.get_area())
t.display_sides()

r = Rectangle()
r.input_sides()
print('The area of the rectangle is:', r.get_area())
print('The diagonal of the rectangle is:', r.get_diagonal())

s = Square()
s.input_sides()
print('The area of the square is:', s.get_area())
print('The diagonal of the square is:', s.get_diagonal())
print('The coordinates of the square corners:\n',
      s.get_corner_coordinates())

```

```

Please enter the 3 unique side lengths for a Triangle
Enter side 1 : 1
Enter side 2 : 2.5
Enter side 3 : 2
The area of the triangle is: 0.9499177595981665
Length of side 1 : 1.0
Length of side 2 : 2.5
Length of side 3 : 2.0
Please enter the 2 unique side lengths for a Rectangle
Enter side 1 : -1
Enter side 2 : 2
Invalid side lengths.
Please provide positive side lengths.
Please enter the 2 unique side lengths for a Rectangle
Enter side 1 : 1.2
Enter side 2 : 1.9
The area of the rectangle is: 2.28
The diagonal of the rectangle is: 2.247220505424423
Please enter the 1 unique side lengths for a Square
Enter side 1 : 6
The area of the square is: 36.0
The diagonal of the square is: 8.48528137423857
The coordinates of the square corners are:
((-3.0, -3.0), (-3.0, 3.0), (3.0, 3.0), (3.0, -3.0))

```

10.4.1 Implementera jämförelser

Som vi gick igenom i kapitel 9, så kan man genom att använda sig av specifika privata metoder implementera jämförelser för sin egen typ (klass). Dessa metoder fungerar väldigt bra att ärva från en basclass när någon logisk jämförelse mellan objekt kan göras.

Antag t.ex. att vi vill implementera <, >, och == baserat på arean av ett objekt. För de geometriska figurerna Triangle, Rectangle och Square så kan vi ju genom get_area räkna ut arean av ett objekt. Vi kan lägga till nedan funktioner i Triangle och Rectangle:

```

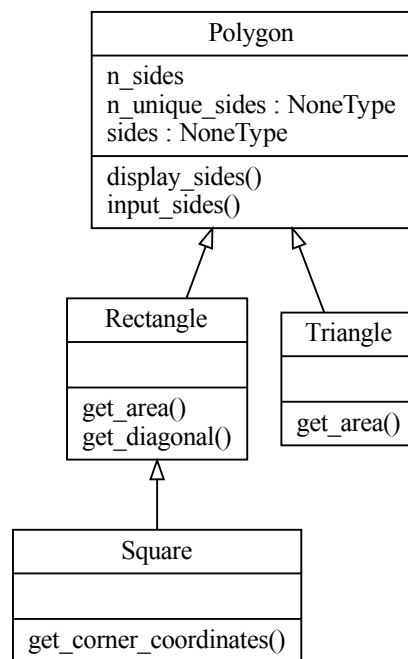
def __lt__(self, other):
    return self.get_area() < other.get_area()

def __gt__(self, other):
    return self.get_area() > other.get_area()

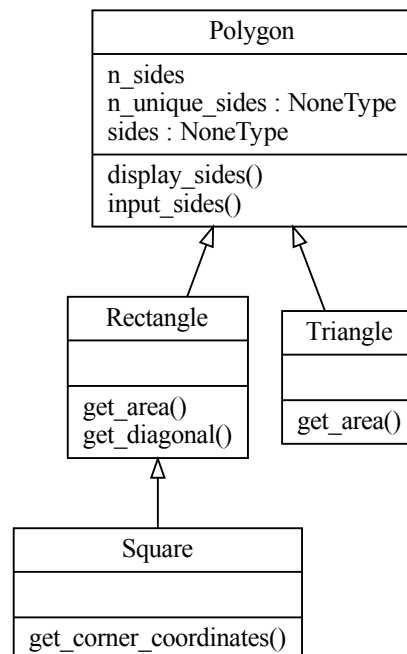
def __eq__(self, other):
    return self.get_area() == other.get_area()

```

Lägg märke till att vi *inte* behöver lägga till detta i Square, då denna klass ärver från Rectangle.



Figur 10.2: Klassdiagram för geometriska figurer.



I fig. 10.2 illustrerar vi klassdiagram för det här exemplet.

10.5 Exempel: Monster Go

Följande exempel är inspirerat av Pokemon Go och är ett lite större exempel. Exemplet är lite konstruerat, men det är för att framhäva viktiga poänger som illustrerar värdet av objekt-orienterad programmering.

Målet med detta exempel är att:

- Illustrera värdet av arv.
- Illustrera värdet av abstraktion: dölja implementationsdetaljer.

10.5.1 En basklass: ProgMon

Klassen ProgMon innehåller attribut och metoder som ska vara gemensamma för alla objekt. Vi försöker alltså även här följa Liskovs substitutionsprincip.

Vi kommer i följande exempel att använda oss mycket av privata metoder och attribut (både de som används av Python och börjar/slutar med `__`, t.ex. `__str__` och de som är privata för vår implementation och börjar med `_`, t.ex. instansattributen `_attack`).

```

class ProgMon:
    def __init__(self):
        self._attack = 0.0
        self._defense = 0.0
        self._caffeinated = False
  
```

```

        self._unit_testing = False

    def __str__(self):
        return "<ProgMon object>"

    def get_attack(self):
        if self._caffeinated:
            return 2 * self._attack
        else:
            return self._attack

    def get_defense(self):
        if self._unit_testing:
            return 2 * self._defense
        else:
            return self._defense

    def fight(self, other_progmon):
        """
        Attack is the best form of defense!
        Returns (True, False) if self wins, (False, True) if other_progmon
        wins and (False, False) otherwise.
        """
        if self.get_attack() > other_progmon.get_defense():
            return (True, False)
        elif other_progmon.get_attack() > self.get_defense():
            return (False, True)
        else:
            return (False, False)

```

10.5.2 Tre subklasser

I subklasserna Hacker, Newbie, och Guru lägger vi in specialisering (data och beteende som är specifika för klassen). Vi gör detta genom att ändra värden för de privata attributen och definiera `__str__`.

```

class Hacker(ProgMon):
    def __init__(self):
        super().__init__() # Call the superclass constructor!
        self._attack = 0.5
        self._defense = 0.25

    def __str__(self):
        return "<Hacker A=" + str(self._attack) + ">"

class Newbie(ProgMon):
    def __init__(self):
        super().__init__()
        self._attack = 0.15
        self._defense = 0.1

```

```

    def __str__(self):
        return "<Newbie>"

class Guru(ProgMon):
    def __init__(self):
        super().__init__()
        self._attack = 1.0
        self._defense = 1.0

```

Provkör med:

```

h = Hacker()
n = Newbie()
g = Guru()
print(n.fight(h))
print(h.fight(n))
print(g.fight(h))

```

```

(False, True)
(True, False)
(True, False)

```

Obs: Gemensam funktionalitet i basklassen gör att vi undviker duplicering av kod.

10.5.3 Anpassning

Antag att man inte gillar tanken på att en Guru försöker vinna kamper. Vi kan implementera en “Don’t win, don’t lose” för en Guru som initierar en “fight”, genom att definiera en ny version av Guru:

```

class Guru(ProgMon):
    def __init__(self):
        super().__init__()
        self._attack = 1.0
        self._defense = 1.0

    # method overloading
    def fight(self, other_progmon):
        '''
        Don't win, don't lose.
        '''
        # same type of return value as in the super class
        return (False, False)

```

Provkör med ny definition: `g.fight(h)` ger `(False, False)`.

Observera att vi ändrar beteendet hos en subclass genom att definiera om en metod i subclassen.

10.5.4 Abstraktion undviker detaljkunskap

Basklassen `ProgMon` definierar ett *gränssnitt* som ger en praktisk abstraktion: du behöver inte kunna några detaljer om basklassen eller dess subclasser om du använder gränssnittet.

10.5.5 Exempel: en hacker dojo

Antag att vi skapar en klass för en “Hacker Dojo”, där ProgMon:s samlas för att träna. Man ska kunna utmana en Hacker Dojo, vilket kräver att man vinner en kamp mot varje medlem för att kunna säga att man vunnit. Förlust mot en medlem ger förlust av utmaningen.

```
class HackerDojo:
    def __init__(self):
        self._members = []

    def add_member(self, m):
        self._members.append(m)

    def challenge(self, pm):
        for monster in self._members:
            win, loose = pm.fight(monster)
            if loose:
                # Lost against one member, challenge failed.
                return False
        # Won against all members of the dojo
        return True
```

Utprovning:

```
dojo = HackerDojo()
dojo.add_member(h)
dojo.add_member(n)
print(dojo.challenge(g))
print(dojo.challenge(n))
```

```
True
False
```

Här vinner dojon mot gurun eftersom vår guru varken vinner eller förlorar (se den omdefinierade metoden `fight` i klassen `Guru`) och i metoden `challenge` så har vi programmerat att `HackerDojo` tycker att den vinner om den inte förlorar någon fjant.

Obs: det enda vi använder i implementationen är gränssnittet från `ProgMon`. Vi får automatiskt den lite annorlunda specialiseringen från `Guru`. Eller snarare: vi behöver inte oroa oss för sådana detaljer.

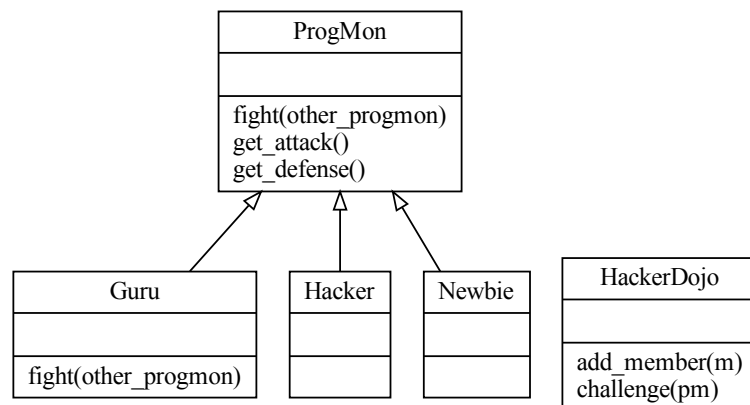
10.5.6 Ändrade detaljer

En av de stora vinsterna med objektorientering (men inte unikt för OO) är abstraktion. Genom att sätta ihop bra gränssnitt (dvs väl valda metoder) “framtidssäkrar” vi ett program: det blir lätt att ändra detaljer i en del av programmet, utan att andra delar påverkas. Detta underlättar ofantligt mycket i större kodprojekt som kan ha tusentals eller miljoner rader kod.

Antag att vi inser att `_attack` inte är ett bra attribut. Styrkan ges av erfarenhet, level, och andra attribut som blir specifika för subklasserna.

- Vilka ändringar behövs göras, och vilka klasser påverkas?
- Är `get_attack` en bra metod?

I fig. 10.3 illustrerar vi det här exemplet med ett klassdiagram.



Figur 10.3: Klassdiagram för Monster Go.

10.6 Uppgifter

1. Skapa en klass Equilateral som ärver från Triangle i exemplet ovan och metodöverlagrar `get_area`.

Tips: Arean för en liksidig triangel är $\sqrt{3}a^2/4$, där a är sidlängden. Man kan från standardmodulen `math` importera funktionen `sqrt`.

2. Skapa en klass Student som ärver från ProgMon (även instansattribut), men som lägger till ett instansattribut `is_learning` av typen `bool` och metodöverlagrar både `get_attack` och `get_defense` i ProgMon. Klassen Students metoder `get_attack` och `get_defense` skall använda sig av ProgMon `get_attack` och `get_defense` men ska multiplicera styrkan med två om Student lärs, dvs om `is_learning` är `True`. Du kan bestämma vad instansattributen `_attack` och `_defense` instansieras till i klassen själv.
3. I uppgift 5 och 6 i kapitel 9 implementerade vi en class Dog. Implementera en klass Tournament som

1. rankar hundarna baserat på antal tricks de kan, och
2. skriver ut resultaten.

Tips: hämta inspiration från hur HackerDojo relaterar till ProgMon.

4. Skriv en basklass Vehicle, som innehåller instansattributen `wheels`, `wings` och `sound`. Utöver detta ska Vehicles metod `__str__` returnera en sträng med information om fordonet (hur många hjul, etc., fordonet har). Skapa tre subclasser Car, Motorcycle och Plane som alla ärver från Vehicle, men har rätt antal hjul/vingar och gör "rätt" ljud.

Exempel-användning och output skulle vara:

```
car = Car()
mc = Motorcycle()
```

```
plane = Plane()
print(car)
print(mc)
print(plane)
```

```
A Car has:
4 wheels
and makes a AAARRRRRRRR! sound

A Motorcycle has:
2 wheels
and makes a VROOM VROOM! sound

A Plane has:
2 wings
and makes a WHOOSH! sound
```

5. Skriv en egen sträng-klass som implementerar ("hanterar") subtraktion och division. Med *subtrahera* en sträng menar vi här att ta bort en delsträng, och $s1 - s2$ innebär att den högraste förekomsten av $s2$ i $s1$ tas bort. Med *division* menas att alla förekomster av strängen till höger om $/$ operatoren tas bort från huvudsträngen, dvs $s1 / s2$ innebär att *alla* förekomster av $s2$ tas bort från $s1$.

Tips: skapa en subclass MyString till str, och läs på om vilka privata metoder som behövs implementeras för $-$ och $/$.

Exempel:

```
s1 = MyString("This is my own string!")
s2 = MyString("My, oh my, oh Oh.... oh")
op1 = MyString('my')
op2 = MyString('oh')
print("Removing first rightmost and then all instances of 'my' from s1:")
print(s1 - op1)
print(s1 / op1)
print("\nRemoving first rightmost and then all instances of 'oh' from s2:")
print(s2 - op2)
print(s2 / op2)
print("\nNo change in s1 expected here:")
print(s1 - 'hello')
print(s1 / 'hello')
```

```
Removing first rightmost and then all instances of 'my' from s1:
This is own string!
This is own string!
```

```
Removing first rightmost and then all instances of 'oh' from s2:
My, oh my, oh Oh....
My, my, Oh....
```

```
No change in s1 expected here:
```

```
This is my own string!  
This is my own string!
```


Kapitel 11

Objektorientering 3: mer om arv

11.1 Olika typer av arv

I förra kapitlet såg vi olika exempel på arv i Python. Olika former av arv har olika namn inom objektorientering:

1. Enkelt arv (eng. *single inheritance*): klass B ärver endast från klass A.
2. Mångnivåarv (eng. *multilevel inheritance*): klass B ärver från A och klass C ärver sen från klass B.
3. Hierarkiskt arv (eng. *hierarchical inheritance*): klass A är superklass till B, C, D...
4. Multipelt arv (eng. *multiple inheritance*): klass C ärver från *både* A och B (men de ärver inte från varandra).
5. Hybridarv (eng. *hybrid inheritance*): en mix av två eller fler typer av arv.

Vi såg även att man kan illustrera olika typer av arv med hjälp av klassdiagram. Förra kapitlet innehöll exempel på enkelt, mångnivå-, hierarkiskt och hybrid-arv, så vi ska nu titta på multipelt arv.

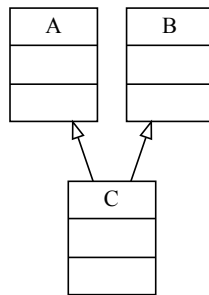
Det är inte alla programmeringsspråk som tillåter multipelt arv, men det gör Python. Betrakta följande exempel:

```
class A:
    pass    # some code

class B:
    pass    # some more code

class C(A, B):
    pass    # even more code
```

Vi kan illustrera detta med hjälp av ett klassdiagram:



Figur 11.1: Klassdiagram för A, B och C

Här har C alltså två superklasser, A och B, vilka inte ärver från varandra. Detta kan verka oskyldigt och man kan undra varför inte alla språk stödjer det. Men rent generellt är det lätt hänt att multipelt arv ger kaos på grund av så kallade *diamantproblem*.

11.1.1 Diamantproblem

Betrakta följande kod:

```
class A:
    def f(self):
        print("f of A called")

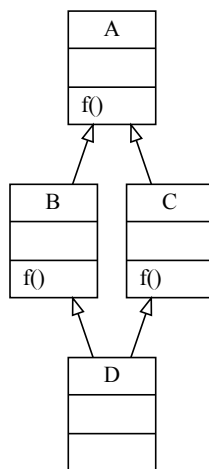
class B(A):
    def f(self):
        print("f of B called")

class C(A):
    def f(self):
        print("f of C called")

class D(B,C):
    pass

d = D()
d.f()
```

Detta kallas för ett diamantproblem då får man en romb om man ritat upp klassdiagrammet:



Figur 11.2: Klassdiagram för A, B, C och D

Kör vi koden ovan får vi

f of B called

Men om vi istället kör följande där arvet är D(C,B) får vi:

```

class A:
    def f(self):
        print("f of A called")

class B(A):
    def f(self):
        print("f of B called")

class C(A):
    def f(self):
        print("f of C called")

class D(C,B):
    pass

d = D()
d.f()
  
```

f of C called

Så utdata har nu ändrats då man ärver i en annan ordning! Denna typ av beroenden vill man helst inte behöva tänka på när man programmerar och därför tycker man att det är ett problem. Diamantproblem kan uppstå med multipelt arv. Man kan lätt föreställa sig hur kod snabbt blir oöverskådlig om man har

många klasser med en massa metoder och attribut vilka ärver från varandra på detta sätt. Denna typ av problem är vad som lätt orsakar kaos i kod med multipelt arv och varför många språk inte tillåter det.

11.2 Exempel: Algebraiska uttryck

Vi ska nu titta på ett till exempel på arv. Poängen med detta exempel är att illustrera två saker:

1. Hur man kan använda klasser för att *strukturera* data.
2. Hur man kan använda rekursion i kombination med klasser.

Denna kombination är väldigt kraftfull och låter oss lösa många olika typer av programmeringsproblem på ett enkelt sätt. Vi börjar med att skriva ett program för att representera och modifiera algebraiska uttryck. Dessa uttryck kan innehålla tal, variabler, +, * och *matchade* paranteser. Ett exempel på ett algebraiskt uttryck är $x * y + 7$ och ett annat är $x * (y + 7)$.

Hur ska vi tänka för att representera dessa i Python? En naiv lösning skulle kunna vara att helt enkelt använda strängar, så $x * y + 7$ skulle representeras som strängen " $x * y + 7$ ". Men låt oss säga att vi vill byta ut x till 3 och y till 2 och sedan beräkna värdet av uttrycket... Detta blir *väldigt* komplicerat med strängar!

Ett annat problem med strängar är hur man ska hantera paranteser. Enligt konventionen att * binder starkare än + så ska ju $x * y + 7$ tolkas som $(x * y) + 7$ och inte $x * (y + 7)$. Har man inte koll på detta kommer man ju få fel svar när man utvärderar uttryck. Ett mycket smidigare sätt är att använda *träd*. På så sätt kan vi representera algebraiska uttryck med två dimensioner istället för bara en!

Ett träd är en datastruktur där ett element antingen är ett löv eller en nod med ett antal grenar som leder till "underträd" (eng. *subtree*). Så $(x * y) + 7$ kan representeras entydigt som ett träd med översta noden + och två underträd som innehåller $x * y$ och 7. Värdet 7 är ett löv i trädet då det inte har några underträd. Uttrycket $x * y$ är även det ett träd, denna gång med noden * längst upp och x och y som löv. Vi kan rita detta som:

```
      +
     / \
    *   7
   / \
  x   y
```

Obs: dataloger ritar träd upp-och-ned jämfört med riktiga träd!

Uttrycket $x * (y + 7)$ är istället följande träd:

```
      *
     / \
    x   +
       / \
      y   7
```

Denna typ av träd kallas *binära* träd då alla noder har två eller noll underträd. Låt oss nu representera denna typ av träd med algebraiska uttryck i Python! Vi börjar med en tom superklass för uttryck som i sin tur har 3 subclasser (en för siffror, en för variabler och en för +):

```
class Expr:
    pass
```

```

class Constant(Expr):

    def __init__(self, value):
        self._value = value

    def __str__(self):
        return str(self._value)

class Var(Expr):

    def __init__(self, name):
        self._name = name

    def __str__(self):
        return self._name

class Plus(Expr):

    def __init__(self, left, right):
        self._left = left
        self._right = right

    def __str__(self):
        return str(self._left) + " + " + str(self._right)

print(Plus(Constant(3), Var("x")))

```

```
3 + x
```

Notera att anropet till `str` är *rekursivt* i `__str__` för `Plus`. Här anropas `str` metoden för vänstra och högra underträdet. Vilken funktion som sen kommer köras beror på vad dessa är.

Låt oss nu lägga till `*`, detta kan göras genom:

```

class Times(Expr):

    def __init__(self, left, right):
        self._left = left
        self._right = right

    def __str__(self):
        return str(self._left) + " * " + str(self._right)

print(Plus(Times(Var("x"), Var("y")), Constant(7)))

```

```
x * y + 7
```

Men denna definition är ju exakt samma som `Plus`, bortsett från att vi skriver ut `*` istället för `+`. Kopiera och klistra in kod är farligt då man lätt introducerar buggar! En bättre lösning är att istället introducera en ny klass för binära operatorer vilken `Plus` och `Times` sen ärver ifrån:

```

class BinOp(Expr):
    def __init__(self, left, right):
        self._left = left
        self._right = right

class Plus(BinOp):
    def __str__(self):
        return str(self._left) + " + " + str(self._right)

class Times(BinOp):
    def __str__(self):
        return str(self._left) + " * " + str(self._right)

print(Plus(Times(Var("x"), Var("y")), Constant(7)))

```

```
x * y + 7
```

Detta verkar ju fungera bra, men det blir fel om vi skriver:

```
print(Times(Var("x"), Plus(Var("y"), Constant(7))))
```

```
x * y + 7
```

Vi får exakt samma som ovan! Detta är knas då vi glömt bort att sätta in paranteser. En naiv lösning på detta problem är att alltid sätta in parenteser runt underuttryck i Plus och Times:

```

class BinOp(Expr):
    def __init__(self, left, right):
        self._left = left
        self._right = right

class Plus(BinOp):
    def __str__(self):
        return "(" + str(self._left) + ") + (" + str(self._right) + ")"

class Times(BinOp):
    def __str__(self):
        return "(" + str(self._left) + ") * (" + str(self._right) + ")"

print(Plus(Times(Var("x"), Var("y")), Constant(7)))

print(Times(Var("x"), Plus(Var("y"), Constant(7))))

```

```

((x) * (y)) + (7)
(x) * ((y) + (7))

```

Detta är ju rätt, men vi har satt in onödigt många paranteser! Vad vi egentligen vill få ut är:

```
print(Plus(Times(Var("x"),Var("y")),Constant(7)))
```

```
print(Times(Var("x"),Plus(Var("y"),Constant(7))))
```

```
x * y + 7
x * (y + 7)
```

Hur kan vi lösa detta?

Lösning: dekorera alla uttryck med hur hårt de binder. Med denna extra information kan vi avgöra om vi ska skriva ut paranteser runt underuttryck eller inte. Vi gör detta genom att lägga till en klassvariabel *prec* (för *precedence*) till Expr klassen och sen skriva över den i Plus och Times. Vi har sedan en hjälpfunktion *parens* som avgör om paranteser ska skrivas ut runt ett uttryck eller inte.

```
class Expr:

    # default precedence very high so that we don't put parantheses
    prec = 1000

class Constant(Expr):

    def __init__(self, value):
        self._value = value

    def __str__(self):
        return str(self._value)

class Var(Expr):

    def __init__(self, name):
        self._name = name

    def __str__(self):
        return self._name

class BinOp(Expr):

    def __init__(self, left, right):
        self._left = left
        self._right = right

# Function for inserting parentheses around a string if precedence p1
# is smaller than precedence p2
def parens(p1,p2,s):
    if p1 < p2:
        return "(" + s + ")"
    else:
        return s
```

```

class Plus(BinOp):

    prec = 1

    def __str__(self):
        s1 = parens(self._left.prec, self.prec, str(self._left))
        s2 = parens(self._right.prec, self.prec, str(self._right))
        return s1 + " + " + s2

class Times(BinOp):

    prec = 2

    def __str__(self):
        s1 = parens(self._left.prec, self.prec, str(self._left))
        s2 = parens(self._right.prec, self.prec, str(self._right))
        return s1 + " * " + s2

print(Plus(Times(Var("x"), Var("y")), Constant(7)))

print(Times(Var("x"), Plus(Var("y"), Constant(7))))

print(Times(Var("x"), Times(Var("y"), Plus(Constant(3), Var("z")))))

x * y + 7
x * (y + 7)
x * y * (3 + z)

```

11.3 Exempel: Binära träd

Vi kan generalisera datatypen för algebraiska uttryck till godtyckliga binära träd. På detta sätt kan vi skriva våra funktioner en gång för alla och kommer inte behöva skriva om dem för varje typ av binära träd vi kan tänkas behöva.

Ett binärt träd är antingen en förgrening (med nodvärde och vänster/höger underträd) eller ett löv (med värde). Vi lägger in följande funktionalitet i dessa:

- `__str__`: konvertera till en sträng.
- `member(x)`: testa om x finns i trädet.
- `map_tree(f)`: applicera funktionen f på alla värden i trädet (notera att detta är en högre ordningens funktion då den tar en funktion som argument).
- `linearize()`: konvertera trädet till en lista.

I detta exempel är huvudklassen `BinTree` tomt, men det är ändå bra att ha med om man en dag vill lägga in något i den som vi gjorde med `prec` värdet för algebraiska uttryck.


```

class BinTree:
    pass

class Branch(BinTree):

    def __init__(self, node, left, right):
        self._node = node
        self._left = left
        self._right = right

    def __str__(self):
        n , l , r = str(self._node) , str(self._left) , str(self._right)
        return "(" + n + "," + l + "," + r + ")"

    def member(self, x):
        return self._node == x or self._left.member(x) or self._right.member(x)

    def map_tree(self, f):
        return Branch(f(self._node), self._left.map_tree(f), self._right.map_tree(f))

    def linearize(self):
        return self._left.linearize() + [self._node] + self._right.linearize()

class Leaf(BinTree):

    def __init__(self, val):
        self._val = val

    def __str__(self):
        return "(" + str(self._val) + ")"

    def member(self, x):
        return self._val == x

    def map_tree(self, f):
        return Leaf(f(self._val))

    def linearize(self):
        return [self._val]

t = Branch(-32, Leaf(2), Branch(1, Branch(23, Leaf(4), Leaf(-2)), Leaf(12)))

print(t)
print(t.member(12))
print(t.member(-123))

print(t.map_tree(lambda x: 0))
print(t.map_tree(lambda x: x ** 2))

```

```
print(t.linearize())
(-32,(2),(1,(23,(4),(-2)),(12)))
True
False
(0,(0),(0,(0,(0),(0)),(0)))
(1024,(4),(1,(529,(16),(4)),(144)))
[2, -32, 4, 23, -2, 1, 12]
```

Notera att alla funktioner som vi definierat är rekursiva. Detta exempel visar hur vi på ett enkelt sätt kan kombinera klasser och rekursion för att representera och modifiera strukturerad data.

11.4 Uppgifter

1. Vilka av exemplen i kapitel 10 är exempel på enkelt, mångnivå, hierarkiskt och hybrid arv?
2. Flytta `__str__` metoden för `Plus` och `Times` till `BinOp` på lämpligt sätt så att vi inte har någon kodduplikation. Exempen ska fortfarande skrivas ut på rätt sätt.
3. Lägg till en klass `Exp` för exponentiering av algebraiska uttryck. Den ska binda hårdare än både `+` och `*`, samt skrivas ut som operatoren `**` (dvs på samma sätt som i Python).
4. Lägg till en funktion `big_constant_sum(n,e)` där `n` är ett positivt heltal och `e` ett algebraiskt uttryck. Funktionen ska skapa ett uttryck där `e` adderas med sig själv `n` gånger. Om `n` är `0` så ska konstanten `0` returneras. Så funktionen ska alltså returnera summan

$$\sum_{i=0}^{n-1} e$$

5. Lägg till en funktion `big_sum(n,e)` där `n` är ett positivt heltal och `e` en funktion från heltal till algebraiska uttryck. Funktionen ska skapa ett uttryck som representerar

$$\sum_{i=0}^n e_i$$

6. Definera evaluering av algebraiska uttryck givet en uppslagstabell `d` från variabler till siffror. För att göra detta ska ni lägga till en metod `evaluate(d)` till `Constant`, `Var`, `Plus`, `Times` och `Exp`. Ni kan anta att alla variabler i uttrycket finns definierade i `d`.
7. † Implementera rosträd. Dessa är en typ av träd där varje nod innehåller ett värde och en lista med fler rosträd. Ett löv med värdet `5` representeras genom `RoseTree(5, [])`. Implementera en klass för rosträd som har samma metoder som `BinTree`.

Kapitel 12

Defensiv programmering

Den som har lärt sig programmera har nog också blivit tvungen att lära sig felsöka. Alla programmerare gör fel, råkar förlita sig på olämpliga antaganden, och missförstår dokumentation. Fel kan göra att programmet avbryter precis där misstaget är, men det kan också uppstå långt senare. Om division med noll uppstår så kanske det är ett symptom på ett tidigare misstag. En variabel kanske har blivit satt till 0, trots att det är orimligt, men var gjordes det och varför? Det kan vara arbetsamt att leta upp orsaken till felet. Programmeringsstilen *defensiv programmering* syftar dels till att hitta fel tidigt, så nära det egentliga misstaget, för att undvika arbetsamma felsökningssessioner, och dels till att tydliggöra sina antaganden i koden.

Defensiv programmering avser att man förväntar sig att funktioner kommer att användas på fel sätt, algoritmer blir implementerade fel, kombinationer av variablers värden uppstår som man inte har tänkt på, och att andra konstigheter sker under programutveckling. Genom att *koda* sina förväntningar på variablers värden med hjälp av **assert** på många platser i ett program kan man upptäcka felen tidigare, närmare felkällan, och därmed förenkla felsökningen.

12.1 Assert

Instruktionen **assert** condition anger att uttrycket condition förutsattes vara sant. Om uttrycket är sant händer inget, det är det förväntade uppträdandet, men om det är falskt avbryter programmet med hjälp av ett särfall, **AssertionError**. Denna typ av instruktion finns i många språk och är ett hjälpmedel för felsökning, utveckling, och underhåll av program.

Några exempel:

```
assert len(data_list) > 0
assert x != 0
assert x > 0 and y > 0
```

Dessa rader ställer tre tänkbara krav på tillståndet i ett program. Instruktionen **assert** ska utläsas som "detta gäller".

12.1.1 Avbrott och felmeddelande

Säg att vi har skrivit en divisionsfunktion på följande sätt:

```
def divide(x,y):
    assert y != 0
    return x / y
```

Om vi nu anropar divide med 0 som andra argument

```
print(divide(10,2))
print(divide(10,0))
```

så kommer ett felmeddelande skrivas ut:

```
Traceback (most recent call last):
  File "f12.py", line 6, in <module>
    print(divide(10,0))
  File "f12.py", line 2, in divide
    assert y != 0
AssertionError
```

Notera att särffallet `AssertionError` inte är tänkt att fångas med `except`; man använder `assert` för att hitta allvarliga programmeringsfel, inte för att hantera körfel (som t.ex., namngiven fil finns ej, minnet räcker inte, man har rekurerat för djupt, o.s.v.).

12.1.2 Generellt

Det generella fallet kan skrivas

```
assert condition [, comment_string]
```

där condition är ett boolskt uttryck och comment_string är en möjlig förklarande text. Man kan alltså skriva villkoren ovan som:

```
assert len(data_list) > 0, 'User submitted empty data'
assert x != 0, 'x==0 must be avoided early in the application!'
assert x > 0 and y > 0, 'Both x and y have to be positive!'
```

för att hjälpa till att förstå felen som uppstår.

Exempel:

```
def divide(x,y):
    assert y != 0, "Second argument cannot be 0 when dividing"
    return x / y
print(divide(10,0))
```

ger

```
Traceback (most recent call last):
  File "f12.py", line 6, in <module>
    print(divide(10,0))
  File "f12.py", line 2, in divide
    assert y != 0, "Second argument cannot be 0 when dividing"
AssertionError: Second argument cannot be 0 when dividing
```

12.1.3 Tumregler för assert

assert är implementerat med hjälp av sårfall, men syftet är att uttrycka antaganden i koden, inte att hantera oväntade tillstånd.

- **assert** är för problem under kodning/utveckling.
- **raise** och **try/except** är för oväntade effekter under körning.

Man kan säga att **assert** används för att fånga oväntade tillstånd i koden, **raise** används för att signalera eventuella förväntade fel och **try/except** används för att fånga mer eller mindre förväntade fel. Ett program ska inte vara beroende av **assert**, utan fungera minst lika bra utan. I allmänhet går ett program långsammare med **assert** eftersom det är mer kod att exekvera, men man ska inte dra sig för att lägga till **assert** i sin kod. Om snabbhet är viktigt kan man starta sitt program med `python -O kod.py` istället för `python kod.py`; flaggan `-O` deaktiverar alla **assert**-instruktioner.

12.1.3.1 Exempel 1: remove_trailing_spaces

Nedan följer ett exempel på **assert** i `remove_trailing_spaces`:

```
def remove_trailing_spaces(s, spaces=" \t"):
    assert len(spaces) > 0
    i = len(s)
    while i > 0 and s[i-1] in spaces:
        i -= 1
    return s[0:i]
```

12.1.3.2 Exempel 2: newton_raphson

Nedan följer ett exempel på **assert** i en funktion för att beräkna roten av en funktion med hjälp av Newton-Raphsons algoritm:

```
def newton_raphson(f, f_prime, x, precision):
    assert isinstance(x, float)
    assert isinstance(f(x), float)
    assert isinstance(f_prime(x), float)
    assert 0.0 < precision < 1.0, 'precision must be in the interval (0,1)'

    ready = False
    while not ready:
        x_next = x - f(x)/f_prime(x)      # Main line of code to be repeated
        ready = abs(x-x_next) < precision # Test for stopping criterion
        x = x_next                        # Preparing the next iteration

    return x_next
```

Som du ser kan vi testa typen för ett objekt med hjälp av **isinstance**. Funktionen **type**, som returnerar objektets typ, är också användbar, men **isinstance** är mer generell; **isinstance** kan som andra argument ta en tupel av klasser och returnerar om det första argumentet är en instans av *någon* av klasserna *eller* dess subklasser.

Observera även att **assert** kan ses som extra dokumentation kring de antagande som görs i programmet.

12.1.4 Villkor och invarianter

Vi kan dela upp **assert**-användandet i tre sorter:

- **förvillkor**: uttryck som ska vara sanna när en funktionskropp exekveras (eng. *preconditions*)
- **eftervillkor**: uttryck som ska vara sanna när en funktion returnerar (eng. *postconditions*)
- **loop invarianter**: uttryck som ska vara sanna i en loop (eng. *loop invariants*)

12.1.4.1 Exempel 3

Låt oss säga att vi har en databas som kopplar unika identifierare (representerade som `int`) med namn (representerade som `str`). För att det ska gå snabbt att slå upp någon genom antingen dess identifierare eller namn har vi två uppslagstabeller, en från identifierare till namn och en från namn till identifierare. För att detta ska fungera måste ju nycklarna i den ena vara samma som värdena i den andra, och vice versa.

```
class MyDB:

    def __init__(self):
        self._id2name_map = {}
        self._name2id_map = {}

    def add(self, id, name):
        self._name2id_map[name] = id
        self._id2name_map[id] = name

    def by_name(self, name):
        id = self._name2id_map[name]
        return id
```

En första sak som kan bli fel här är ju att användaren helt enkelt skriver över en av `_id2name_map` och `_name2id_map`, eller att någon annan metod vi senare lägger till har en bugg som bryter mot invarianten för klassen. Vi kan lägga in en **assert** i `by_name` där vi dubbelkollar att invarianten ej har blivit bruten:

```
class MyDB:

    def __init__(self):
        self._id2name_map = {}
        self._name2id_map = {}

    def add(self, id, name):
        self._name2id_map[name] = id
        self._id2name_map[id] = name

    def by_name(self, name):
        id = self._name2id_map[name]
        assert self._id2name_map[id] == name
        return id
```

Om vi nu lägger in en buggig metod eller en användare använder klassen på fel sätt så kommer `by_name` generera ett **AssertionError**. En annan invariant är ju att ett `id` ska vara en `int` och namn `str`. I många programmeringsspråk med striktare typsystem kan vi specificera detta i signaturen för klassen eller

konstruktorn. I Python kan vi lösa detta genom att lägga in en **assert** tillsammans med `isinstance`:

```
class MyDB:

    def __init__(self):
        self._id2name_map = {}
        self._name2id_map = {}

    def add(self, id, name):
        assert isinstance(id, int), ("id is not an integer: " + str(id))
        assert isinstance(name, str), ("name is not a string: " + str(name))
        self._name2id_map[name] = id
        self._id2name_map[id] = name

    def by_name(self, name):
        id = self._name2id_map[name]
        assert self._id2name_map[id] == name
        return id

db = MyDB()
db.add(2,2)
db.add("hej", "du")
```

12.2 Testning

Det är svårt att garantera att ens kod är korrekt. Man kan resonera matematiskt om kod och algoritmer, men även då är det lätt att det blir fel. Oavsett hur mycket man planerar, tänker och granskar, så är det bra att också noggrant testa sin kod. Men hur gör man det på bästa sätt?

Erfarenheten visar att det varken är tillräckligt eller praktiskt att testa om ett helt program, själva *produkten*, fungerar som det ska. Om man vill upptäcka problem tidigt och skydda sig mot fel på grund av ändringar i koden, då bör man testa programmets funktioner och metoder.

Ett vanligt arbetssätt är att (1) skriva lite kod, kanske en funktion, (2) verifiera att koden fungerar, och sen fortsätta skriva mer kod. Det kan vara enkelt att verifiera koden: man väljer några parametrar till sin funktion och ser om resultatet blir som det var tänkt. Ett scenario som i praktiken är ännu vanligare är följande:

1. Ändra i befintlig kod, för att lägga till ny funktionalitet.
2. Verifiera den nya funktionaliteten.
3. Verifiera den gamla funktionaliteten.

Punkt 3 blir lätt bortglömd, men den kan också vara lite jobbig att ta hand om. Det är ju på den nya funktionaliteten man har sitt fokus, inte den gamla, och hur var det nu vi testade den koden? Om du ändrar i en funktion `fun`, kommer du då också ihåg att testa andra delar av din kod som är beroende av `fun`? Allteftersom tiden går och ett projekt växer blir det allt svårare verifiera gammal funktionalitet. Och om man jobbar med andras kod blir det förstås ännu svårare att komma på hur man bör testa de funktioner som finns. Så hur gör man?

Det som behövs är *systematisk* och *automatisk* testning, vilket man åstadkommer genom att koda

testningen, och metodiken för detta kallas *enhetstestning* (eng. *unit testing*). Idag finns det så pass bra stöd för testning i vanliga programmeringsspråk att det ofta är enkelt att inkludera tester i sitt projekt. I det här avsnittet tittar vi på några olika sätt att arbeta med tester.

Ni kan läsa mer på: <https://realpython.com/python-testing/>

12.2.1 Testning med assert

Kodtestning kan göras både manuellt och automatiskt. För enkel manuell testning kan vi använda **assert**.

Låt oss betrakta en liten Python-modul (nedan) för att jobba med polynom. I modulen nedan representeras polynom som en lista, där listan håller värdet på koefficienten till potensen, där index i i listan motsvarar x^i . Till exempel:

Polynom	Python-representation
x^4	[0, 0, 0, 0, 1]
$4x^2 + 5x^3$	[0, 0, 4, 5]
$5 + 4x + 3x^2 + 2x^3 + x^4$	[5, 4, 3, 2, 1]

Modulen nedan innehåller tre funktioner för att utföra beräkningar på polynom. Vi kan beräkna värdet av ett polynom för ett visst värde på x , samt addera och subtrahera polynomen.

```
# My module for handling polynomials

def eval_poly(p_list, x):
    """Evaluates a polynomial at a given value x.
    """
    degree = 0
    val = 0
    for coeff in p_list:
        val += coeff * x**degree
        degree += 1
    return val

def add_poly(p0, p1):
    """Adds polynomials
    """
    poly_add = []
    p0_cpy = p0.copy()
    p1_cpy = p1.copy()
    while p0_cpy and p1_cpy:
        poly_add.append(p0_cpy.pop(0) + p1_cpy.pop(0))
    if p0_cpy:
        poly_add += p0_cpy
    elif p1_cpy:
        poly_add += p1_cpy
    return poly_add
```



```
def sub_poly(p0, p1):
    """Subtracts polynomials
    """
    poly_sub = []
    p0_cpy = p0.copy()
    p1_cpy = p1.copy()
    while p0_cpy and p1_cpy:
        poly_sub.append(p0_cpy.pop(0) - p1_cpy.pop(0))
    if p0_cpy:
        poly_sub += p0_cpy
    elif p1_cpy:
        poly_sub += [-x for x in p1_cpy]
    return poly_sub
```

Med vår polynom-modul kan vi till exempel utföra följande beräkningar:

```
p = [2,0,1]
q = [-2,1,0,0,1]
y1 = add_poly(p, q)
y2 = sub_poly(p, q)
y3 = eval_poly(p, 2)
print(y1, y2, y3, sep = '\n')
```

```
[0, 1, 1, 0, 1]
[4, 0, 0, 0, 1]
6
```

I framtiden kanske vi vill lägga till funktioner för att derivera eller multiplicera polynom, eller beräkna värdet av derivatan i en viss punkt med mera. Vissa av dessa kommande funktioner kan komma att använda sig av (d.v.s. bero på) de tidigare funktionerna vi implementerat. Det är även vanligt att vi inser att vissa funktioner kan behöva ändras, generaliseras eller förenklas för att göra modulen enkel att underhålla.

Till exempel, i modulen ovan verkar `add_poly` och `sub_poly` rätt krångliga. Vi noterar att subtraktion av ett polynom q från ett polynom p är detsamma som att addera negationen $-q$ till p , dvs $p - q = p + (-q)$. Vi inser att implementeringen av negering är enkel, och kan därför förenkla (nästan helt ta bort) koden i `sub_poly`. Inte bara har vi tagit bort en komplex funktion, utan vi har utökat vår modul till att kunna negera polynom. Den nya funktionen `neg_poly` och den omskrivna funktionen `sub_poly` ser vi nedan.

```
def neg_poly(p_list):
    """Negates a given polynomial.
    """
    p_neg = []
    for coeff in p_list:
        p_neg.append(-coeff)
    return p_neg

def sub_poly(p0, p1):
    """Subtracts polynomials
    """
    return add_poly(p0, neg_poly(p1))
```

I detta exempel är poängen att funktioner och klasser i en modul kan komma att ändras och utökas. Även om vi i detta lilla exempel har full koll på de fyra funktionerna i vår polynom-modul så kan stora moduler med hundratals funktioner och klasser där vissa interagerar (t.ex. `sub_poly` och `add_poly` ovan) snabbt kan bli oöverskådliga. Det är därför bra att redan från början tänka på att skapa tester för funktionerna för att se att de bibehåller förväntad funktionalitet. Till exempel kan vi skapa en tillhörande test-fil som vi döper till `test_polynomials.py`, där vi skriver följande kod.

```
import polynomials as pol # Import our polynomials.py module

print("Run tests for polynomial module")
p = [2,0,1]
q = [-2,1,0,0,1]
r = [] # always good to check corner cases

assert (pol.eval_poly(p,0) == 2)
assert (pol.eval_poly(p,1) == 3)
assert (pol.eval_poly(q,2) == 16)
assert (pol.eval_poly(q,-2) == 12)
assert (pol.eval_poly(r,1) == 0)
assert (pol.add_poly(p,r) == p)
assert (pol.add_poly(p,q) == pol.add_poly(q,p))
assert (pol.neg_poly(pol.neg_poly(q)) == q)
assert (all(x == 0 for x in pol.sub_poly(p,p)))
assert (pol.sub_poly(p,q) == [4, -1, 1, 0, -1])
assert (pol.eval_poly(add_poly(p,q),12) == pol.eval_poly(p,12) + pol.eval_poly(q,12))
print("Tests completed successfully.")
```

Kör vi vårt program `test_polynomials.py` kommer tester att exekveras upp till första `assert` som inte går igenom. Om vi modifierar våra funktioner hoppas vi på att någon av testerna fångar upp ifall vi råkat introducera någon bugg. Det finns ingen garanti att vi kommer fånga upp alla möjliga buggar med våra tester, men med väl designade tester kan vi hoppas fånga de flesta.

Ett problem är dock att vi endast ser resultatet av det första `assert` som inte håller. Ett bättre sätt för testning av mer komplexa program är *enhetstester*.

12.2.2 Enhetstester med unittest

Modulen `unittest` har funnits med i Pythons standardbibliotek sedan version 2.1 och används av många projekt, både kommersiella och open-source. För att använda `unittest` måste vi:

1. Gruppera alla tester i klasser där testerna är metoder.
2. Använda specifika `assert`-metoder från `unittest.TestCase` istället för de inbyggda `assert` uttrycken.

För att skriva om exemplet `test_polynomials.py` ovan så att det använder `unittest` gör vi följande:

- Importera `unittest`
- Skapa en klass `TestPoly` som ärver från `TestCase` klassen
- Skriv om alla `assert` till metoder i den här klassen som använder `self.assertEqual()`
- Ändra så att `unittest.main()` körs när filen laddas

Så här kan filen `test_polynomials.py` se ut:

```
import unittest
import polynomials as pol

p = [2,0,1]
q = [-2,1,0,0,1]
r = [] # always good to check corner cases

class TestPoly(unittest.TestCase):

    def test_eval_poly_p0(self):
        self.assertEqual(pol.eval_poly(p,0), 2, "Should be 2")

    def test_eval_poly_p1(self):
        self.assertEqual(pol.eval_poly(p,1), 3, "Should be 3")

    # ...

    def test_add_poly_p_r(self):
        self.assertEqual(pol.add_poly(p,r), p, "Adding empty list should not modify p")

    def test_add_poly_symmetric(self):
        self.assertEqual(pol.add_poly(p,q), pol.add_poly(q,p), "Function is not symmetric")

    def test_eq_poly_q0(self):
        self.assertEqual(pol.sub_poly(p,p), [0 for x in range(len(p))], "Not all 0")

    # ...

if __name__ == '__main__':
    unittest.main()
```

Och om vi kör den får vi:

```
.....
-----
Ran 5 tests in 0.001s
OK
```

Om vi har en bugg, som exempelvis att `eval_poly` alltid råkar returnera 0, blir resultatet:

```
.....FFFF
=====
FAIL: test_eval_poly_p0 (__main__.TestPoly)
-----

Traceback (most recent call last):
  File "test_polynomials.py", line 19, in test_eval_poly_p0
    self.assertEqual(eval_poly(p,0), 2, "Should be 2")
AssertionError: 0 != 2
```

```

+ 0 : Should be 2

=====
FAIL: test_eval_poly_p1 (__main__.TestPoly)
-----
Traceback (most recent call last):
  File "test_polynomials.py", line 22, in test_eval_poly_p1
    self.assertEqual(eval_poly(p,1), 3, "Should be 3")
AssertionError: 0 != 3
+ 0 : Should be 3

-----

Ran 5 tests in 0.002s

FAILED (failures=2)

```

Vi ser alltså att 2 av testerna felar och vi kan nu börja debugga vår kod. Genom att skriva bra tester som vi kör varje gång vi gör några ändringar kan vi alltså vara säkra på att vår kod fortsätter fungera som vi förväntar oss trots att vi förändrar den. Detta är väldigt användbart i större projekt där man är många som samarbetar. Många system för att arbeta på kod tillsammans (exempelvis [Github](#)) har stöd för denna typ av automatisk testning för varje ändring som någon vill göra till kodbasen. Denna utvecklingsteknik kallas för "kontinuerlig integration" (eng. *continuous integration*).

12.2.3 Slumptestning (à la QuickCheck):

Det är svårt att skriva bra tester för hand. Ett alternativ som kan vara väldigt smidigt är att använda ett bibliotek som genererar tester åt en automatiskt, dvs genom så kallad "slumptestning". För denna typ av testning skriver man en specifikation, t.ex. `add_poly(p,q) == add_poly(q,p)`, och biblioteket genererar sedan slumpmässig indata (dvs, polynom i detta fall) för att testa så att specifikationen håller.

Python har stöd för detta genom [hypothesis](#) biblioteket.

12.3 Uppgifter

1. Jaccard-index (J) är ett mått på hur lika två mängder A och B är och är definierat som

$$J := \frac{|A \cap B|}{|A \cup B|}$$

där $\cap$ och $\cup$ är *snittet* och *unionen* av mängderna A och B (se [Wikipedia](#)). Implementera denna formel som funktionen `jaccard_index(l1,l2)` i Python där l1 och l2 är två listor med element.

Tips: Konvertera listorna till mängder och använd deras metoder. Till exempel ska funktionen kunna användas så här

```

print(jaccard_index(['dog', 'cat', 'mouse'], ['dog', 'cat', 'mouse']))
print(jaccard_index(['dog', 'cat'], ['dog', 'cat', 'mouse']))
print(jaccard_index(['dog', 'cat'], ['mouse', 'rat']))

```

```
1.0
0.6666666666666666
0.0
```

2. Använd modulen `unittest` för att skriva enhetstester för din funktion i uppgift 1.
3. En begränsning med Jaccard-index är att det inte tar hänsyn till repeterade element. Till exempel resulterar koden nedan i samma Jaccard-index som ovan, även fast vi kanske skulle beskriva dessa två *multimängder* (dvs samma element kan förekomma flera gånger) som “mer olika”.

```
print(jaccard_index(['dog', 'dog', 'dog', 'cat', 'mouse'],
                    ['dog', 'cat', 'cat', 'cat', 'cat', 'mouse']))
print(jaccard_index(['dog', 'cat', 'cat', 'cat'],
                    ['dog', 'cat', 'mouse', 'mouse']))
```

```
1.0
0.6666666666666666
```

Implementera kod för att beräkna viktat Jaccard-index. Låt A och B vara två *multimängder* och A_z och B_z beteckna hur många gånger elementet z förekom i A respektive B . Då definierar vi viktat Jaccard-index J_w som

$$J_w := \frac{\sum_{z \in (A \cup B)} \min(A_z, B_z)}{\sum_{z \in (A \cup B)} \max(A_z, B_z)}$$

Till exempel:

```
print(weighted_jaccard_index(['dog', 'dog', 'dog', 'cat', 'mouse'],
                             ['dog', 'cat', 'cat', 'cat', 'cat', 'mouse']))
print(weighted_jaccard_index(['dog', 'cat', 'cat', 'cat'],
                             ['dog', 'cat', 'mouse', 'mouse']))
```

```
0.375
0.3333333333333333
```

När du implementerar detta, tänk på tips om struktur och kod från delkapitlet “Bra kod”. Till exempel: separera kod i små funktioner som gör en sak, ha inte för långa uttryck på samma rad. Tänk på hur koden bör struktureras.

4. Använd modulen `unittest` för att skriva enhetstester för dina funktioner i uppgift 3.

Bilaga A

Exempellösningar

A.1 Kapitel 1

1. x och y byter värde
2. T.ex. $(x - y - y) ** 2$
3. $s * 16$
4. T.ex.:

```
s = 'hej'
s2 = s+s
s4 = s2+s2
s8 = s4+s4
s16= s8+s8
print(s16)
```

5. T.ex.:

```
name = input("Vad heter du i förnamn?")
surname = input("Vad heter du i efternamn?")
print("Hej " + name + " " + surname)
```

6. Talen konkateneras då de är strängar. För att addera talen, konvertera strängarna till heltal:

```
age = int(input("Ålder? "))
num = int(input("Favorittal? "))
print(age + num)
```

7. Svaret blir 46.666666666666664 och numeriskt sett är det konstigt, ja till och med fel, att sista decimalen är 4. Anledningen är att det korrekta svaret inte kan representeras korrekt i typen float och det här är det närmsta talet vi kan få med den precision som 32-bitars flyttal ger.

A.2 Kapitel 2

1. b, c

2. Tex.:

```
num = int(input("Enter an integer: "))
if num % 2:
    print("Odd")
else:
    print("Even")
```

3.

x	y	x and y	x or y
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

4. Ett exempel:

```
x = True
y = False
z = False
```

ger

```
True
False
True
False
```

5. Tex.:

```
def division(x, y):
    if x == 0 and y == 0:
        print("Indeterminate form...")
    elif y == 0:
        print("The result is infinite...")
    else:
        return x/y
```

6. Alla utom det första:

```
False
True
True
True
```

7. Tex.:

```
def nand(x, y):
    return not (x and y)
```

```
def nor(x, y):  
    return (not x) and (not y)  
  
def xnor(x, y):  
    return ((not x and not y) or (x and y))  
  
# alternativ xnor  
def xnor(x, y):  
    return (x==y)
```

8. Om funktionerna är definierade som ovan, så ger t.ex.

```
xnor(47, 33)
```

```
33
```

vilket kanske man inte förväntar sig. Eftersom både 47 och 33 är sanna kanske man kan vänta sig att svaret ska bli True, men det stämmer inte pga att argumenten inte är av typ bool.

A.3 Kapitel 3

1. Listan [1, 2, 'a', 'b'].
2. Otillåtet att jämföra tex strängar och flyttal eller heltal

```
TypeError: '<' not supported between instances of 'str' and 'int'
```

3. Tex:

```
def vowels(s):  
    vs = "AEIOUYaeiouy"  
    out = ""  
    for v in s:  
        if v in vs:  
            out += v  
    return out
```

4. Ändra först testet i if satsen till att använda not in. Sen kan ni skriva:

```
def vowels_or_consonants(s, save='v'):  
    vs = "AEIOUYaeiouy"  
    out = ""  
  
    if save != 'c' and save != 'v':  
        print("illegal argument to save")  
        # Here we should raise an error, more about this later  
        return None  
  
    for v in s:  
        if save == 'v' and v in vs:  
            out += v  
        elif save == 'c' and v not in vs:  
            out += v  
  
    return out
```

5. Om vi inte har blocket

```
if term != '':  
    out.append(term)
```

så läggs inte sista ordet in om strängen inte slutar med mellanslag. Om vi inte har `if term != ''` utan låter `out.append(term)` vara på indenteringsnivå 1 så läggs en tom sträng till i slutet om strängen slutar på mellanslag. Notera även att `split` endast hanterar ett mellanslag, så det blir även fel om vi har flera mellanslag i rad. Vi kan fixa till det genom att även testa om `term` är tom i loopen:

```
def split(s):  
    out = []  
    term = ""  
  
    for c in s:
```

```

    if c == " ":
        # do not add term if empty
        if term != '':
            out.append(term)
            term = ''
    else:
        term += c

# add the final word unless it is empty
if term != '':
    out.append(term)

return out

```

6. Problemet är att vi inte använt `copy()` på `list` när vi skapar `list2`. Gör man inte det kommer `list` och `list2` bara vara olika namn på samma lista, så ändrar man en ändras den andra. Testa följande istället:

```

list = [1,2,3]
list2 = list.copy()
list2.reverse()
print(list)

```

7. Alternativ 1

```

mylist = ["Ho", "ho", "ho!"]
m = mylist.copy()
while m:
    x = m.pop()
    print(x)

print(mylist)

```

Alternativ 2

```

mylist = ["Ho", "ho", "ho!"]
n = len(mylist)
while n > 0:
    print(mylist[n-1])
    n -= 1

print(mylist)

```

Alternativ 3

```

mylist = ["Ho", "ho", "ho!"]
for i in mylist[::-1]:
    print(i)

print(mylist)

```

8. Om **break**:

```
10
9
```

Om **continue** skrivs allt utom 7 ut. Om **pass** så skrivs alla siffror ut.

9. Till exempel:

```
mylist = []

while True:
    ans = int(input("Enter a single digit (quit with 0): "))
    # if 0 is *not* to be included in the list.
    if ans == 0:
        break
    else:
        mylist.append(ans)

print(mylist)
```

Ett annat alternativ är att använda sig av `:=` operatoren, som är ett relativt nytt tillägg i Python (version ≥ 3.8 , så använd denna operator med försiktighet). `:=` operatoren utvärderar *och* tilldelar i en villkorssats, så vi kan undvika en rad i kodblocket på villkorssatsen:

```
mylist = []

while ans := int(input("Enter a single digit (quit with 0): ")):
    mylist.append(ans)

print(mylist)
```

Här tilldelas `ans` först det inlästa värdet och *efter* detta utvärderas villkorssatsen. Eftersom `0` utvärderas till falskt (`bool(0) == False`), så kommer **while**-loopen att avslutas när användaren skriver in `0`.

10. Till exempel:

```
def naiveGCD(a,b):
    for i in range(min(a,b), 0, -1):
        if a % i == 0 and b % i == 0:
            return i

a = 12
b = 3
print(naiveGCD(a,b))
```

eller med

```
def naiveGCD(a,b):
    i = min(a,b)
    while i > 0:
        if a % i == 0 and b % i == 0:
            return i
        i -= 1
```

11. Till exempel:

```
def portkod(pw = '1337'):
    curr = ""

    while True:
        n = input("Enter digit: ")

        curr += n # add new digit

        # make sure our list is only 4 digits long
        if len(curr) > 4:
            curr = curr[1:]

        print(curr) # print what the user has entered

    if curr == pw:
        print("Door unlocked!")
        return True

portkod()
```

12. Uttrycket `lst[1:1]` ger dig listan som börjar på position 1, men utan att inbegripa position 1, vilket kan tolkas som mellanrummet mellan position 0 och 1. När denna *slice* tilldelas ett värde så sätts det då in i mellanrummet.

A.4 Kapitel 4

1. Till exempel

```
a = ['a', 'b', 'c']
b = [1, 2, 3]

d = {}
for i in range(len(a)):
    d[a[i]] = b[i]
```

2. Till exempel:

```
def naiveGCD(a,b):
    for i in range(min(a,b), 0, -1):
        if a % i == 0 and b % i == 0:
            return i

print("x y GCD")
for x in range(2, 21):
    for y in range(2, 21):
        gcd = naiveGCD(x, y)
        if x != y:
            print(x, y, gcd)
```

3. Det finns många sätt att göra det på! Om du är osäker på om din lösning så kan du testa på någon i din omgivning. Går det att följa ditt detaljerade recept?
4. Några möjliga kandidater är:
- mix_ingredients, som utför punkt 1.
 - finalize_mixing, som utför punkt 2.
 - make_pancakes, som utför gräddningen, punkt 3 och 4.
5. Kommer senare.
6. Ändra `for y in range(2, 21):` till `for y in range(x, 21):`.
7. Till exempel:

```
def vowels(s):
    vs = "AEIOUYaeiouy"
    d = {vs[i] : i for i in range(len(vs))}
    out = ""

    for v in s:
        if v in d:
            out += str(d[v])
        else:
            out += v
    return out

print(vowels("Hej"))
```

8. Till exempel:

```
def annotate_word(s):
    out = {}
    term = ""
    cnt = 1
    for c in s:
        if c == " ":
            out[term] = cnt
            term = ""
            cnt += 1
        else:
            term += c

    # add the final word unless it is empty
    if term != "":
        out[term] = cnt
    return out
```

9. Alternativ 1: Skriv `out[cnt] = term` istället för `out[term] = cnt`.

Alternativ 2: Lägg till nedan rad i slutet på funktionen.

```
out_reversed = { v : k for k, v in out.items() }
```

10. Ja, vi kan ha en lista som värde.

```
def annotate_word(s):
    out = {}
    term = ""
    cnt = 1
    for c in s:
        if c == " ":
            if term in out:
                out[term].append(cnt)
            else:
                out[term] = [cnt]
            term = ""
            cnt += 1
        else:
            term += c

    # add the final word unless it is empty
    if term != "":
        if term in out:
            out[term].append(cnt)
        else:
            out[term] = [cnt]
    return out

print(annotate_word("hej du hej hej du"))
```

```
{'hej': [1, 3, 4], 'du': [2, 5]}
```

A.5 Kapitel 5

1. Vi får ett `TypeError`. Vi kan fänga det genom

```
def divide_by_elems(filename,x):
    try:
        x = int(x)
    except TypeError:
        print("divide_by_elems:",
              "Second argument cannot be interpreted as an integer.")
        return None

    # ... rest of function here
```

2. Till exempel:

```
def divide_by_elems(filename,x):
    quotients = []
    with open(filename, 'r') as h:
        for n in h:
            try:
                frac = x / int(n)
                quotients.append(frac)
            except ZeroDivisionError:
                print("divide_by_elems: Division by zero.")
            except ValueError:
                print("At least one line contains characters that",
                      " cannot be converted with int().")

    return quotients

print(divide_by_elems('numbers.txt', 2))
```

3. Till exempel:

```
ys = [i for i in xs if i > 0]
```

4. Till exempel:

```
ys = [i if i > 0 else -i for i in xs]
```

eller

```
[abs(i) for i in xs]
```

5. Till exempel:

```
[p for p in range(2,100) if is_prime(p)]
```

6. Till exempel:

```
[x1*x2 for x1 in xs for x2 in xs ]
```

7. Till exempel:


```
[xs[i]*xs[j] for i in range(len(xs)) for j in range(len(xs)) if i != j]
```

8. *Utelämnad då vi inte vill ge ut facit till labb 1.*

A.6 Kapitel 6

1. Till exempel om det inte är tänkt att vi ska ändra på informationen i en datastruktur. Då kan vi spara informationen i en tuple. Python kommer då att meddela oss när programmet försökt ändra något istället får att tyst utföra ändringen som hade gjorts med en lista. Detta är väldigt viktigt när vi börjar använda andra personers kod och inte har full insikt i hur deras funktioner och moduler fungerar.
2. Till exempel:

```
def is_palindrome(s):
    backward_s = s[::-1]
    if s == backward_s:
        return True
    else:
        return backward_s
```

3. Ja, notationen `s[::-1]` fungerar för sekvenstyper i python.
4. Till exempel:

```
def occurrences(s):
    """Returns number of occurrences of letters in string.
    """
    s = s.lower()
    d = {}
    for c in s:
        if not c.isalpha():
            c = 'non_alphas'
        if c not in d:
            d[c] = 0
        d[c] += 1
    return d
```

5. Funktionen `occurrences` fungerar ju bra på en sträng, så om vi läser in hela filen till *en sträng*, kan vi direkt använda oss av funktionen. Man läser hela filen med `.read`. Detta inkluderar tecknen för ny rad (`\n`), så vill man inte ha med dessa i uträkningen, så kan man ta bort de med strängfunktionen `replace`.

```
with open('palindrome.txt', 'r') as fp:
    # reads contents of entire file in one go.
    # Note: memory consuming if a large file.
    file_content = fp.read()

    # if newline characters, \n, are to be counted
    chars_with_newline = occurrences(file_content)

    # if newline characters are to be skipped, modify the string by
    # replacing the '\n' character with the empty string ''
    file_content_no_newline = file_content.replace('\n', '')
    chars_without_newline = occurrences(file_content_no_newline)
```

```
print(chars_with_newline)
print(chars_without_newline)
```

6. Till exempel:

```
def fibonacci(stop=55):
    i = 0
    yield i
    j = 1
    yield j
    while i + j < stop:
        yield i + j
        i, j = j, i + j
```

7. Till exempel:

```
def map_gen(f, it):
    """
    Applies a function to the elements of an iterable and returns a
    generator.
    """
    return (f(i) for i in it)
```

8. Till exempel:

```
def is_string_palindrome(s):
    """
    Checks if string is a palindrome
    """
    return s == s[::-1]

def palindrome_rows(filename):
    """
    Checks if each row of file is a palindrome string
    """
    return (is_string_palindrome(row.strip('\n'))
            for row in open(filename, 'r'))

# OR

def palindrome_rows(filename):
    with open(filename, 'r') as fp:
        for row in fp:
            yield is_string_palindrome(row.strip('\n'))
```

9. Till exempel:

```
with open('almost_lowercase.txt') as f:
    file_content = f.read()
    capitalized_word = file_content.lstrip(file_content.lower()).split()[0]
```

A.7 Kapitel 7

1. Till exempel:

```
import random

random_integers = [random.randint(1, 100) for i in range(10)]

random_floats = [random.uniform(1, 100) for i in range(10)]
```

2. Till exempel:

```
def is_perfect(x):
    sum = 0
    for i in range(1, x):
        if(x % i == 0):
            sum = sum + i
    return sum == x

def is_prime(x):
    if x >= 2:
        for y in range(2,x):
            if not ( x % y ):
                return False
    else:
        return False
    return True

def main(x):
    print("The number is perfect:", is_perfect(x))
    print("The number is prime:", is_prime(x))

x = int(input("Provide a number to analyze: \n"))
main(x)
```

3. Till exempel:

```
choice = input(
    "Type 'G' to manually enter number or 'R'"
    "to randomly generate number: \n")
if choice == "G":
    x = int(input("Provide a number to analyze: \n"))
elif choice == "R":
    x = random.randint(1, 100)
else:
    print("You failed")
```

Vår nya kod bör ligga nedanför funktionerna `is_perfect(x)` and `is_prime(x)`. Vi kan ha det i `main` eller utanför. Det beror lite på hur vi vill kunna använda `main`. Ska den ha med denna ny funktionalitet eller ej? Vanligtvis ska inte funktioner som man vill importera innehålla interaktiva delar (kommunikation med användare) i koden.

4. Vi delar upp koden så att varje funktion tar och returnerar parameterar. Vi har tagit bort alla globalt definierade variabler (seq, k, substrings) och vi kan återanvända varje enskild funktion i andra program.

```
# Given a sequence as input find distances
# between the copies of the most frequent substring

def get_substrings(seq, k):
    substrings = {}
    for i in range(len(seq) - k + 1):
        substring = seq[i:i+k]
        if substring in substrings:
            substrings[substring] += 1
        else:
            substrings[substring] = 1
    return substrings

def find_most_freq_substring(substrings):
    max_count = 0
    most_freq_substring = ""
    for substring, count in substrings.items():
        if count > max_count:
            most_freq_substring = substring
            max_count = count
    return most_freq_substring

def find_positions(query_substring, seq):
    positions = []
    k = len(query_substring)
    for i in range(len(seq) - k + 1):
        substring = seq[i:i+k]
        if substring == query_substring:
            positions.append(i)
    return positions

def get_distances(positions):
    distances = []
    for p1, p2 in zip(positions[:-1], positions[1:]):
        distances.append(p2 - p1)
    return distances

## Driver code
def main():
    seq = input("Enter a dna string: ")
    k = 3
    substrings = get_substrings(seq, k)
    max_substring = find_most_freq_substring(substrings)
    positions = find_positions(max_substring, seq)
    distances = get_distances(positions)
```

```
print(distances)
```

5. Till exempel:

```
import itertools
numbers = [13, 24, 42, 66, 78]
string_numbers = [str(i) for i in numbers]
# permutations
perm = itertools.permutations(string_numbers, r=2)
results = [int(''.join(i)) for i in perm]
```

6. Utelämnad då vi inte vill ge ut facit till labb 1.

A.8 Kapitel 8

1. Till exempel:

```
def rec_prod_m(n,m):  
    if n <= 0:  
        return 1  
    else:  
        return n * rec_prod_m(n-m, m)
```

2. `prod([])` kommer att returnera 0 vilket gör att alla andra funktionsanrop på rad 4 i funktionen kommer att vara multiplicerade med 0. Till exempel, när listan har ett element kvar (säg `xs = [23]`) kommer `return` på rad 4 i funktionen att ha värdena `x = 23` och `xs = []`. Detta ger

```
return (23 * prod([])) # ==> (23 * 0)
```

3. Till exempel:

```
f1 = lambda x, y: x + y  
f2 = lambda x, y: x - y  
f3 = lambda x, y: x * y  
f4 = lambda x, y: x / y  
f5 = lambda x, y: x ** y  
  
print((6*(2+3)/5)**2)  
print(f5(f3(f4(f1(2,3),5),6),2))  
  
print((10 - 2**3)*5)  
print(f3(f2(10,f5(2,3)),5))
```

4. Till exempel:

```
def get_evens(xs):  
    return list(map(lambda x: x % 2 == 0, xs))  
  
l = [1,4,2,3,4,5]  
print(get_evens(l))
```

5. 7 funktionsanrop görs totalt för 3 (inklusive anropet `f(3)`) och 13 anrop med 4. Till exempel för `f(3)`

```
f(3) = f(2) + f(1) + f(0)  
      = (f(1) + f(0) + f(-1)) + 1 + 1  
      = 1 + 1 + 1 + 1 + 1  
      = 5
```

och för `f(4)`:

```
f(4) = f(3) + f(2) + f(1)
```

Vi vet att `f(3)` blir 7 anrop, `f(2)` blir 4, och `f(1)` blir 1, så totalt får vi $1 + 7 + 4 + 1 = 13$ anrop.

6. Till exempel någon av följande funktioner:


```

def collatz_rec(n):
    if n == 1:
        return []
    if n % 2 == 0:
        return [n] + collatz_rec(n//2)
    else:
        return [n] + collatz_rec(3*n+1)

def collatz_rec2(l):
    if l[-1] == 1:
        l.pop(-1)
        return l
    if l[-1] % 2 == 0:
        l.append(l[-1]//2)
    else:
        l.append(3*l[-1]+1)
    return collatz_rec2(l)

def collatz_rec3(n):
    while n != 1:
        return [n] + collatz_rec3(n//2 if not n % 2 else 3*n+1)
    return []

def collatz_rec4(n, collatz_seq):
    if n == 1:
        return collatz_seq
    else:
        collatz_seq.append(n)
        n = n//2 if not n % 2 else 3*n+1
        return collatz_rec4(n, collatz_seq)

```

Dess funktioner fungerar på lite olika sätt men ger samma resultat. För att beräkna collatz-sekvensen för 10, så kallas funktionerna på följande sätt:

```

collatz_rec(10)
collatz_rec2([10])
collatz_rec3(10)
collatz_rec4(10,[])

```

7. Till exempel:

```

joinstrings = lambda xs: foldr(xs, lambda x,y: x + y, "")

```

8. Till exempel:

```

def fac(n):
    ret = 1
    for i in range(1, n + 1):

```

```

        ret *= i
    return ret

def fib(n):
    i, j = 0, 1
    if n == 0:
        return 0
    elif n == 1:
        return 1
    k = 1
    while k < n:
        i, j = j, i + j
        k += 1
    return j

```

9. Om xs inte kopieras kommer varje funktionsanrop med `smallereq()` (och `greater()`) att potentiellt modifiera listan genom att ta bort alla element (pga `.pop()`). Detta resulterar i att efter `smallereq` så finns bara tomma listan att ge till `greater` i ett rekursivt anrop.
10. Till exempel:

```

def pascal(n):
    if n == 1:
        return [1]
    else:
        p_line = pascal(n-1)
        line = [ p_line[i] + p_line[i+1] for i in range(len(p_line)-1)]
        line.insert(0,1)
        line.append(1)
        return line

```

A.9 Kapitel 9

1. Till exempel:

```
def __str__(self):
    return "<" + self._fname + " " + self._lname
        + ", " + self._email + ">"
```

```
def __str__(self):
    return "<Course: " + self._code + " " + self._name + ">"
```

2. Till exempel:

```
class Participant:

    # Changed!
    def __init__(self, fname, lname, email, is_teacher = False):
        self._fname = fname                # Changed!
        self._lname = lname                # Changed!
        self._email = email
        self.is_teacher = is_teacher        # Changed!

    def name(self):
        return self._fname + " " + self._lname    # Changed!

    def email(self):
        return self._email

    def __str__(self):
        if self.is_teacher:
            return "<" + self._fname + " " + self._lname
                + ", " + self._email + "(Teacher) >"
        else:
            return "<" + self._fname + " " + self._lname
                + ", " + self._email + ">"
```

3. Till exempel:

```
class IntSet:
    """An IntSet is a set of integers"""

    def __init__(self):
        """Create an empty set of integers"""
        self.vals = []

    def insert(self,e):
        """Assumes e is an integer and inserts e into self"""
        if e not in self.vals:
            self.vals.append(e)

    def member(self,e):
        """Assumes e is an integer.
```

```

        Returns True if e is in self, and False otherwise"""
    return e in self.vals

def delete(self,e):
    """Assumes e is an integer and removes e from self
    Raises ValueError if e is not in self"""
    try:
        self.vals.remove(e)
    except:
        raise ValueError(str(e) + ' not found')

def get_members(self):
    """Returns a list containing the elements of self.
    Nothing can be assumed about the order of the elements"""
    return self.vals

def __eq__(self, other_intset):
    for e in self.vals:
        if e not in other_intset.vals:
            return False
    for e in other_intset.vals:
        if e not in self.vals:
            return False
    return True

    # eller:
    # return (all([e in other_intset for e in self.vals ] ) and
    #         all([e in self.vals for e in other_intset ] ))

    # eller:
    # return sorted(self.vals) == sorted(other_intset)

def __neq__(self, other_intset):
    return not self.__eq__()

def intersection(self, other_intset):
    i_new = IntSet()
    i_new.vals = [e for e in self.vals if e in other_intset.vals]
    return i_new

def __add__(self, other_intset):
    i_new = IntSet()
    i_new.vals = ([e for e in other_intset.vals if e not in self.vals ]
                  + self.vals)
    return i_new

def __sub__(self, other_intset):
    i_new = IntSet()

```

```
i_new.vals = [e for e in self.vals if e not in other_intset.vals ]  
return i_new
```

4. Se svaret för uppgift 3.

5. Till exempel:

```
class Dog:  
  
    kind = 'canine'          # class variable shared by all instances  
  
    def __init__(self, name):  
        self.name = name  
        self.tricks = []    # creates a new empty list for each dog  
  
    def add_trick(self, trick):  
        self.tricks.append(trick)
```

6. Till exempel:

```
class Dog:  
    kind = 'canine'  
    # class variable shared by all instances  
    def __init__(self, name):  
        self.name = name  
        self.tricks = []  
        # creates a new empty list for each dog  
  
    def add_trick(self, trick):  
        self.tricks.append(trick)  
  
    def __str__(self):  
        return self.name + " knows: " + ','.join(self.tricks)  
  
    def __gt__(self, other_dog):  
        return len(self.tricks) > len(other_dog.tricks)
```

A.10 Kapitel 10

1. Till exempel:

```
from math import sqrt

class Equilateral(Triangle):

    __name__ = 'Equilateral Triangle'

    def __init__(self, sides=None):
        super().__init__(sides)

    def check_sides(self):
        a, b, c = self.sides
        equi_tri_condition = a == b == c
        if not equi_tri_condition:
            raise ValueError(
                "All sides must be same length in a " + self.__name__
            )
        super().check_sides()

    def get_area(self):
        a, b, c = self.sides
        # calculate area
        return sqrt(3) * a**2 / 4
```

2. Till exempel:

```
class Student(ProgMon):
    """A Pokemon Go Monster Student.... person.
    """
    def __init__(self, is_learning,
                  is_caffeinated=True,
                  is_unit_testing=False):
        super().__init__()
        self._attack = 0.05
        self._defense = 0.1
        self.is_learning = is_learning
        # convenience attribute to use for the multiplication of the
        # base class ProgMon's get_* functions.
        self._learning_multiplier = 2 if is_learning else 1

    def __str__(self):
        return "<Student>"

    def get_attack(self):
        return self._learning_multiplier * ProgMon.get_attack(self)
```

```
def get_defense(self):
    return self._learning_multiplier * ProgMon.get_defense(self)
```

3. Till exempel:

```
class Tournament():
    """Documentation for Tournament

    """

    # Note: either an immutable type should be used as a default argument
    # for participants (like an empty tuple) or like below, None.
    # an alternative would be to have the call signature as:
    # __init__(self, participants=())
    # and cast participants to a new list in the constructor, i.e.
    # self.participants = list(participants)
    def __init__(self, participants=None):
        if not participants:
            self.participants = []
        else:
            self.participants = participants

    def add_participant(self, participant):
        self.participants.append(participant)

    def start_tournament(self):
        """Decide who the winner of the Tournament is, and rank the rest.

        Based on the number of tricks (i.e. size of the list containing tricks)
        """
        # sort the list of participants based on how many tricks they
        # know (key is number of tricks for a Dog). Since `sorted` is
        # by default ascending, we reverse the list.
        # Sorted can make use of comparison operators such as `__lt__`, `__leq__`,
        # and more (in this case it finds the __gt__ method from class Dog())
        self.ranking = sorted(self.participants, reverse = True)
        self.winner = self.ranking[0] if self.ranking else None

    def print_results(self):
        print("Tournament Results\n" + "-"*18)
        print(self.winner.name, "is the winner!\n" +
              "Tricks:\n\t", self.winner.tricks)
        print("\nTournament runner-ups:")
        for i, d in enumerate(self.ranking[1:], start=2):
            print(str(i) + " place:", d.name)
```

Notera: i konstruktorn för Tournament så används ett standardvärde (default value) `None` som är en icke-muterbar typ. Generellt sett är det viktigt att standardvärden till funktioner och klasser är av icke-muterbar typ. Standardvärdet för en nyckelordsparameter sätts då en funktion/klass definieras, och dessa argument omdefinieras inte varje gång en funktion kallas. En muterbar typ

som standardvärde (t.ex. `participants=[]`) skulle sättas *en* gång. Antag att vi skapar ett objekt `t = Tournament()`. Varje gång `participants` (eller `self.participants` för den delen) modifieras i objektet `t`, så kommer standardvärdet att modifieras. Det här vill vi inte, eftersom om man skapar en ny turnering med `t2 = Tournament()` så kommer standardvärdet till `participants` i `t2` inte vara den tomma listan. Med ovan konstruktor (och specifikt det icke-muterbara standardvärdet) undviks detta, och en ny instans av `Tournament`s är alltid tom (om man inte specificerar `participants` som argument). Se Listor som muterbara typer i Kapitel 6.

4. Till exempel:

```
class Vehicle():
    def __init__(self, wheels=0, wings=0, sound=''):
        self.wheels = wheels
        self.wings = wings
        self.sound = sound
        self.__name__ = 'Abstract vehicle'

    def __str__(self):
        msg = 'A ' + self.__name__ + ' has:\n'
        msg_add = ''
        if self.wheels:
            msg_add += str(self.wheels) + ' wheels\n'
        if self.wings:
            msg_add += str(self.wings) + ' wings\n'
        if self.sound:
            if msg_add:
                msg_add += 'and '
            msg_add += 'makes a ' + self.sound + ' sound\n'
        return msg + msg_add

class Car(Vehicle):
    def __init__(self):
        super().__init__(wheels=4, sound='AAARRRRRRRR!')
        self.__name__ = 'Car'

class Motorcycle(Vehicle):
    def __init__(self):
        super().__init__(wheels=2, sound='VROOM VROOM!')
        self.__name__ = 'Motorcycle'

class Plane(Vehicle):
    def __init__(self):
        super().__init__(wings=2, sound='WHOOSH!')
        self.__name__ = 'Plane'
```

5. Till exempel:

```
class MyString(str):
    def __init__(self, args):
        super().__init__()
```



```
def __sub__(self, other):  
    return ''.join(self.rsplit(other, 1))  
  
def __truediv__(self, other):  
    return ''.join(self.split(other))
```

A.11 Kapitel 11

1. Samtliga är exempel på hierarkiskt arv. De två första är även exempel på mångnivå arv och alltså även hybridarv. Ingen av dem är exempel på multipelt arv.
2. Till exempel:

```
class BinOp(Expr):
    def __init__(self, left, right):
        self._left = left
        self._right = right

    def __str__(self):
        s1 = parens(self._left.prec, self.prec, str(self._left))
        s2 = parens(self._right.prec, self.prec, str(self._right))
        return s1 + self.op + s2

class Plus(BinOp):
    prec = 1
    op = " + "

class Times(BinOp):
    prec = 2
    op = " * "
```

3. Om man gjort uppgiften ovan räcker det att skriva:

```
class Exp(BinOp):
    prec = 3
    op = " ** "

print(Exp(Times(Var("x"), Var("y")), Constant(7)))
print(Times(Var("x"), Exp(Var("y"), Constant(7))))
print(Times(Var("x"), Exp(Var("y"), Plus(Constant(3), Var("z")))))

(x * y) ** 7
x * y ** 7
x * y ** (3 + z)
```

4. Till exempel:

```
# using recursion
def big_constant_sum(n,e):
    if n == 0:
        return Constant(0)
    elif n == 1:
        return e
    else:
        return Plus(e, big_constant_sum(n-1,e))

# OR
# using iteration
```

```
def big_constant_sum_it(n,e):
    if n == 0:
        return Constant(0)
    elif n == 1:
        return e
    else:
        expr_so_far = e
        for i in range(n-1):
            expr_so_far = Plus(expr_so_far,e)
        return expr_so_far
```

```
print(big_constant_sum(5,Var("x")))
# print(big_constant_sum_it(5,Var("x")))
```

```
x + x + x + x + x
```

5. Till exempel:

```
def big_sum(n,e):
    if n == 0:
        return e(0)
    else:
        return Plus(e(n),big_sum(n-1,e))
```

```
# tests
print(big_sum(5,lambda n: Var("x") if n == 1 else Var("y")))
```

```
def foo(n):
    if n == 0:
        return Constant(10)
    elif n == 1:
        return Var("x")
    elif n == 2:
        return Times(Var("z"),Var("y"))
    else:
        return Constant(n)
```

```
print(big_sum(10,foo))
```

```
y + y + y + y + x + y
10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + z * y + x + 10
```

6. De ändringar som behöver göras är:

```
class Constant(Expr):
    def __init__(self, value):
        self._value = value

    def __str__(self):
        return str(self._value)
```

```

    def evaluate(self,d):
        return self._value

class Var(Expr):
    def __init__(self, name):
        self._name = name

    def __str__(self):
        return self._name

    def evaluate(self,d):
        return d[self._name]

# BinOp code unchanged...

class Plus(BinOp):
    prec = 1
    op = " + "

    def evaluate(self,d):
        return self._left.evaluate(d) + self._right.evaluate(d)

class Times(BinOp):
    prec = 2
    op = " * "

    def evaluate(self,d):
        return self._left.evaluate(d) * self._right.evaluate(d)

class Exp(BinOp):
    prec = 3
    op = " ** "

    def evaluate(self,d):
        return self._left.evaluate(d) ** self._right.evaluate(d)

vals = { 'x' : 2, 'y' : 3, 'z' : 5 }

# x * y + 7
e1 = Plus(Times(Var("x"),Var("y")),Constant(7))
# x * (y + 7)
e2 = Times(Var("x"),Plus(Var("y"),Constant(7)))
# x * y ** (3 + z)
e3 = Times(Var("x"),Exp(Var("y"),Plus(Constant(3),Var("z"))))

print(e1.evaluate(vals))
print(e2.evaluate(vals))
print(e3.evaluate(vals))

```

```
13
20
13122
```

7. Till exempel:

```
# helper function to print lists the way we want
def to_string(xs):
    if xs == []:
        return '[]'
    elif len(xs) == 1:
        return '[' + str(xs[0]) + ']'
    else:
        out = ''
        for x in xs[:-1]:
            out += str(x) + ', '
        return '[' + out + str(xs[-1]) + ']'

class RoseTree:
    def __init__(self, v, ts):
        self._val = v
        self._subtrees = ts

    def __str__(self):
        return "(" + str(self._val) + "," +
            to_string([ str(x) for x in self._subtrees ]) + ")"

    def member(self, x):
        return (self._val == x or
            any (map(lambda v: v.member(x), self._subtrees)))

    def map_tree(self, f):
        return RoseTree(f(self._val), map(lambda v: v.map_tree(f), self._subtrees))

    def linearize(self):
        return ([self._val] +
            [ x for l in map(lambda v: v.linearize(), self._subtrees)
              for x in l ])

# tests
t = RoseTree(-32, [RoseTree(2, [])
                    , RoseTree(1, [RoseTree(23, [RoseTree(4, [])
                                                    , RoseTree(-2, [])])
                    , RoseTree(12, [])])])

print(t)
print(t.member(12))
print(t.member(-123))
print(t.map_tree(lambda x: 0))
print(t.map_tree(lambda x: x ** 2))
```

```

print(t.linearize())
(-32,[(2,[]),(1,[(23,[(4,[]),(-2,[])]),(12,[])]))
True
False
(0,[(0,[]),(0,[(0,[(0,[]),(0,[])]),(0,[])]))
(1024,[(4,[]),(1,[(529,[(16,[]),(4,[])]),(144,[])]))
[-32, 2, 1, 23, 4, -2, 12]

```

A.12 Kapitel 12

1. Till exempel:

```
def jaccard_index(l1, l2):
    s1 = set(l1)
    s2 = set(l2)
    return float(len(s1.intersection(s2)) / len(s1.union(s2)))
```

2. Tänk på att testa lite olika specialfall; tomma listor, tomt snitt, identiska mängder, fel argument. När man testar funktioner är det inte bara viktigt att se att de gör rätt, utan även returnerar fel där det är förväntat. Det beror på att när man bygger på funktioner så kan det hända att man intruducera buggar som gör att koden råkar köra utan fel för indata som det är tänkt att funktionen inte ska fungera för.

Om du har skrivit dina lösningar i en fil som heter kap12.py, skapa då en fil (ex test_kap12.py) som till exempel se ut såhär:

```
import unittest
import kap12

class TestJaccard(unittest.TestCase):
    def test_identical_jaccard_index(self):
        l1 = [2,0,1]
        l2 = [2,1,0,0,1]
        self.assertEqual(kap12.jaccard_index(l1,l2),1.0, "Should be 1.0")

    def test_disjoint_jaccard_index(self):
        l1 = [1,2,3]
        l2 = [0,0,0]
        self.assertEqual(kap12.jaccard_index(l1,l2),0, "Should be 0")

    def test_empty_jaccard_index(self):
        l1 = [1,2,3]
        l2 = []
        self.assertEqual(kap12.jaccard_index(l1,l2),0, "Should be 0")

    def test_wrong_argument_jaccard_index(self):
        # check that jaccard_index fails with TypeError when
        # argument is not iterable
        with self.assertRaises(TypeError):
            l1 = [1,2,3]
            l2 = 1
            kap12.jaccard_index(l1,l2)

    def test_empty_lists_jaccard_index(self):
        # check that jaccard_index fails with ZeroDivisionError when
        # empty sets
        with self.assertRaises(ZeroDivisionError):
            l1 = []
            l2 = []
```

```

        kap12.jaccard_index(l1,l2)

unittest.main()

```

3. Till exempel:

```

def make_weighted_operation(l1, l2, f):
    # returns total number of elements shared
    return sum([f(l1.count(item), l2.count(item)) for item in set(l1 + l2)])

def weighted_jaccard_index(l1, l2):
    # for intersection
    count_intersect = make_weighted_operation(l1, l2, min)
    # for union
    count_union = make_weighted_operation(l1, l2, max)
    return float(count_intersect) / count_union

```

Ovan kod är dock beräkningstung då funktionen måste stega igenom listan för att räkna antalet förekomster av elementet `item` för varje unikt item. För totala mängden element n , och för m unika items sker alltså $m \cdot n$ iterationer. Lösningen nedan sparar vikterna i uppslagstabell och kräver därför bara ett fåtal loopar (7 st funktionsanrop) som alla är av storlek n . Vi har under denna kod illustrerat tiden det tar för de två olika lösningarna.

```

def make_weighted_set(l):
    w_set = {}
    for element in l:
        if element not in w_set:
            w_set[element] = 1
        else:
            w_set[element] += 1
    return w_set

def weighted_jaccard_index_faster(list1, list2):

    w_set1 = make_weighted_set(list1)
    w_set2 = make_weighted_set(list2)

    key_union = set(list(w_set1.keys()) + list(w_set2.keys()))
    min_total = 0
    max_total = 0
    for key in key_union:
        # get retrieves the element if present, otherwise returns the value
        # that is specified in the second argument (in this case 0)
        min_total += min(w_set1.get(key,0),w_set2.get(key,0))
        max_total += max(w_set1.get(key,0),w_set2.get(key,0))

    return min_total/max_total

from time import time
import random

```



```

l1 = [random.randint(1, 1000) for i in range(100000)]
l2 = [random.randint(1, 1000) for i in range(100000)]

t_start = time()
print(weighted_jaccard_index(l1, l2))
print("Elapsed:", time() - t_start)

t_start = time()
print(weighted_jaccard_index_faster(l1, l2))
print("Elapsed:", time() - t_start)

```

ger

```

0.8919685933213508
Elapsed: 4.24 seconds
0.8919685933213508
Elapsed: 0.02 seconds

```

```

from time import time
import random
l1 = [random.randint(1, 10000) for i in range(1000000)]
l2 = [random.randint(1, 10000) for i in range(1000000)]

t_start = time()
print(weighted_jaccard_index(l1, l2))
print("Elapsed:", time() - t_start)

t_start = time()
print(weighted_jaccard_index_faster(l1, l2))
print("Elapsed:", time() - t_start)

```

ger

```

Elapsed: 459.42 seconds
0.8938228234057563
Elapsed: 0.28 seconds

```

4. Testfilen kan ha samma struktur som i uppgift 2, men kom ihåg att enhetstesta alla funktionerna separat. Till exempel för första lösningen ovan.

```

import unittest
import kap12

class TestWeightedJaccard(unittest.TestCase):
    def test_identical_wjaccard_index(self):
        l1 = [2, 0, 1]
        l2 = [2, 1, 0, 0, 1]
        self.assertEqual(kap12.weighted_jaccard_index(
            l1, l2), 3/5, "Should be 3/5")

    def test_disjoint_wjaccard_index(self):

```

```

    l1 = [1, 2, 3]
    l2 = [0, 0, 0]
    self.assertEqual(kap12.weighted_jaccard_index(l1,l2), 0, "Should be 0")

def test_empty_wjaccard_index(self):
    l1 = [1, 2, 3]
    l2 = []
    self.assertEqual(kap12.weighted_jaccard_index(l1,l2), 0, "Should be 0")

def test_wrong_argument_wjaccard_index(self):
    # check that jaccard_index fails with TypeError when
    # argument is not iterable
    with self.assertRaises(TypeError):
        l1 = [1, 2, 3]
        l2 = 1
        kap12.weighted_jaccard_index(l1, l2)

def test_empty_lists_wjaccard_index(self):
    # check that jaccard_index fails with TypeError when
    # argument is not iterable
    with self.assertRaises(ZeroDivisionError):
        l1 = []
        l2 = []
        kap12.weighted_jaccard_index(l1, l2)

def test_make_weighted_operation_min(self):
    l1 = [1, 2, 3]
    l2 = [0, 0, 0]
    f = lambda x, y: min(x, y)
    self.assertEqual(kap12.make_weighted_operation(
        l1, l2, f), 0, "Should be 0")

def test_make_weighted_operation_max(self):
    l1 = [1, 2, 3]
    l2 = [0, 0, 0]
    f = lambda x, y: max(x, y)
    self.assertEqual(kap12.make_weighted_operation(
        l1, l2, f), 6, "Should be 6")

def test_make_weighted_operation_empty(self):
    l1 = []
    l2 = []
    f = lambda x, y: max(x, y)
    self.assertEqual(kap12.make_weighted_operation(
        l1, l2, f), 0, "Should be 6")

unittest.main()

```